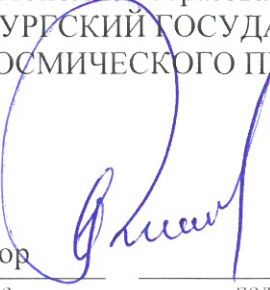


МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой № 43
д-р техн. наук, профессор

должность, уч. степень, звание

 06.06.25

подпись, дата

М.Ю. Охтилев

инициалы, фамилия

БАКАЛАВРСКАЯ РАБОТА

на тему Разработка приложения для обучения игре на гитаре с функцией генерации табулатуры

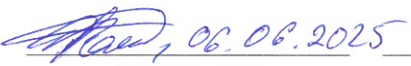
выполнена Орловым Владимиром Ивановичем
_____ фамилия, имя, отчество студента в творительном падеже

по направлению подготовки 09.03.04 Программная инженерия
_____ код _____ наименование направления

направленности 02 Проектирование программных систем
_____ код _____ наименование направленности

_____ наименование направленности

Студент группы № Z0431  06.06.2025 В.И. Орлов
_____ подпись, дата _____ инициалы, фамилия

Руководитель
доцент, канд. техн. наук,  06.06.2025 А.В. Фомин
_____ должность, уч. степень, звание _____ подпись, дата _____ инициалы, фамилия

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой №43

д-р техн. наук, профессор

должность, уч. степень, звание

 31.03.25

подпись, дата

М.Ю. Охтилев

инициалы, фамилия

ЗАДАНИЕ НА ВЫПОЛНЕНИЕ БАКАЛАВРСКОЙ РАБОТЫ

студенту группы

Z0431

номер

Орлову Владимиру Ивановичу

фамилия, имя, отчество

на тему Разработка приложения для обучения игре на гитаре с функцией генерации табулатуры

утвержденную приказом ГУАП от 28.03.2025 № 11-390/25

Цель работы: Разработка клиент-серверного мобильного приложения для обучения игре на гитаре с функцией генерации табулатуры

Задачи, подлежащие решению: Создание мобильного приложения для обучения игре на гитаре, создание REST API, авторизации пользователей, средства для просмотра и создания новых уроков и их модерирования, генерация табулатуры.

Содержание работы (основные разделы): Анализ предметной области, разработка требований к программе, разработка программы, тестирование.

Срок сдачи работы « 06 » июня 2025

Руководитель

доцент, к.т.н.

должность, уч. степень, звание

 31.03.25

подпись, дата

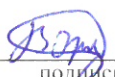
А.В. Фомин

инициалы, фамилия

Задание принял(а) к исполнению

студент группы №

Z0431

 31.03.25

подпись, дата

В.И. Орлов

инициалы, фамилия

РЕФЕРАТ

Выпускная квалификационная работа, 187 страниц, 67 рисунков, 3 таблицы, 15 источников, 3 приложения.

Ключевые слова: Генерация табулатуры, мобильное приложение, гитара, уроки, быстрое преобразование Фурье.

Актуальность: Актуальность данной темы обусловлена потребностью начинающих гитаристов в получении знаний и изучении любимых композиций без сильных финансовых вливаний.

Цель работы: Разработка мобильного клиент-серверного приложения для обучения игре на гитаре, которое должно помочь пользователям систематизировать и улучшить процесс обучения, а также предоставить инструментарий для комфортного освоения музыкального инструмента.

В процессе использовались программные средства: Android studio, IntelliJ IDEA, Spring Framework.

Примененные языки программирования: Kotlin.

Полученные результаты: Разработано приложение для обучения игре на гитаре. Оно генерирует табулатуру, позволяет создавать и просматривать структурированные уроки, предоставляет справочник аккордов, а также включает систему авторизации, личный кабинет, редактор уроков, сортировку и поиск, генерацию табулатуры из аудио-, MIDI-файлов и микрофона с сохранением в PDF и возможностью делиться ими с пользователями приложения. Также реализован метроном.

Область применения: Изучение основ музыкальной теории, развитие музыкальных способностей, самообразование.

ABSTRACT

Final qualifying work, 187 pages, 67 figures, 3 tables, 15 sources, 3 appendices.

Keywords: Tablature generation, mobile application, guitar, lessons, Fast Fourier Transform.

Relevance: The relevance of this topic is due to the need of novice guitarists to acquire knowledge and study their favorite compositions without significant financial investment.

Purpose of the work: Development of a mobile client-server application for learning to play the guitar, which should help users systematize and improve the learning process, as well as provide tools for comfortable mastering the musical instrument.

Software tools used in the process: Android Studio, IntelliJ IDEA, Spring Framework.

Programming languages used: Kotlin.

Results obtained: A guitar learning application has been developed. It generates tablature, allows creating and viewing structured lessons, provides a chord reference, and includes an authorization system, personal account, lesson editor, sorting and search, tablature generation from audio, MIDI files, and microphone with saving to PDF and the ability to share them with application users. A metronome has also been implemented.

Application area: Studying the basics of music theory, developing musical abilities, self-education.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ	6
ВВЕДЕНИЕ	7
1 Анализ предметной области	8
1.1 Актуальность проблемы.....	8
1.2 Анализ подходов генерации табулатуры	8
1.3 Сравнение существующих аналогов приложений для обучения игре на гитаре	13
1.4 Цели и задачи разработки программы.....	15
2 Разработка и анализ требований.....	17
2.1 Требования к архитектуре приложения	17
2.2 Функциональные требования	17
2.3 Нефункциональные требования	20
2.4 Требования к пользовательскому интерфейсу	21
2.5 Выбор средств реализации	23
3 Проектирование программы	26
3.1 Проектирование архитектуры программы.....	26
3.2 Описание структур и базы данных	29
3.3 Описание сервисов и окружения приложения	33
4 Тестирование	38
4.1 Тестирование запуска, регистрации и авторизации пользователя	38
4.2 Тестирование личного кабинета	48
4.3 Тестирование ленты и редактора уроков	65
4.4 Тестирование справочника аккордов	77
4.5 Тестирование инструментов генерации табулатур и метронома	79
ЗАКЛЮЧЕНИЕ	94
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	95
Приложение А	97
Приложение Б	99
Приложение В.....	175

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

ЧОТ – частота основного тона

ДПФ – дискретное преобразование Фурье

БПФ – быстрое преобразование Фурье

MIDI (от англ. musical instrument digital interface) – цифровой интерфейс музыкального инструмента

PDF (от англ. portable document format) – формат переносного документа

БД – база данных

REST (от англ. representational state transfer) – передача состояния представления

API (от англ. application programming interface) – интерфейс прикладного программирования

ОС – операционная система

JVM (от англ. java virtual machine) – виртуальная машина Java

IoC – (от англ. inversion of control) – инверсия управления

DI – (от англ. dependency injection) – внедрение зависимостей

UML (от англ. unified modeling language) – унифицированный язык моделирования

HTTP (от англ. hypertext transfer protocol) – протокол передачи гипертекста

ORM (от англ. object relational mapper) – объектно-реляционное преобразование

JSON (от англ. JavaScript object notation) – нотация объектов JavaScript

JWT (от англ. JSON web token) – стандарт для создания токенов доступа, основанный на формате JSON

IDE (от англ. integrated development environment) – Интегрированная среда разработки

ВВЕДЕНИЕ

В современном мире происходит активное внедрение цифровых и информационно-коммуникационных технологий во все сферы жизни, в том числе и в образование, все больше людей стремятся к освоиванию и развитию своих навыков, как например игре на музыкальных инструментах.

Гитара, будучи популярным и доступным инструментом, привлекает множество лиц. Однако, традиционные методы обучения, такие как музыкальная школа или занятия с репетитором, могут быть дорогостоящими и требовать много свободного времени. Отсутствие плана обучения негативно сказывается на прогрессе ученика. Помимо этого, в ходе самообучения любители не сильно углубляются в музыкальную теорию, а отдают приоритет практике из-за чего чтение нот часто представляет сложность для начинающих гитаристов. Табулатура же показывает гитаристу нужные струны, лады, а также в каком порядке всё это должно звучать. Благодаря такой системе начинающий музыкант, не знающий нотной грамоты, может выучить несложные композиции и освоить исполнительские приёмы [1].

Поэтому целью работы будет являться разработка приложения для обучения игре на гитаре, сочетающее структурированные уроки, справочник популярных аккордов и функцию генерации табулатуры, которая будет способствовать упрощению и ускорению освоения навыка игры на гитаре и позволит создавать собственные аранжировки.

В первой главе данной работы будут рассмотрены актуальность данной работы, рассмотрены существующие аналоги с плюсами и минусами, приведены возможные способы решения, а также установлена цель работы. Вторая глава будет посвящена выбору средств разработки, а также обоснованию выбора операционной системы, библиотек, необходимых аппаратных конфигураций. В третьей главе будет рассказано про реализацию приложения, описание программной архитектуры, структур и базы данных. Четвертая глава будет посвящена тестированию.

1 Анализ предметной области

1.1 Актуальность проблемы

Многим начинающим гитаристам тяжело разобраться в нотной грамоте, и чтобы полноценно изучить данную тему, необходимо потратить много времени, но с помощью табулатуры (схематическая запись музыки для некоторых струнных инструментов, включая гитару) можно быстрее перейти к практике. Однако существует такая проблема, что многие особенно малоизвестные, мелодии предоставлены только в виде нотной нотации, а не табулатуры. Большинство существующих приложений игнорируют данную проблему начинающих музыкантов.

Современные технологии и алгоритмы цифровой обработки сигналов позволяют автоматизировать задачу определения высоты звуков.

Однако, автоматическое распознавание высоты звука является ресурсоёмкой процедурой, чувствительной к качеству исходных данных. Звуковые файлы, подлежащие анализу, требуют корректного сегментирования и применения подходящей оконной функции. Присутствие шумов в сигнале может отрицательно сказаться на точности результата.

Кроме того, сам процесс распознавания сопряжен с определенными трудностями. Громкость, тембр исполняемого инструмента и характерные техники исполнения могут повлиять на качество определения высоты звука.

Таким образом, имеет смысл рассмотреть возможные подходы к генерации табулатуры.

1.2 Анализ подходов генерации табулатуры

Первый метод генерации табулатуры будет использовать преобразование аналогового звукового сигнала в ноты.

Звуки — это колебания воздуха: колебания давления воздуха, равные среднему значению. Среднее давление окружающего воздуха составляет примерно 100 Ньютонов на квадратный метр площади, а колебания, вызванные обычным разговором, примерно на семь порядков (в

десятикратной степени) меньше. Мы будем рассматривать звук как реальную функцию времени, описывающую это изменение. В большинстве случаев, представляющих интерес, эту функцию можно разложить на сумму чистых синусоидальных колебаний различной частоты. Эти синусоидальные колебания и являются составляющими звука. Термин частота относится к числу раз в секунду, когда колебание проходит полный цикл. Частота измеряется в циклах в секунду, или герцах (Гц). Принято описывать диапазон восприятия человеческого уха как 20 Гц - 20 000 Гц (20 кГц). В некоторых случаях, представляющих особый интерес, таких как звуки, издаваемые вибрирующими струнами, частоты составляющих звука связаны особенно простым образом: все они являются целыми числами, умноженными на базовую частоту. Синусоидальная составляющая с этой самой низкой частотой называется основной частотой звука. Когда мы говорим о частоте звука, мы имеем в виду частоту основной частоты. Мы используем измерение частоты, чтобы упорядочить набор всех синусоид и большое количество звуков с помощью их фундаментальной составляющей [2].

Частота определяет высоту звука. Именно высота звука определяет то, какая нота звучит.

Октавы — это установленные отрезки звукового диапазона. Весь звук, который используют в музыке, разделен на 9 участков, то есть на 9 октав. Например, в первой октаве находится звук с частотой в диапазоне от 261,63 Гц до 523,25 Гц. (Подробнее в таблице 1).

Анализ исследований в области обработки музыки показал, что с позиций обработки сигналов мелодию можно представить, как последовательность частот основного тона (ЧОТ), определяемых в моменты звучания музыкального инструмента, то есть на участках, где инструмент активен [3]. Таким образом, в основе понимания того, какая нота была исполнена музыкальным инструментом, лежит знание того, на какой частоте основного тона происходили колебания в данный отрезок времени. Для каждой ноты существует дискретное значение частоты (таблица 1)

Таблица 1 – Соответствие нот частотам основного тона

Нота	Октава								
	Субкон- тркта ва	Контр- октава	Большая октава	Малая октава	1 октава	2 октава	3 октава	4 октава	5 октава
До	-	32.7	65.41	130.82	261.63	523.25	1046.5	2093.0	4186.0
До-диез	-	34.65	69.3	138.59	277.18	554.36	1108.7	2217.4	4434.8
Ре	-	36.95	73.91	147.83	293.66	587.32	1174.6	2349.4	4698.4
Ре-диез	-	38.88	77.83	155.56	311.13	622.26	1244.5	2489.0	4978.0
Ми	20.61	41.21	82.41	164.81	329.63	659.26	1318.5	2637.0	5274.0
Фа	21.82	43.65	87.31	174.62	349.23	698.46	1396.9	2793.8	-
Фа- диез	23.12	46.25	92.5	185.0	369.99	739.98	1480.0	2960.0	-
Соль	24.5	49.0	98.0	196.0	392.0	784.0	1568.0	3136.0	-
Соль- диез	25.95	51.9	103.8	207.0	415.3	830.6	1661.2	3332.4	-
Ля	27.5	55.0	110.0	220.0	440.0	880.0	1720.0	3440.0	-
Си- бемоль	29.13	58.26	116.54	233.08	466.16	932.32	1864.6	3729.2	-
Си	30.87	61.74	123.48	246.96	493.88	987.75	1975.5	3951.0	-

Метод определения ЧОТ состоит из 3 основных шагов:

- разделение его на окна(фреймы),
- поиск возможных значений ЧОТ,
- выбор наиболее вероятных значений.

Аудиосигнал, как правило, не стационарен, то есть его характеристики меняются во времени. Чтобы анализировать изменения ЧОТ во времени, сигнал разбивается на короткие перекрывающиеся отрезки, называемые фреймами или окнами. Обычно используется перекрытие 50%, чтобы избежать потери информации на границах окон и сделать переход между ними более плавным.

На втором этапе для поиска возможных значений ЧОТ применяется

быстрое преобразование Фурье (БПФ) [4]. Этот алгоритм часто применяется в цифровой обработке сигналов. В частности, для создания гитарных тюнеров.

Быстрое преобразование Фурье (БПФ, fast Fourier transform, FFT) – это алгоритм вычисления, который успешно использует свойства периодичности тригонометрических функций для того, чтобы избежать ненужных вычислений в дискретном преобразовании Фурье (ДПФ). В отличие от ДПФ со сложностью вычисления $O[N^2]$, БПФ выполняется за $O[N\log N]$.

$$X_k = \sum_{n=0}^{N-1} x_n * e^{-i*2\pi kn/N} \quad (1)$$

где,

k – Индекс частотной компоненты (от 0 до $N-1$),

X_k – частотная компонента ДПФ. Она представляет собой амплитуду и фазу сигнала на частоте k . Это комплексное число,

N – Общее количество отсчетов во входном сигнале,

n – Индекс текущего отсчета во временной области (от 0 до $N-1$),

x_n – n -ый отсчет входного сигнала во временной области. Исходный сигнал,

e – число Эйлера (приблизительно 2.71828),

i – мнимая единица,

π – число пи (приблизительно 3.141592)

Преобразование x_n в X_k является переводом из временного пространства в пространство частотное. Принцип работы БПФ представлен на рисунке 1

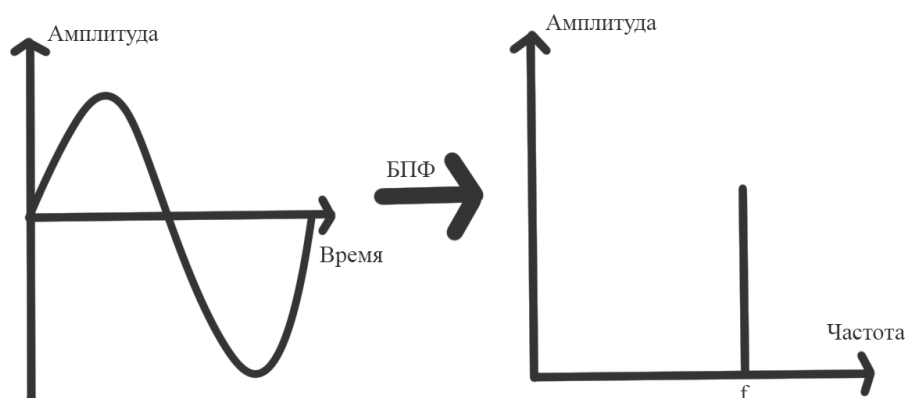


Рисунок 1 – Принцип работы быстрого преобразования Фурье

БПФ применяется к каждому фрейму в результате чего получается спектр мощности, который показывает амплитуду каждой частотной составляющей в сигнале.

В последнем этапе применяется фильтр, который убирает значения выходящих за диапазон частот гитары (от 82 Гц до 1175 Гц) и другие не желательные частотные составляющие.

Далее необходимо произвести квантование (округление) полученной частоты до ближайшей ноты. Для этого применяется формула для вычисления отклонения от ноты Ля 1-ой октавы (А4 равной 440 Гц) в полутонах:

$$12 * \log_2 \left(\frac{f}{440} \right) \quad (2)$$

где,

f – частота полученная после БПФ.

Полученное значение округляется до ближайшего целого числа, которое соответствует количеству полутонов от А4.

Для второго метода генерации табулатуры будет использоваться не аналоговый сигнал или оцифрованный, а синтезированный передаваемый в файле формата MIDI.

MIDI (Musical Instrument Digital Interface) – это унифицированный протокол, обеспечивающий обмен информацией между электронными музыкальными инструментами. Он позволяет передавать данные о ноте (ее номер), силе нажатия, длительности звучания и других параметрах.

Стандартный MIDI-файл (SMF — Standard MIDI File) представляет собой специализированный формат файлов, предназначенный для хранения данных, генерируемых и/или воспроизводимых секвенсором. Секвенсор может быть реализован как в виде программного обеспечения для компьютера, так и в виде аппаратного модуля.

Извлечение звуков MIDI является выраженным клавишно-ориентированным протоколом, поэтому процесс извлечения звуков кодируется двумя простыми сообщениями — взять ноту (Note On) и снять

ноту (Note Off). Исполнитель при нажатии клавиши задает сразу три параметра: момент начала звучания, динамику и высоту тона. Длительность звука определяется по моменту отпускания клавиши. Несмотря на это такая модель подходит и для инструментов с другим принципом звукоизвлечения (как например гитара).

1.3 Сравнение существующих аналогов приложений для обучения игре на гитаре

Существуют различные приложения на тему обучения игре на гитаре, но качество и объем предоставляемых возможностей сильно варьируется.

Для сравнения были выбраны следующие приложения: Yousician, Fender play, Ultimate Guitar, Chord ai и GuitarTuna [5]. Результаты сравнения представлены в таблице 1.

Пройдемся по основным достоинствам и недостаткам данных приложений:

1) Yousician

Достоинства: интерактивные уроки, практика навыков игры

Недостатки: Ограничение по времени в бесплатной версии, переусложненный интерфейс.

2) Fender play

Достоинства: обучающие уроки по мимо гитары доступны для ба-гитары и укулеле.

Недостатки: обязательная подписка

3) Ultimate guitar

Достоинства: поиск и использование аккордов, табулатур и текстов песен. Возможность сохранять в избранное, есть официальная подборка от команды поддержки приложения.

Недостатки: отсутствие русского языка, отсутствие уроков как таковых, большая часть функционала доступна по подписке.

4) Chord ai

Достоинства: большая база аккордов, возможностью определения аккордов по звучанию.

Недостатки: отсутствие обучающих уроков, ограничение бесплатной версии.

5) GuitarTuna

Достоинства: точный тюнер для настройки гитары, встроенный метроном, высокий выбор инструментов и настроек.

Недостатки: Отсутствие обучающих уроков как таковых (предлагает скачать другое приложение), ограниченный функционал в бесплатной версии.

Таблица 2 – Таблица сравнения достоинств и недостатков приложений

Приложение	Yousician	Fender play	Ultimate Guitar	Chord ai	GuitarTuna
Пользовательский интерфейс	Интерфейс перегружен	Простой, минималистичный	Устаревший, перегружен	Простой	Простой
Встроенный тюнер и метроном	есть	есть	нет	Нет метронома	есть
Обучающие уроки	есть	есть	нет	нет	интеграция Yousician
Социальные взаимодействия	ограничены	ограничены	ограниченны	нет	нет
Справочные материалы	Аккорды, гаммы	аккорды	Огромная база аккордов и табов	Большая база аккордов	аккорды
Генерация табулатуры	нет	нет	нет	Возможность распознавать аккорды, но не табулатуру	нет

Был проведен анализ каждого приложения, изучены достоинства и недостатки по следующим критериям: пользовательский интерфейс, встроенный тюнер и метроном, обучающие уроки, социальные взаимодействия, справочные материалы, генерация табулатуры. Все найденные на рынке приложения обладают рядом недостатков, основным негативным фактором среди рассмотренных программных систем выступает функционала для генерации табулатуры, а также излишняя перегруженность интерфейса, из чего можно сделать вывод, что на рынке отсутствуют приложения, сочетающие в себе обучающую базу, справочные материалы и инструменты для генерации табулатуры доступных на русском языке, что в очередной раз подтверждает актуальность работы.

1.4 Цели и задачи разработки программы

Целью данной работы будет являться разработка современного приложения для обучения игре на гитаре, которое должно помочь пользователям систематизировать и улучшить процесс обучения, а также предоставить различный инструментарий для комфортного освоения музыкального инструмента.

Задача работы заключается в создании приложения, которое должно включать в себя следующие функции:

- 1) авторизация и регистрация пользователей в системе,
- 2) личный кабинет. Должна быть возможность редактирования личных данных пользователя, просмотр, редактирование и удаление своих черновиков и опубликованных уроков, а также табулатур,
- 3) встроенный редактор уроков. Благодаря нему приложение сможет активно пополняться новыми уроками как от администрации, так и сообщества пользователей приложения. Редактор должен иметь полный набор инструментов для написания уроков, таких как форматирование текста, загрузка картинок и видео,

4) просмотр уроков, созданных администрацией приложения, а также тематических статей от пользователей,

5) сортировка и поиск уроков. Это поможет лучше ориентироваться и просматривать интересующие темы,

6) генерация табулатуры из аудиофайлов, микрофонного входа или файла формата MIDI с дальнейшим сохранением в PDF, личный кабинет или в общий доступ в приложении,

7) просмотр и поиск аккордов по названию,

8) встроенный метроном.

Теперь, когда была поставлена цель работы и стали понятны задачи, которые необходимо решить для реализации приложения, можно перейти к разработке требований к системе.

2 Разработка и анализ требований

2.1 Требования к архитектуре приложения

Разрабатываемое программное обеспечение для обучения игре на гитаре должно базироваться на двухуровневой или клиент-серверной архитектуре [6]. Это предполагает его структурное разделение на следующие компоненты: клиентскую и серверную части. В свою очередь, серверная часть включает в себя backend-приложение и базу данных (БД).

Клиентская часть приложения должна быть реализована в виде мобильного приложения для операционной системы Android. В условиях современного цифрового общества мобильные устройства стали неотъемлемой частью повседневной жизни каждого человека, что обуславливает актуальность и востребованность данного подхода.

Выбор Android для разработки клиентской части приложения обоснован доминированием на рынке мобильных устройств, открытой архитектурой и гибкостью, развитой экосистемой инструментов и библиотек, а также более низкой стоимостью разработки по сравнению с альтернативами.

Сервер должен предоставлять доступ через REST API для обеспечения эффективного обмена данными между компонентами системы.

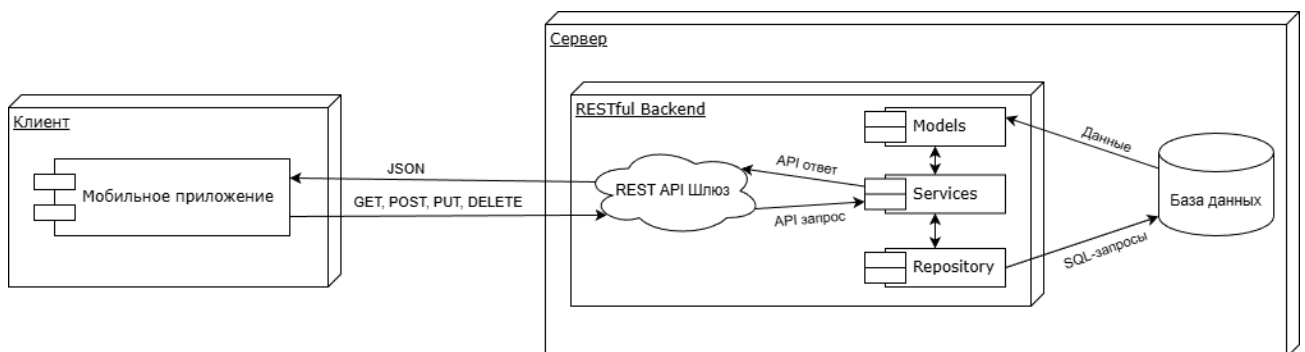


Рисунок 2 – Диаграмма компонентов для выбранной архитектуры

2.2 Функциональные требования

В процессе разработки приложения для обучения игре на гитаре с функцией генерации табулатуры первоочередной задачей является

определение того, что должна делать система. Для этого выделим основные функциональные требования для приложения:

Приложение должно поддерживать следующий список пользовательских ролей:

- 1) гость (не авторизированный пользователь),
- 2) пользователь,
- 3) администратор

Функционал доступный пользователю с ролью «Гость»:

- 1) авторизация и регистрация пользователя,
- 2) просмотр, сортировка и поиск уроков, а также выложенных табулатур по названию,
- 3) поиск аккордов,
- 4) использование метронома.

Для роли «Пользователь» доступен весь предыдущий функционал, а также:

- 1) использование личного кабинета. В нем пользователь сможет редактировать личные данные (имя, почту, пароль, изображение профиля пользователя), удалить аккаунт или выйти из системы,
- 2) создавать посты из редактора с возможностью сохранения их в черновики или выкладывания в общую ленту,
- 3) создавать, просматривать, редактировать или удалять свои черновики, посты и сохраненные табулатуры,
- 4) ставить оценку и комментировать посты пользователей,
- 5) генерировать табулатуру из MIDI, WAV или записи с микрофона,
- 6) сохранять сгенерированную табулатуру в личный кабинет или в PDF на устройство,
- 7) делиться созданной табулатурой внутри приложения.

На рисунке 3 изображена диаграмма вариантов использования для авторизированных и неавторизированных пользователей описывающая основной функционал.

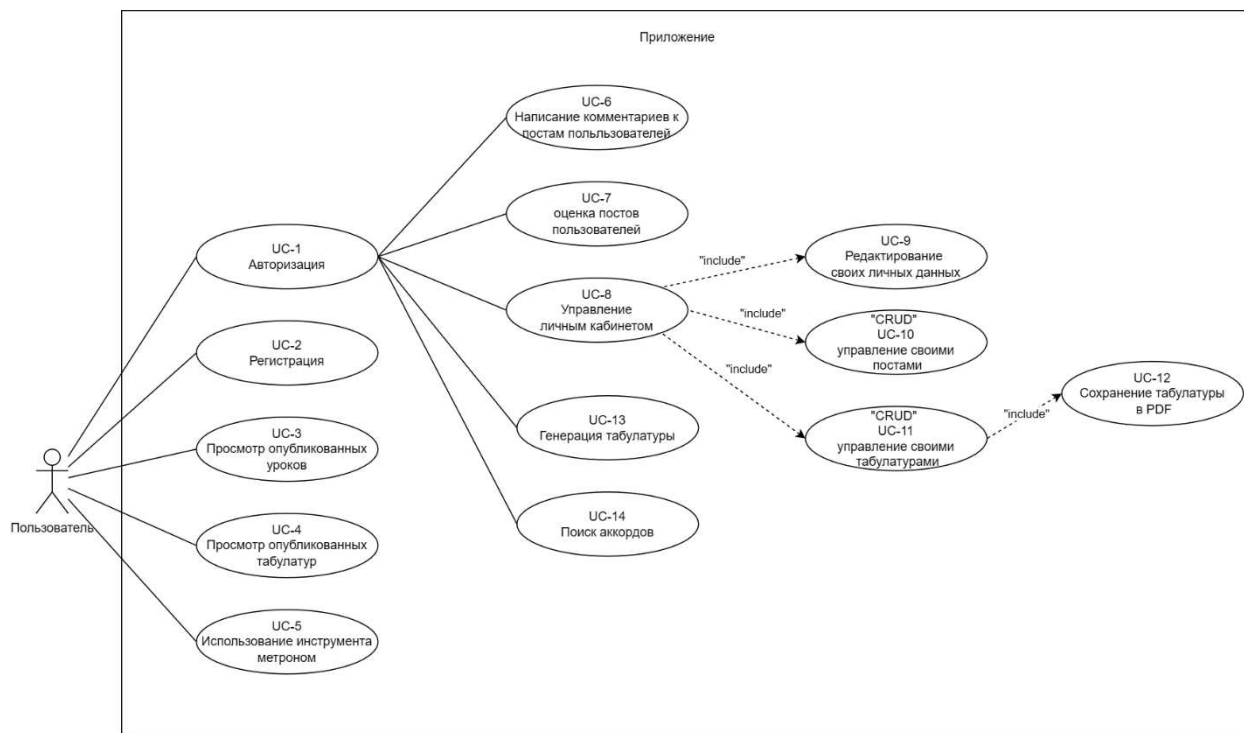


Рисунок 3 - Диаграмма вариантов использования для пользователя

Для роли «Администратор» доступен также весь предыдущий функционал, а также:

- 1) удаление постов пользователей,
- 2) удаление опубликованных табулатур,
- 3) создание, редактирование и удаление основного курса уроков.

На рисунке 4 изображена диаграмма вариантов использования для задач администраторов описывающая их основной функционал.

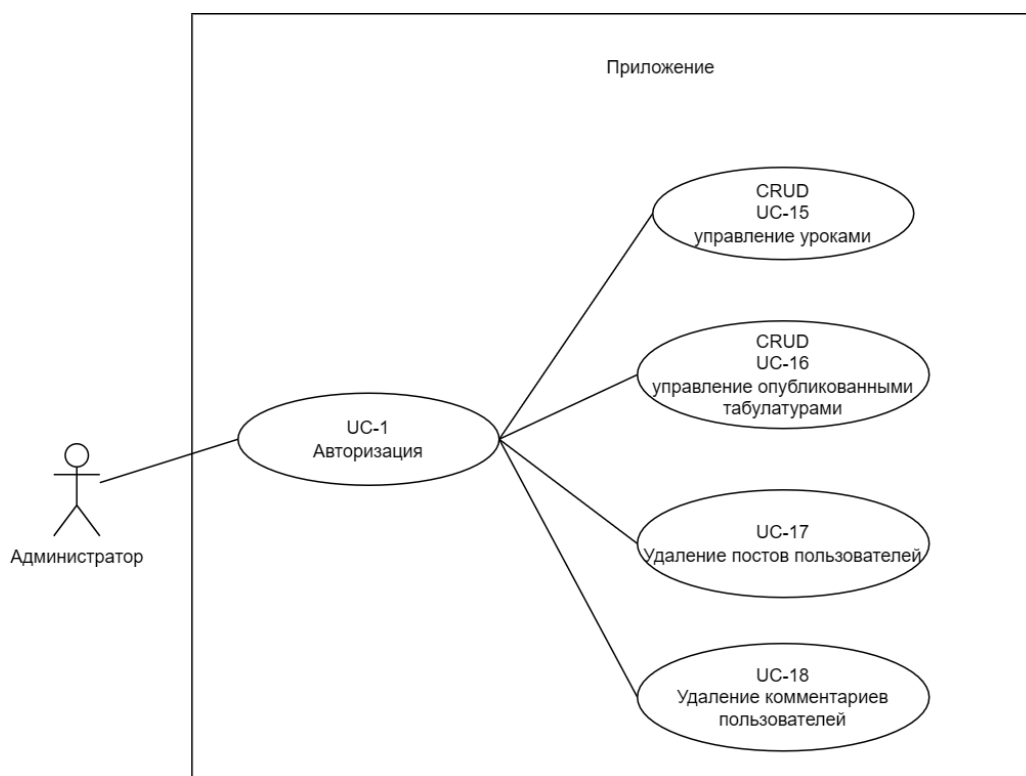


Рисунок 4 – Диаграмма вариантов использования для администратора

2.3 Нефункциональные требования

Данный раздел описывает нефункциональные требования к приложению для обучения игре на гитаре с функцией генерации табулатуры. Эти требования определяют качество, производительность и другие характеристики системы, не связанные напрямую с её функциональностью. Соблюдение этих требований критически важно для обеспечения удобства использования, стабильности и соответствия ожиданиям пользователей.

Рассмотрим основные нефункциональные требования к продукту:

- 1) производительность. Время, которое требуется приложению для ответа на действие пользователя не должно превышать более 2 секунд,
- 2) безопасность. Приложение должно поддерживать надежные методы шифрования паролей и хранить пароли в БД в виде хеша,
- 3) адаптивность. Приложение должно корректно работать на устройствах различных типов экранов,

4) совместимость. Приложение должно поддерживать ОС начиная с версии Android 8.0 Oreo.

2.4 Требования к пользовательскому интерфейсу

В данном разделе представлено описание экранов и возможных переходов между ними. Для визуализации структуры экранов и логики переходов была разработана карта экранов и переходов (рисунок 5). На представленной схеме отражена последовательность взаимодействия пользователя с интерфейсом. Представлены следующие макеты экранов:

1) Основные экраны доступные из меню

- инструменты – из этого экрана должен иметься доступ к экрану генерации табулатуры и метроному;
- аккорды – справочник с аккордами;
- уроки – начальный экран приложения, открывающийся при его запуске. Содержит ленту уроков и переход к редактору для авторизированных пользователей;
- табулатуры – содержит ленту из опубликованных табулатур пользователей;
- личный кабинет – авторизированные пользователи получают доступ к экранам для управления аккаунтом, черновикам, своим постам и сохраненным табулатурам, неавторизованный пользователь на этом экране должен иметь возможность перейти к экрану авторизации.

2) дочерние экраны

- генерация табулатуры;
- метроном;
- редактор;
- управление аккаунтом;
- черновик;
- авторизация;
- регистрация.

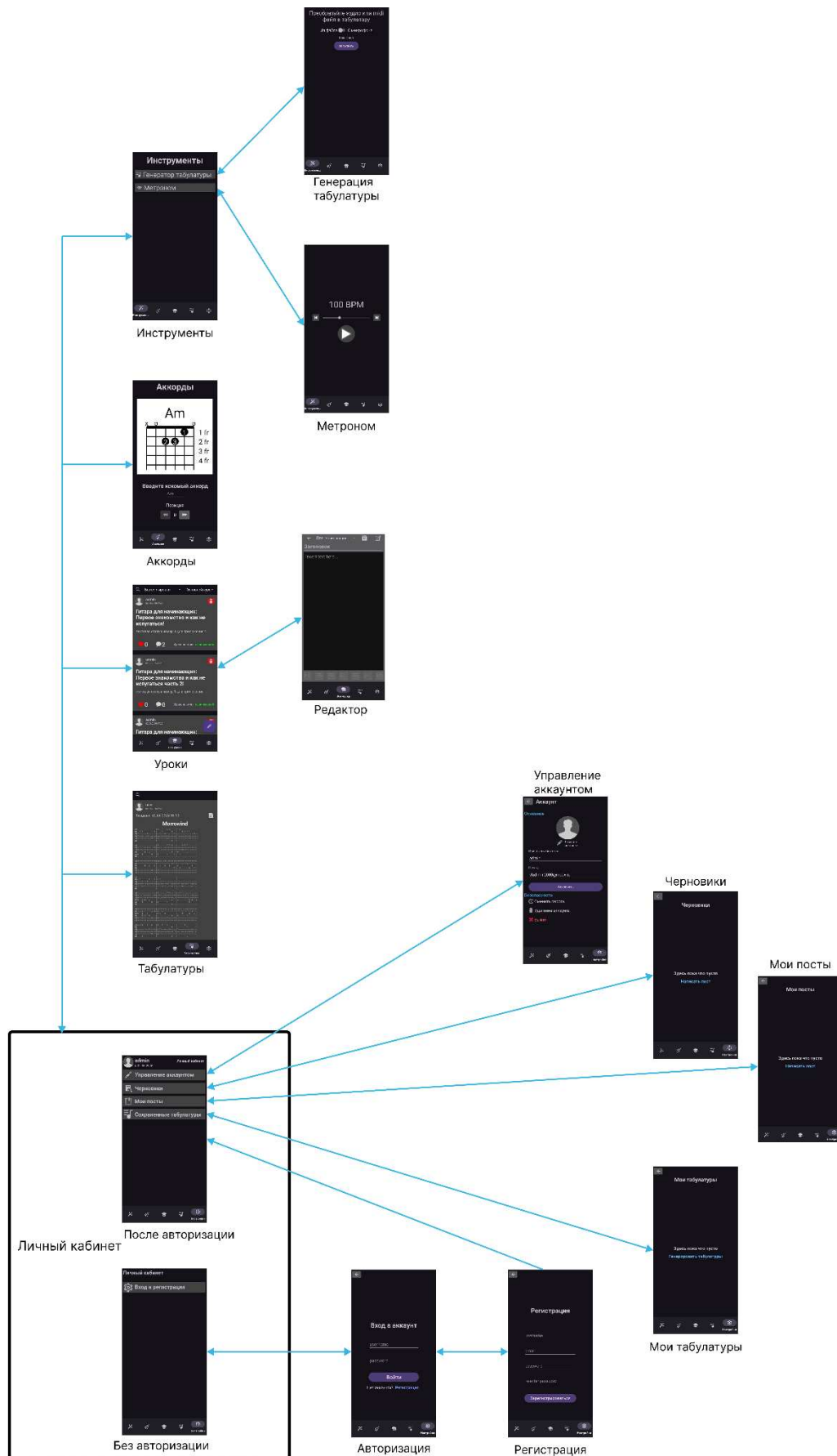


Рисунок 5 – Карта экранов и переходов

2.5 Выбор средств реализации

На основании требований, изложенных ранее для разрабатываемого мобильного приложения обучения игре на гитаре с функцией генерации табулатуры, был произведен выбор технологий, которые будут использоваться в рамках реализации проекта.

В качестве основного языка разработки был выбран Kotlin.[7]

Kotlin — кроссплатформенный, статически типизированный, объектно-ориентированный язык программирования, работающий поверх Java Virtual Machine (JVM) и разрабатываемый компанией JetBrains. В 2019 году Google объявили этот язык приоритетным для разработки android-приложений.

Для реализации разделения прав доступа к серверу через REST API на основе системы ролей, в соответствии с функциональными требованиями и выбранной архитектурой приложения, был выбран Spring Framework [8].

Для написания серверной части потребуются следующие модули этого фреймворка.

1) Spring boot

Spring boot – это развитие фреймворка Spring, который основан на принципах IoC (Inversion of Control) и DI (Dependency Injection). Использование Spring Boot позволяет автоматизировать процесс документирования, включая создание UML-диаграмм, а также управление зависимостями приложения от сторонних библиотек.

2) Spring web

Модуль фреймворка Spring, предназначенный для создания веб-приложений, в частности, RESTful API. Он предоставляет набор аннотаций и классов для обработки HTTP-запросов и ответов, маршрутизации запросов, сериализации и десериализации данных, а также обработки исключений. Spring Web также способствует автоматической генерации документации API (например, с использованием Swagger или Spring REST Docs) и предоставляет инструменты для валидации входных данных.

3) Spring data

Фреймворк Spring, направленный на упрощение доступа к данным из различных источников, включая реляционные и NoSQL базы данных. Он использует объектно-реляционное отображение (ORM) посредством Hibernate (по умолчанию) для маппинга между объектами и таблицами базы данных. Spring Data предоставляет репозитории - интерфейсы, которые автоматически реализуются фреймворком и содержат набор стандартных операций CRUD (Create, Read, Update, Delete) для работы с данными, что значительно сокращает объем кода, необходимого для взаимодействия с базой данных.

4) Spring security

Spring security — это мощная и легко настраиваемая платформа аутентификации и контроля доступа. Это стандарт для обеспечения безопасности приложений на базе Spring [9].

Помимо этого потребуется библиотека JSON (javascript object notation) JWT (java web token) для создания и верификации JWT токенов.

JWT - это ключ аутентификации пользователя. Используется для запросов к защищенным методам API.

Так как БД должна работать с большим объемом данных, иметь высокую производительность и масштабируемость была выбрана объектно-реляционная база данных PostgreSQL [10].

В качестве IDE для разработки серверной части приложения была выбрана IntelliJ IDEA, т.к. она имеет продвинутые инструменты разработки и отладки, удобный пользовательский интерфейс и активное сообщество.

Далее рассмотрим выбранные технологии для клиентской части.

Клиентское приложение будет написано в IDE Android studio. Android studio является официальным IDE от компании Google для написания нативных приложений под ОС android. [11]

Рассмотри модули, которые необходимо подключить:

1) Retrofit2 – это типобезопасный HTTP-клиент для взаимодействия с REST API,

2) TarsosDSP – библиотека цифровой обработки сигналов изначально написанный для Java. Она специализируется на анализе и обработке аудиосигналов. Библиотека выбрана в связи с функциональными требованиями, [12]

3) Glide — это библиотека для загрузки и отображения изображений. Она автоматически управляет кэшированием изображений, чтобы улучшить производительность и избежать повторной загрузки одних и тех же изображений. Glide также поддерживает различные форматы изображений и умеет выполнять их преобразования (например, изменение размера, обрезка),

4) RichEditor – это библиотека, включающая функционал для написания WYSIWYG (от англ. what you see is what you get – что видишь, то и получаешь) текстового редактора, [13]

WYSIWYG-редактор — это инструмент для создания и редактирования контента (текста, изображений, таблиц) в режиме реального времени, где результат отображается так, как он будет выглядеть после публикации,

5) ChordDroid – это специализированная библиотека, которая упрощает работу с отображением аккордов в Android-приложениях [14].

Таким образом, выбранный стек технологий позволяет эффективно реализовать функционал мобильного приложения для обучения игре на гитаре с функцией генерации табулатуры и далее можно переходить к ее реализации.

3 Проектирование программы

3.1 Проектирование архитектуры программы

Архитектура разрабатываемого программного обеспечения разделена на два проекта: клиентскую и серверную составляющие. Рассмотрим архитектуру каждой из этих частей более подробно.

Клиентское приложение построенное в Android studio основано на архитектурном шаблоне MVVM.

MVVM (Model-View-ViewModel) — это архитектурный шаблон проектирования, который делит приложение на три ключевых компонента: Model, View и ViewModel. Эти компоненты работают вместе, обеспечивая чёткое разделение задач, что приводит к улучшению поддерживаемости, тестируемости и масштабируемости.

Model (Модель) - это уровень данных в приложении. Она представляет бизнес-логику, источники данных (например, базы данных или API) и структуры данных (например, модели или сущности). Модель отвечает за обработку данных приложения включая их получение с удалённого сервера. Она не знает о View или ViewModel.

View (Представление) - это уровень пользовательского интерфейса. Оно отображает данные для пользователя и взаимодействует с ним через различные компоненты пользовательского интерфейса, такие как кнопки, текстовые поля и списки. В разрабатываемом приложении это Activity, Fragment и xml разметка. Представление наблюдает за ViewModel и реагирует на изменения в данных.

ViewModel (Модель-представление) выступает в качестве посредника между моделью и представлением. Она получает данные из модели, при необходимости обрабатывает или форматирует их и предоставляет в виде, удобном для представления. ViewModel хранит данные, связанные с пользовательским интерфейсом, с учётом жизненного цикла, то есть она сохраняет данные при изменении конфигурации (например, при повороте экрана). Она также предоставляет механизму представления возможность

привязываться к данным, с помощью привязки данных или наблюдателей. На рисунке 6 представлена общая схема работы архитектурного шаблона MVVM.

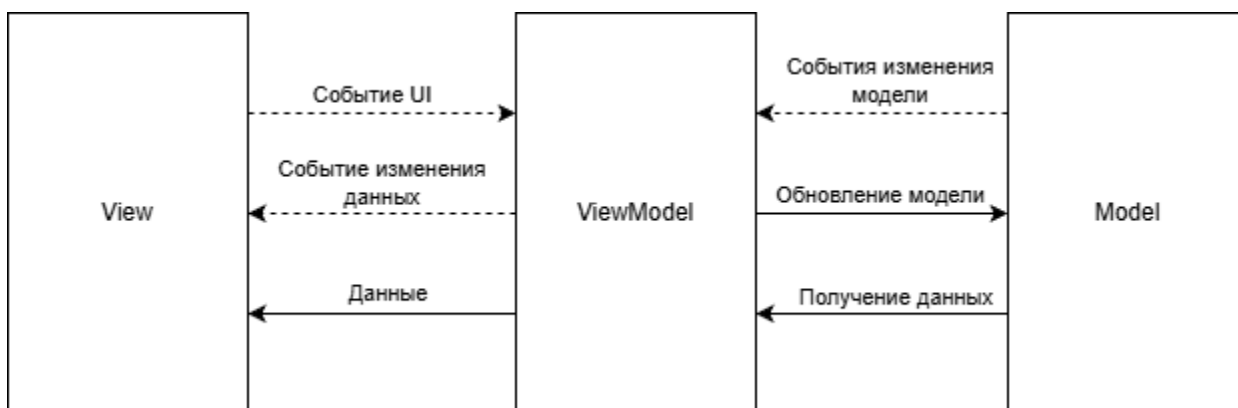


Рисунок 6 – Архитектура MVVM

Серверная часть реализована на фреймворке Spring. Spring framework построен на так называемой инверсии управления (Inversion of Control, IoC), которая также иногда называется внедрением зависимостей (Dependency Injection, DI). Это шаблон проектирования, при использовании которого зависимости экземпляров класса определяются на основе аргументов конструктора, а также полей объекта, значение которых определяется уже после создания экземпляров класса.

Данный подход эффективно сочетается с концепцией проектирования предметной области, известной как Simple domain model. В рамках этой модели функциональность объектов выделяется в отдельные классы-сервисы, что обеспечивает сосредоточение бизнес-логики приложения в этих сервисах. В случаях, когда предметная область включает значительное количество объектов, такой подход значительно облегчает последующую поддержку и модификацию разработанного программного обеспечения.

В качестве модели архитектуры REST API приложения на Spring используется шаблон Model-View-Controller (MVC).

MVC (Model-View-Controller) — это архитектурный паттерн, который разделяет приложение на три взаимосвязанные части:

Model (Модель) - представляет данные приложения и бизнес-логику. Она отвечает за управление данными, их извлечение, обновление и выполнение бизнес-правил. Модель не знает ни о представлении (View), ни о контроллере (Controller). Когда состояние модели изменяется, она уведомляет об этом заинтересованные компоненты (обычно контроллер).

View (Представление) - отображает данные модели пользователю. Оно отвечает за визуальное представление информации и взаимодействие с пользователем. Представление получает данные от модели и возвращает в формате JSON для конечного клиента на мобильном приложении. Представление не содержит бизнес-логики.

Controller (Контроллер) - обрабатывает запросы от пользователя, взаимодействует с моделью для обновления данных, и выбирает соответствующее представление для отображения пользователю. Контроллер действует как посредник между моделью и представлением. Он получает входные данные от пользователя передает их модели для обработки, а затем выбирает, какое представление отображать пользователю. На рисунке 7 представлена общая схема работы архитектурного шаблона MVC.

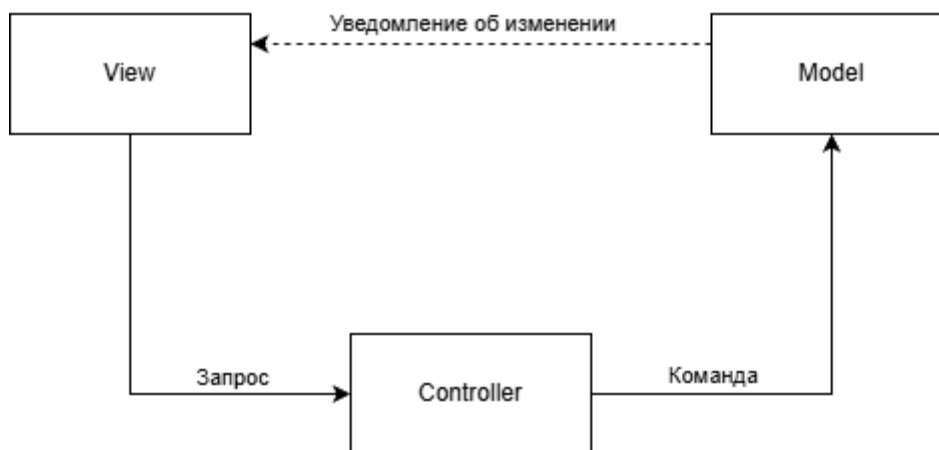


Рисунок 7 – Архитектура MVC

3.2 Описание структур и базы данных

Реализуемое приложение оперирует следующими сущностями предметной области:

- 1) roles –содержит роли авторизированных пользователей (“USER” И “ADMIN”),
- 2) users – содержит данные о зарегистрированных пользователях приложения включая: имя пользователя, почту, пароль (хранящийся в виде хеша), дата регистрации, путь к изображению профиля и роли пользователя
- 3) skill_level – содержит уровни сложности уроков (“Начинающий”, “Уверенный”, “Продвинутый”),
- 4) status – содержит текущий статус урока (“Черновик”, “Опубликован”, “Распубликован”),
- 5) article – урок с данными о заголовке, содержимом статьи, его авторе, уровне сложности, рейтинге, статусе и дате публикации,
- 6) tabs – табулатуры, сохраненные пользователями, содержит информацию о названии табулатуры, саму табулатуру, авторе, дате и времени создания,
- 7) articles_rating – в ней содержится информации о пользователях и уроках, которые они оценили,
- 8) comment – комментарии пользователей содержит информацию об авторе, дате, тексте, уроке к которому этот комментарий был написан.

Схема, используемая базой данных приложения, находится в третьей нормальной форме [15]. Диаграмма отношений сущностей, описывающая таблицы БД приложения приведена на рисунке 8.

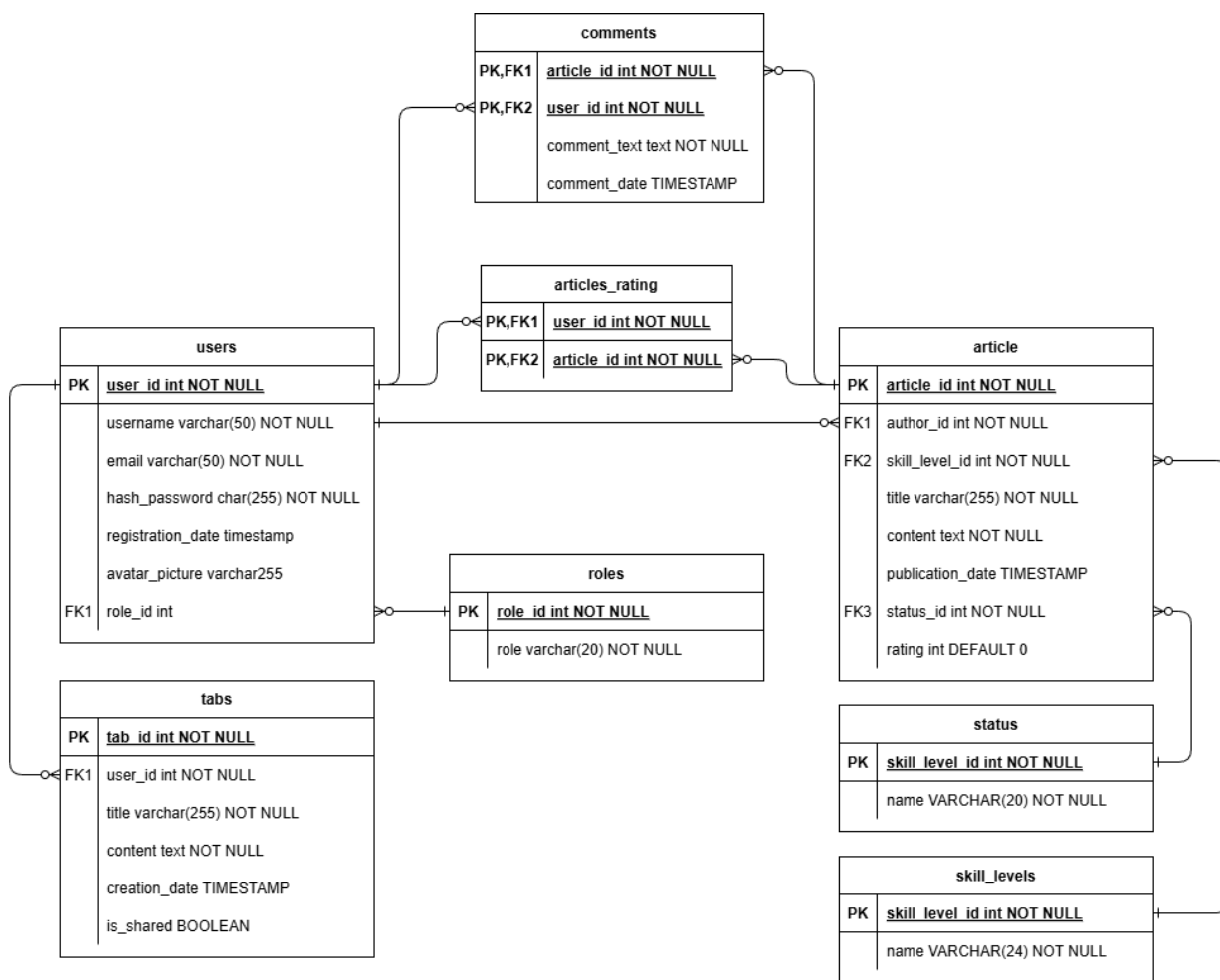


Рисунок 8 – ERD базы данных для разрабатываемого приложения

В приложении А представлен SQL-скрипт, который осуществляет подготовку базы данных для работы с приложением. В таблице 3 можно найти описание полей и их значений в таблицах БД.

Таблица 3 – Словарь данных

Имя	Тип данных	Значение
roles		
role_id	integer	Первичный ключ, авто инкрементируемое значение
role_name	varchar (50)	Название роли, уникальное, не может быть пустым
users		
user_id	integer	Первичный ключ, авто инкрементируемое значение

Продолжение таблицы 3

username	varchar (50)	Псевдоним пользователя, уникальное, не может быть пустым
email	varchar (255)	электронная почта пользователя, уникальное, не может быть пустым
password	varchar (255)	Пароль пользователя, хранится в виде хеша, не может быть пустым
registration_date	TIMESTAMP WITH TIME ZONE	Дата и время регистрации, не может быть пустым, по умолчанию принимает значение текущего времени сервера
avatar_picture	varchar (255)	Содержит путь к изображению профиля
role_id	integer	Внешний ключ – ссылка на роль пользователя, не может быть пустым
status		
status_id	integer	Первичный ключ, авто инкрементируемое значение
status_name	varchar (20)	Название статуса, уникальное, не может быть пустым
skill_level		
skill_level_id	Integer	Первичный ключ, авто инкрементируемое значение
name	varchar (24)	Название для требуемого уровня навыка, уникальное, не может быть пустым
articles		
article_id	Integer	Первичный ключ, авто инкрементируемое значение
title	Varchar (255)	Заголовок статьи, не может быть пустым
content	TEXT	Содержимое статьи, не может быть пустым
publication_date	TIMESTAMP WITH TIME ZONE	Дата публикации, значение по умолчанию текущее время сервера
rating	Integer	Рейтинг статьи, значение по умолчанию 0
author_id	Integer	Внешний ключ – ссылка на пользователя, создавшего эту статью, не может быть пустой

Продолжение таблицы 3

skill_level_id	Integer	Внешний ключ – ссылка на значение требуемого уровня навыков, не может быть пустой
status_id	Integer	Внешний ключ – ссылка на статус статьи, не может быть пустой
tabs		
tab_id	Integer	Первичный ключ, авто инкрементируемое значение
title	varchar (255)	Название табулатуры, не может быть пустым
content	TEXT	табулатура, не может быть пустой
creation_date	TIMESTAMP WITH TIME ZONE	Дата и время создания табулатуры, не может быть пустым, по умолчанию текущее время сервера
author_id	INTEGER	Внешний ключ – ссылка на пользователя, создавшего данную табулатуру, не может быть пустым
is_shared	BOOLEAN	Значение показывает опубликована табулатура или нет: 0 – скрыта, 1 – опубликована.
comment		
article_id	Integer	Внешний ключ – ссылка на статью, часть составного первичного ключа, не может быть пустым
user_id	Integer	Внешний ключ - ссылка на пользователя, написавшего комментарий, часть составного первичного ключа, не может быть пустым
Comment_text	TEXT	Текст комментария, не может быть пустым
Comment_date	TIMESTAMP WITH TIME ZONE	Дата и время создания комментария

Продолжение таблицы 3

article_rating		
article_id	Integer	Внешний ключ – ссылка на статью, часть составного первичного ключа, не может быть пустым
user_id	Integer	Внешний ключ - ссылка на пользователя, написавшего комментарий, часть составного первичного ключа, не может быть пустым

3.3 Описание сервисов и окружения приложения

Проект мобильного приложения реализованного в Android studio включает следующие основные компоненты:

1) AndroidManifest.xml – это файл манифеста для Android-приложения, который определяет его структур, функции и взаимодействие с системой. Содержащий информацию о таких данных приложения, как необходимые разрешения для выполнения определенных функций приложения, параметры приложения, начальную Activity и т.п.

2) build.gradle.kts – модуль системы автоматической сборки gradle, в котором содержится информация о всех подключенных библиотеках.

3) res – папка, содержащая все статические ресурсы приложения такие как xml код экранов, шрифты, иконки, цвета, строковые значения и т.п.

4) директория с классами фрагментов переключаемых из Activity. Содержит следующие классы:

- AccordsFragment – представление экрана справочника аккордов;
- AcountManagerFragment – представление для экрана управления профилем пользователя;
- ArticleFragment – представление для экрана открытого урока или поста пользователей;
- MidiFragment – представление для экрана генерирования табулатуры;

— DraftsFragment – представление для экрана с черновиками пользователя;

— EditorFragment – представление для экрана с редактором постов и уроков;

— LessonsFragment – представление для экрана с лентой уроков и постов;

— LoginFragment – представление для экрана с формой авторизации;

— MetronomeFragment – представление для экрана с метрономом;

— MyPostFragment – представление для экрана с постами пользователя;

— MyTabsFragment - представление для экрана с табулатурами пользователя;

— PermissionDeniedFragment – представление для экрана отображаемого при отсутствии выделенных разрешений пользователем;

— RegisterFragment – представление для экрана с формой регистрации;

— SettingsFragment – представление для экрана личного кабинета;

— TabsFragment – представление для экрана с лентой опубликованных табулатур;

— ToolsChoiceFragment – представление для экрана с выбором инструментов.

5) директория utils для вспомогательных singleton классов:

— Metronome – содержит логику работы метронома;

— NoteUtils – Содержит методы для преобразования частоты в ноту вида названия ноты и октавы (Пример «А3») а так же метод для вычисления номера ноты в формате MIDI;

— PdfCreator – включает в себя метод для формирования pdf файла из табулатуры;

— NetworkUtils – содержит методы проверяющие наличия сети и доступа к интернету

6) директория guitarPitch содержит класс NoteDurationDetector предназначенный для распознавания нот

7) директория adapter содержит адаптеры для элемента интерфейса RecyclerView предназначенный для динамического отображения уроков, комментариев и табулатур

8) директория retrofit – содержит в себе модели DTO («объект передачи данных» от англ. Data Transfer Object) и интерфейсы для доступа к REST API

9) директория viewModel содержит классы предназначен для хранения и управления данными, связанными с пользовательским интерфейсом (UI), в соответствии с жизненным циклом Activity и Fragment. Содержит классы ArticleViewModel, DraftViewModel, TokensViewModel.

10) MainActivity основная точка запуска приложения

Проект серверной части состоит из следующих компонентов:

1) build.gradle.kts – модуль системы автоматической сборки gradle, в котором содержится информация о всех подключенных библиотеках;

2) Директория resources – содержит скрипт для преобразования миди файла в табулатуру, код инициализации базы данных, а также файл конфигурации application.yml который используется для определения свойств приложения Spring Boot.

3) Классы Spring Security отвечающие за защиту приложения от угроз безопасности и получения JWT токена пользователями. Включает следующие классы:

— JwtConfigurer – содержит конфигурационные данные для фильтра;

— JwtTokenFilter – фильтр проверяющий наличия допустимого токена JWT в заголовке запроса Authorization;

— JwtTokenProvider – класс отвечающий за создание и проверку токена;

— JwtUser – класс содержащий данные о пользователе по токену;

- JwtUserFactory – класс шаблон для создания новых JwtUser;
- AuthenticationService – класс сервис выполняющий проверку аутентификации пользователя по токену, а также обновление accessToken по действующему refreshToken;
- JwtUserDetailsService – получение данных о пользователе из токена
- SecurityConfig – содержит данные для конфигурации безопасности приложения. В нем задаются правила работы фильтра и доступ к страницам доступных без авторизации.

4) директория controller содержит классы с конечными точками для запросов, разделенных по бизнес-логике. Содержит следующие классы:

- AdminRestController – содержит запросы доступные для администраторов;
- ArticleRestController – содержит запросы связанные с уроками и постами;
- AuthenticationRestController – содержит запросы связанные с авторизацией и регистрацией пользователей;
- CommentRestController – содержит запросы связанные с комментариями;
- TabsRestController – содержит запросы связанные с табулатурами;
- UploadImageRestController – содержит запросы загрузки и получения изображений вставленных в редакторе;
- UserRestController – содержит запросы связанные со всеми пользователями.

5) директория dto содержит классы, предназначенные для отправки пользователям

6) директория model содержит классы сущностей связанных с базой данных PostgreSQL

7) директория repository содержит классы наследники JpaRepository предназначенных для взаимодействия с базой данных. Содержит следующие классы:

- ArticleRepository;
- CommentRepository;
- RefreshTokenRepository;
- RoleRepository;
- SkillLevelRepository;
- StatusRepository;
- TabRepository;
- UserRepository.

8) директория service содержит классы Service содержащую основную логику. Включает следующие классы:

- ArticleService и ArticleServiceImpl;
- CommentServiceImpl;
- SkillLevelServiceImpl;
- StatusServiceImpl;
- TabServiceImpl;
- UserServiceImpl.

9) директория utils содержит вспомогательные классы:

- MidiGeneratorUtils – предназначен для генерации midi файлов;
- NoteUtils – Содержит методы для преобразования частоты в ноту вида названия ноты и октавы (Пример «A3») а так же метод для вычисления номера ноты в формате MIDI;
- PythonUtils – запускает python скрипт для преобразования данных из MIDI файла в табулатуру.

4 Тестирование

Для демонстрации выполнения всех требований к приложению было проведено функциональное ручное тестирование.

4.1 Тестирование запуска, регистрации и авторизации пользователя

При первом запуске приложение запросит доступ к внутреннему хранилищу, если пользователю отклонить запрос, то вместо начального экрана отобразится уведомление (рисунки 9 – 10)

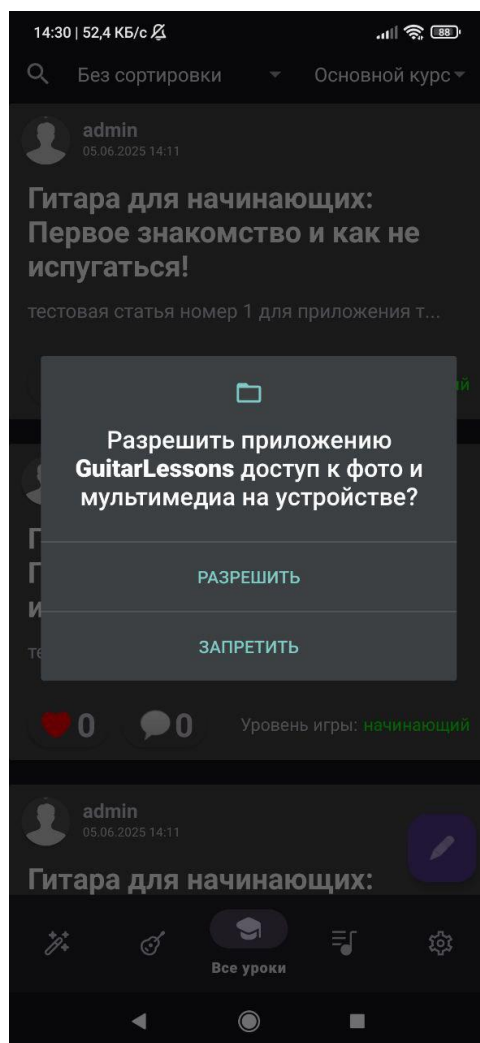


Рисунок 9 – Запрос разрешения на чтение и запись из внутреннего хранилища



Рисунок 10 – Сообщение о необходимости предоставить разрешения

После перехода из меню во вкладку «Настройки» пользователю откроется экран, представленный на рисунке 11.

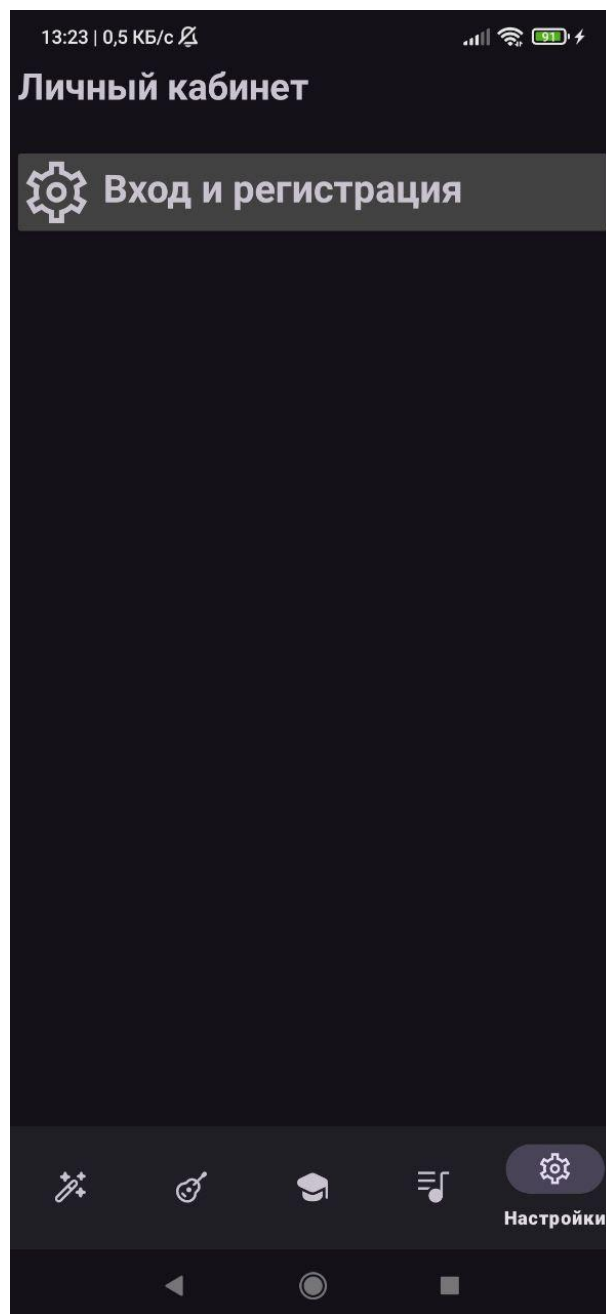
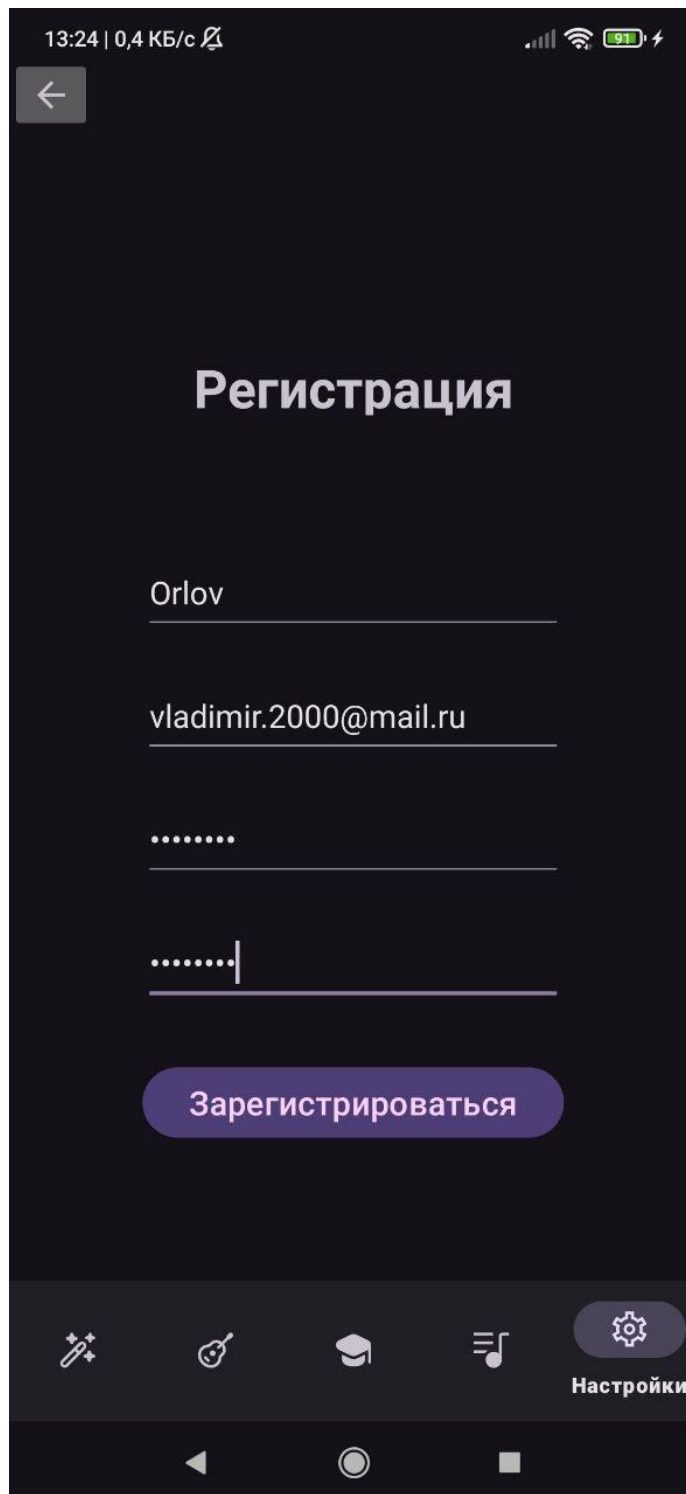


Рисунок 11 – меню настройки для не авторизованного пользователя

Нажав на кнопку «Вход и регистрация» пользователь перейдет к форме авторизации, если у пользователя отсутствует аккаунт при нажатии на кнопку «Зарегистрироваться» он перейдет к форме регистрации. На рисунках 12-18 представлен процесс регистрации и авторизации.



The image shows a mobile application interface for user registration. At the top, the status bar displays the time 13:24, data usage 0,4 KB/c, and a 91% battery level. A back arrow is in the top left corner. The main heading is "Регистрация". Below it are four input fields: a name field containing "Orlov", an email field containing "vladimir.2000@mail.ru", a password field with six dots, and a confirmation password field with six dots and a cursor. A purple button labeled "Зарегистрироваться" is positioned below the password fields. At the bottom, a navigation bar contains five icons: a pencil, a guitar, a graduation cap, a musical note, and a gear. The gear icon is highlighted, and the word "Настройки" is written below it. The bottom of the screen shows the standard Android navigation bar.

13:24 | 0,4 КБ/с

91%

←

Регистрация

Orlov

vladimir.2000@mail.ru

.....

.....|

Зарегистрироваться

Настройки

Рисунок 12 – Форма регистрации нового пользователя в приложении

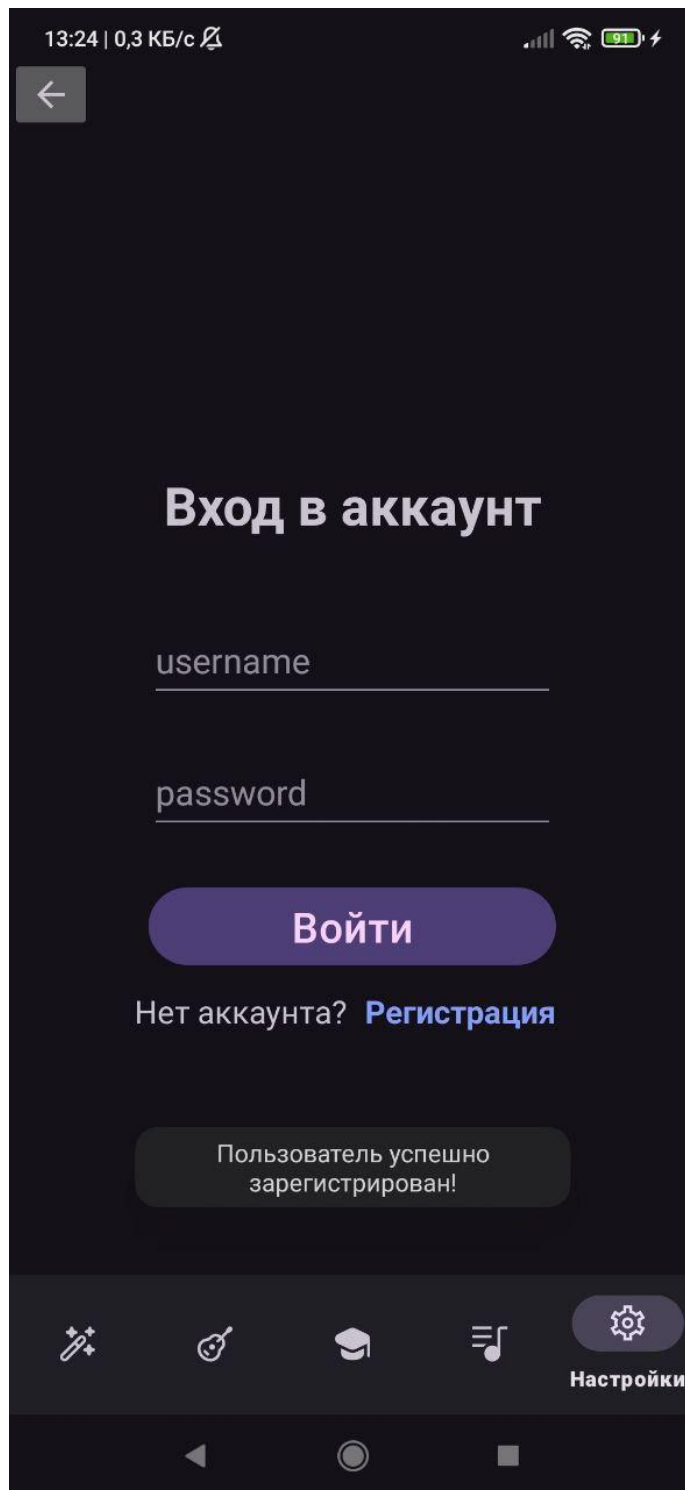


Рисунок 13 – Уведомление о успешной регистрации нового пользователя в приложении

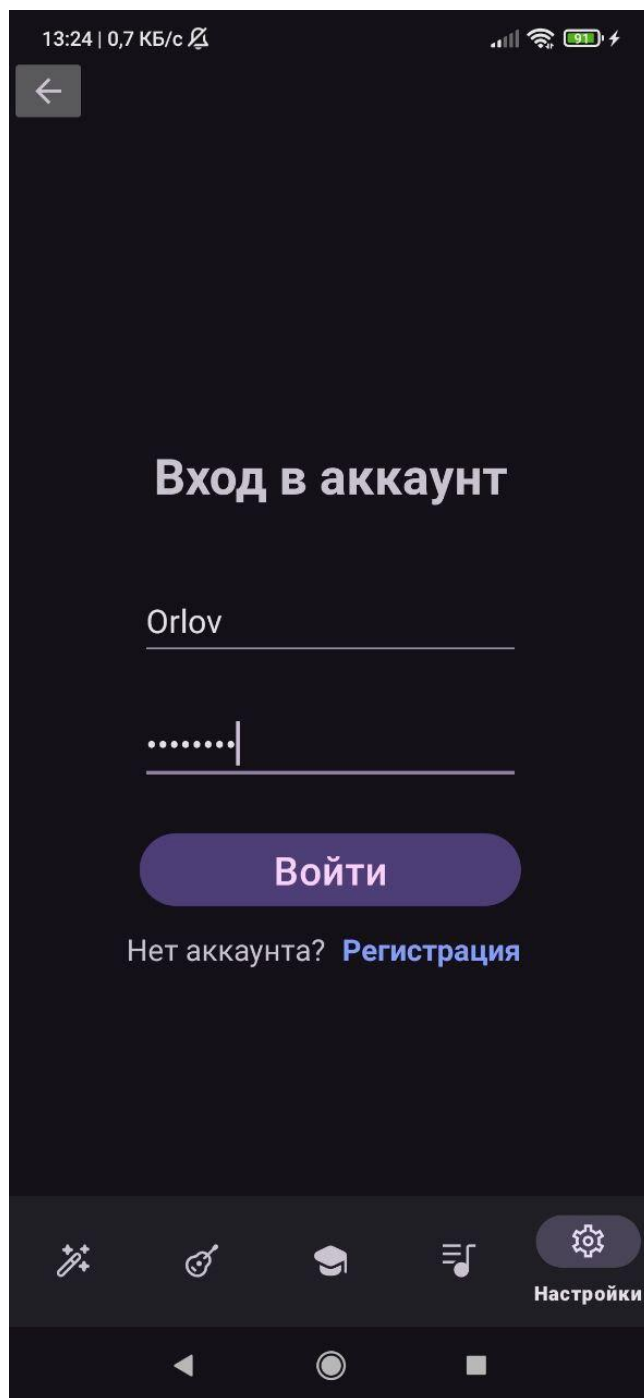


Рисунок 14 – Авторизация пользователя в приложении

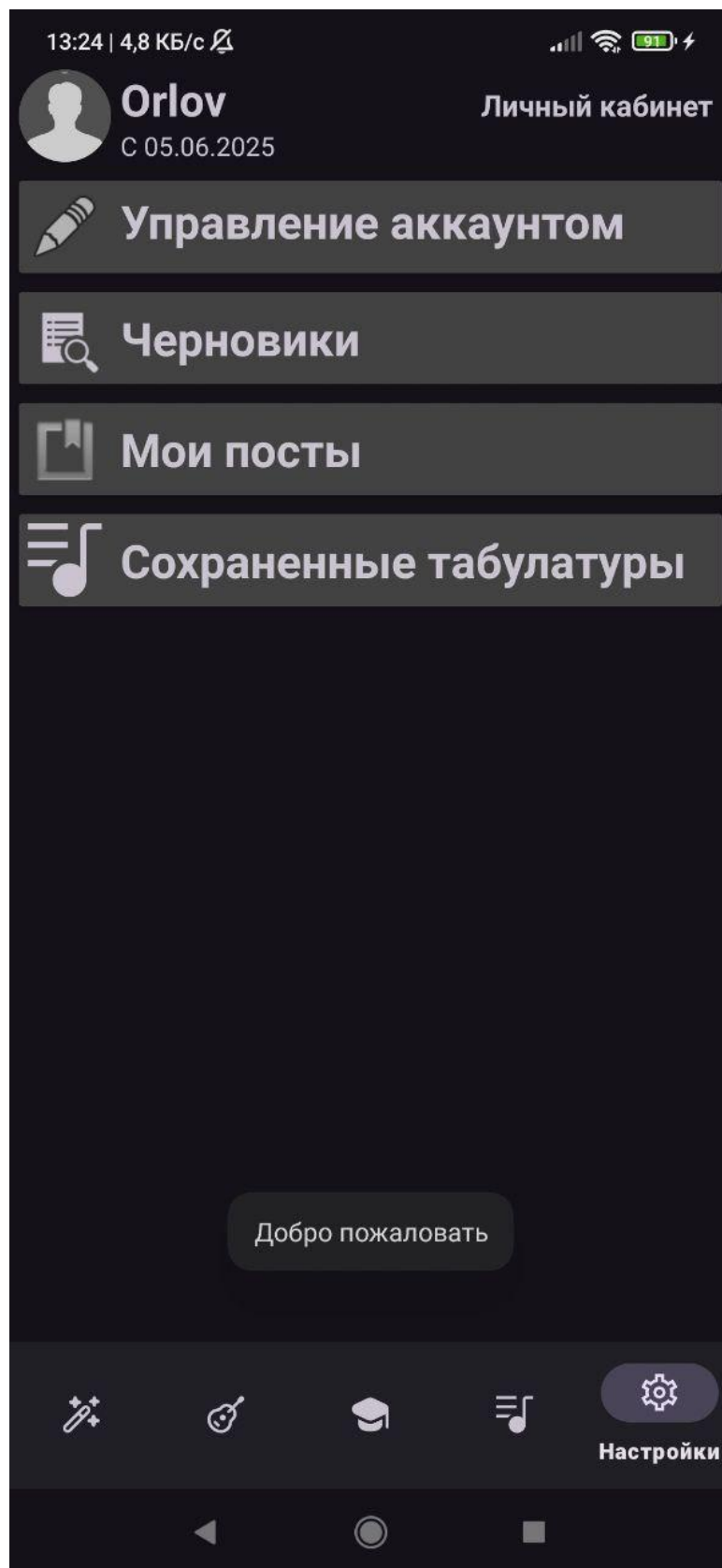


Рисунок 15 – Пользователь попадает в личный кабинет после авторизации

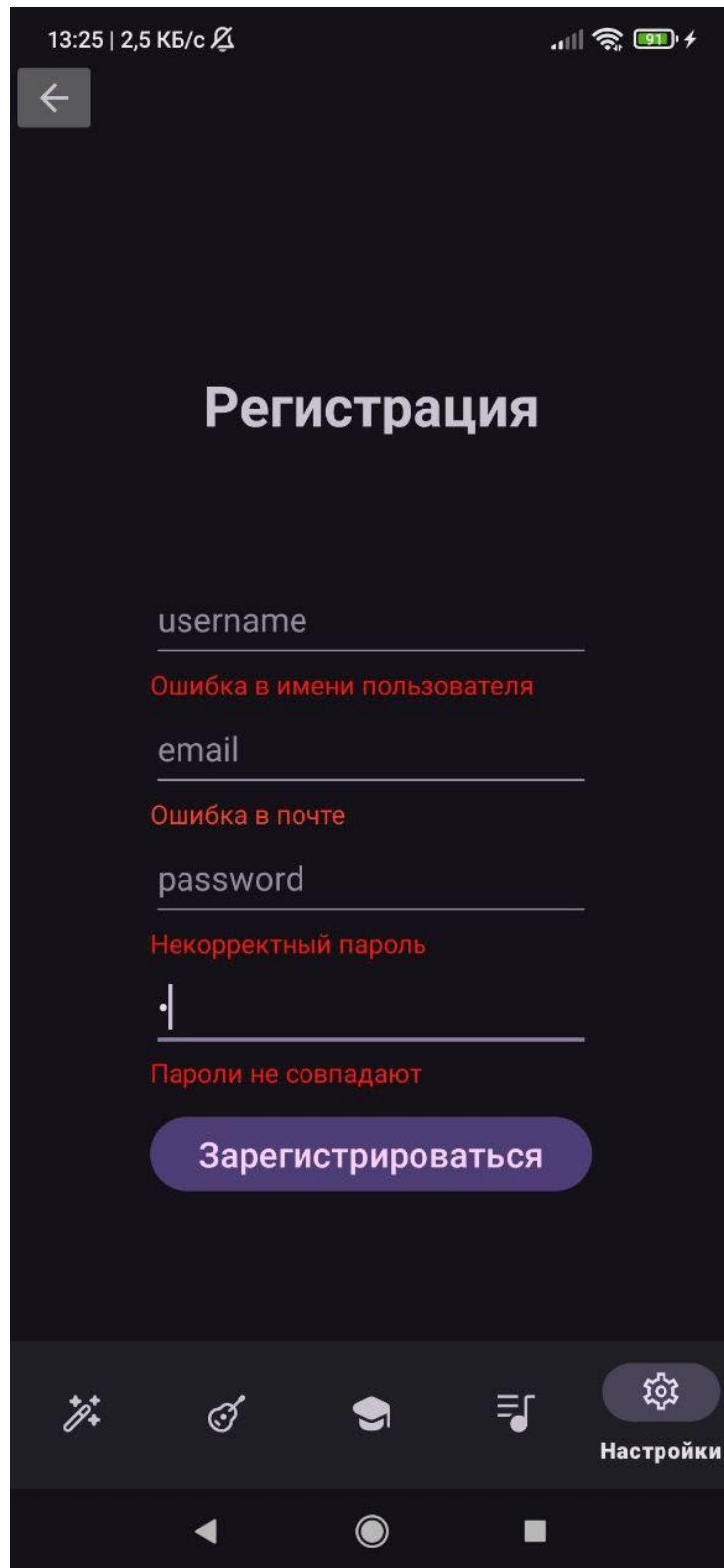


Рисунок 16 – Уведомление о неверно заполненных полях

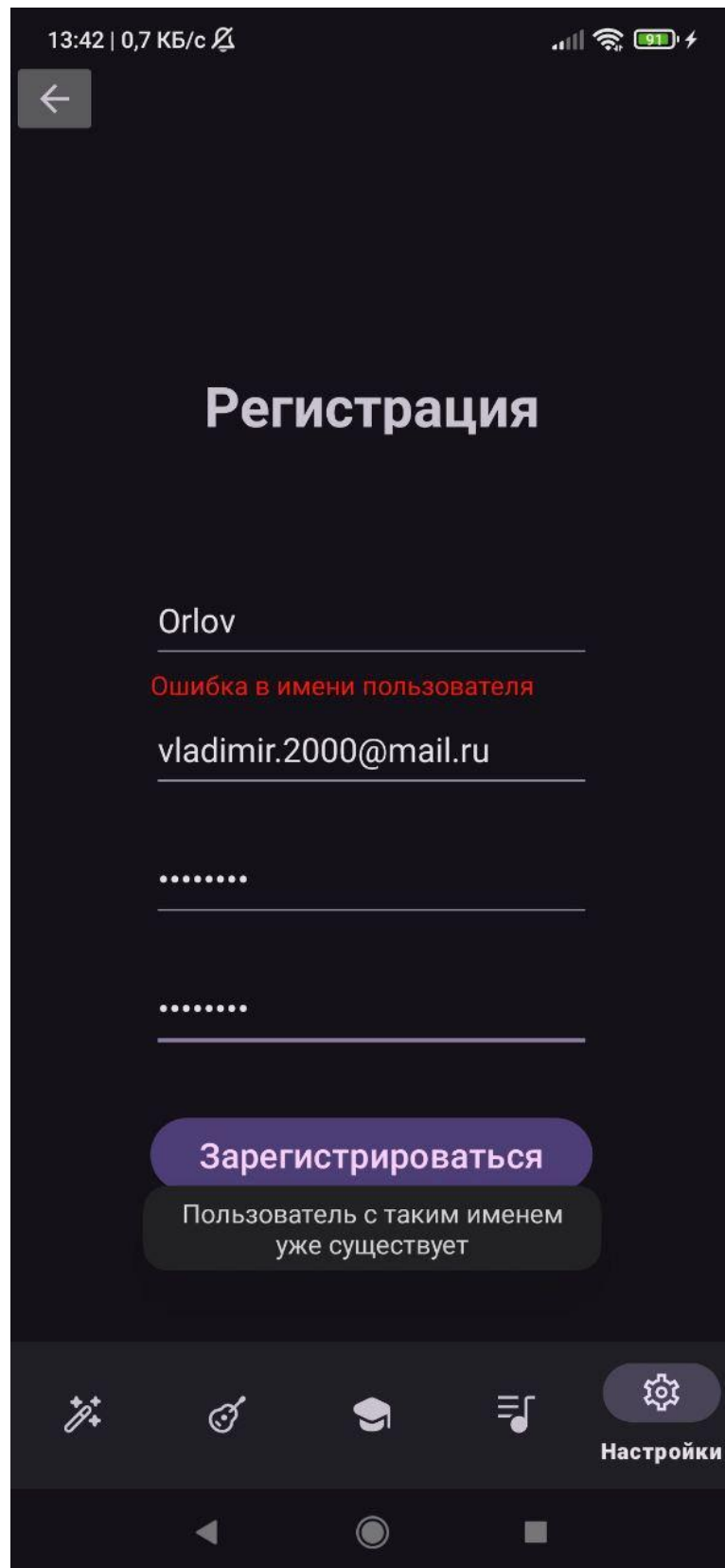


Рисунок 17 – Пользователь с таким именем уже существует

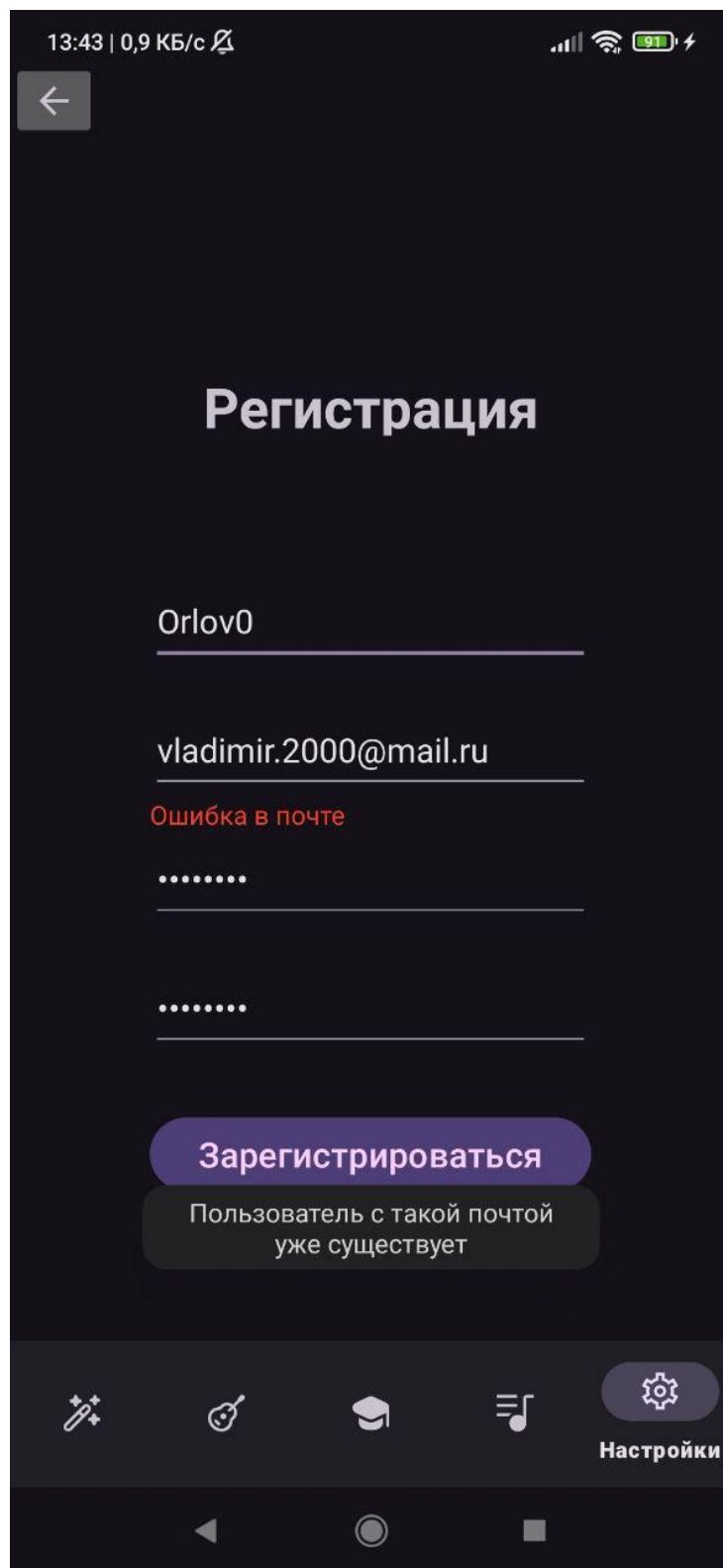


Рисунок 18 – Пользователь с такой почтой уже существует

После успешной регистрации запись о новом пользователе добавляется в базу данных (рисунок 19).

	user_id [PK] bigint	username character varying (255)	email character varying (255)	password character varying (255)
1	1	admin	vladimir.2000@inbox.ru	\$2a\$04\$zq4oX4dZX9TGWmA/BinJ3OHblI9.icUfEfTXRanxLRJAU.rk6/txy
2	2	user2	sanya@inbox.ru	\$2a\$04\$zq4oX4dZX9TGWmA/BinJ3OHblI9.icUfEfTXRanxLRJAU.rk6/txy
3	3	user	vasya@inbox.ru	\$2a\$04\$zq4oX4dZX9TGWmA/BinJ3OHblI9.icUfEfTXRanxLRJAU.rk6/txy
4	4	Orlov	vladimir.2000@mail.ru	\$2a\$10\$TNf09YomrrIQ6g37d2BpPeWMIr/9UdIkUU9TJbTVIaomDR5nCsK...

Рисунок 19 – Фрагмент данных пользователей в таблице «users» базы данных приложения

Если пользователь запускает приложение ранее авторизовавшись он останется в системе пока действуют access и refresh токены.

4.2 Тестирование личного кабинета

Авторизованный пользователь получает доступ личному кабинету со следующими вкладками «Управление аккаунтом», «Черновики», «Мои посты», «Сохраненные табулатуры».

При нажатии на кнопку «Управление аккаунтом» перед пользователем откроется профиль, представленный на рисунке 20.

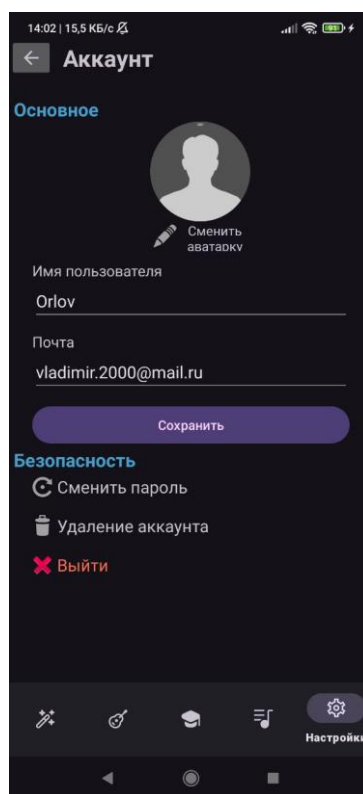


Рисунок 20 – Профиль пользователя в приложении

Здесь у пользователя есть возможность поменять изображение профиля и имя пользователя с почтой. Так же пользователь может поменять пароль, выйти из аккаунта или удалить его. Тестирование функций представлена на рисунках 21-27.

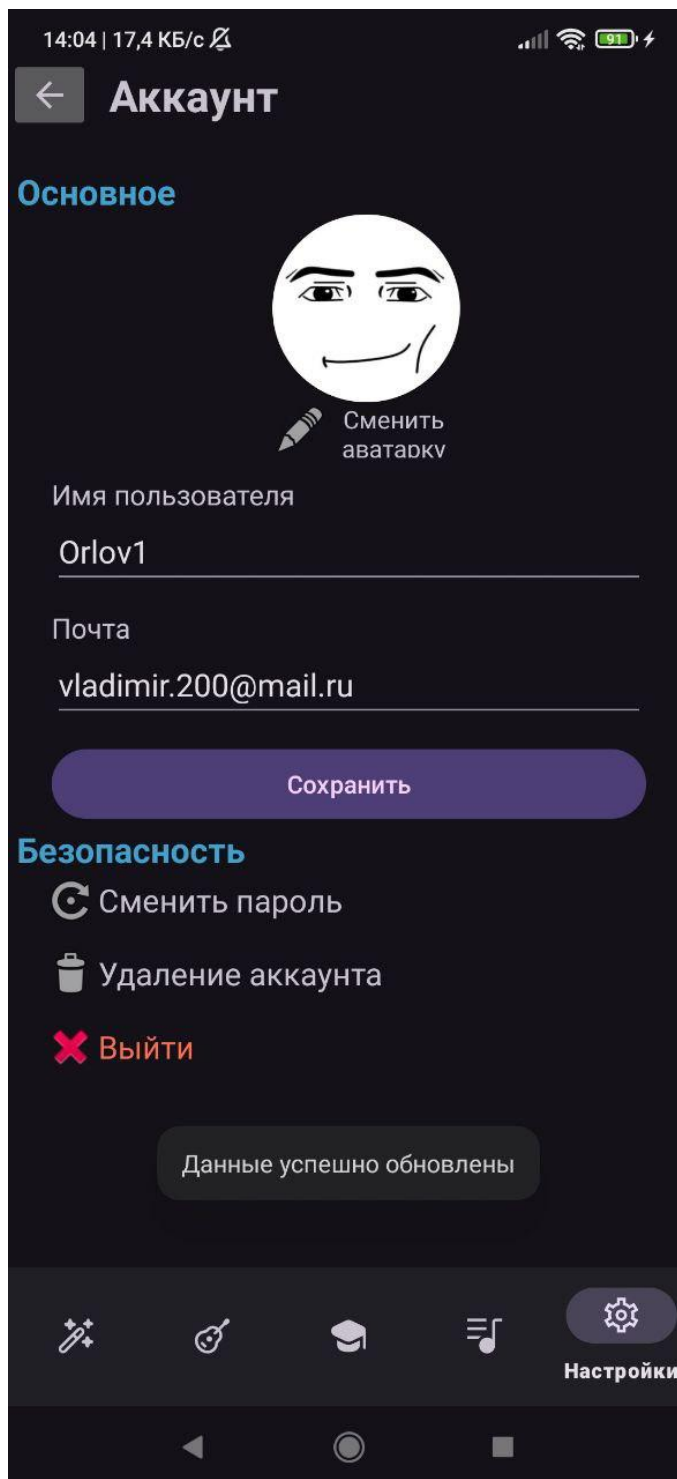


Рисунок 21 – Редактирование изображения, имени и почты пользователя

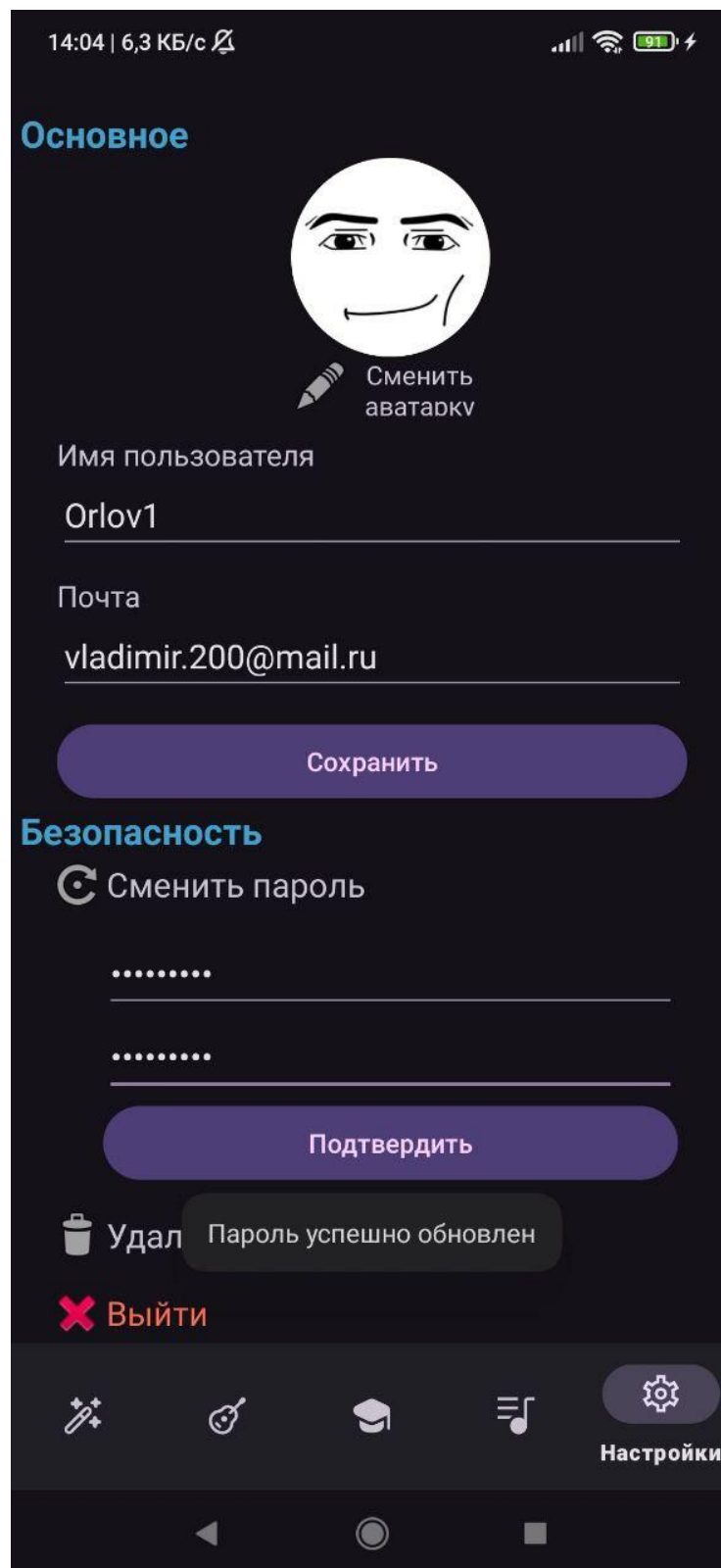


Рисунок 22 – Изменение пароля пользователя

	user_id [PK] bigint	username character varying (255)	email character varying (255)	password character varying (255)	registration_date timestamp with time zone	avatar_picture character varying (255)
1	1	admin	vladimir.2000@inbox.ru	\$2a\$04\$zq4oX4dZX9TGWmA/BinJ30Hbll9.icUfEFTXRanxLRJAU.rk6/txy	2025-06-05 13:23:08.110169+03	[null]
2	2	user2	sanya@inbox.ru	\$2a\$04\$zq4oX4dZX9TGWmA/BinJ30Hbll9.icUfEFTXRanxLRJAU.rk6/txy	2025-06-05 13:23:08.110169+03	[null]
3	3	user	vasya@inbox.ru	\$2a\$04\$zq4oX4dZX9TGWmA/BinJ30Hbll9.icUfEFTXRanxLRJAU.rk6/txy	2025-06-05 13:23:08.110169+03	[null]
4	4	Orlov1	vladimir.200@mail.ru	\$2a\$10\$/NdFvoJE7SHCIARGK8Ms.uZilxXymFNONmSnZeM.AEF.TE00tT...	2025-06-05 13:24:43.481673+03	http://192.168.31.139:8080/api/v1/uploadma

Рисунок 23 – Изменение данных пользователя в таблицы «users» БД

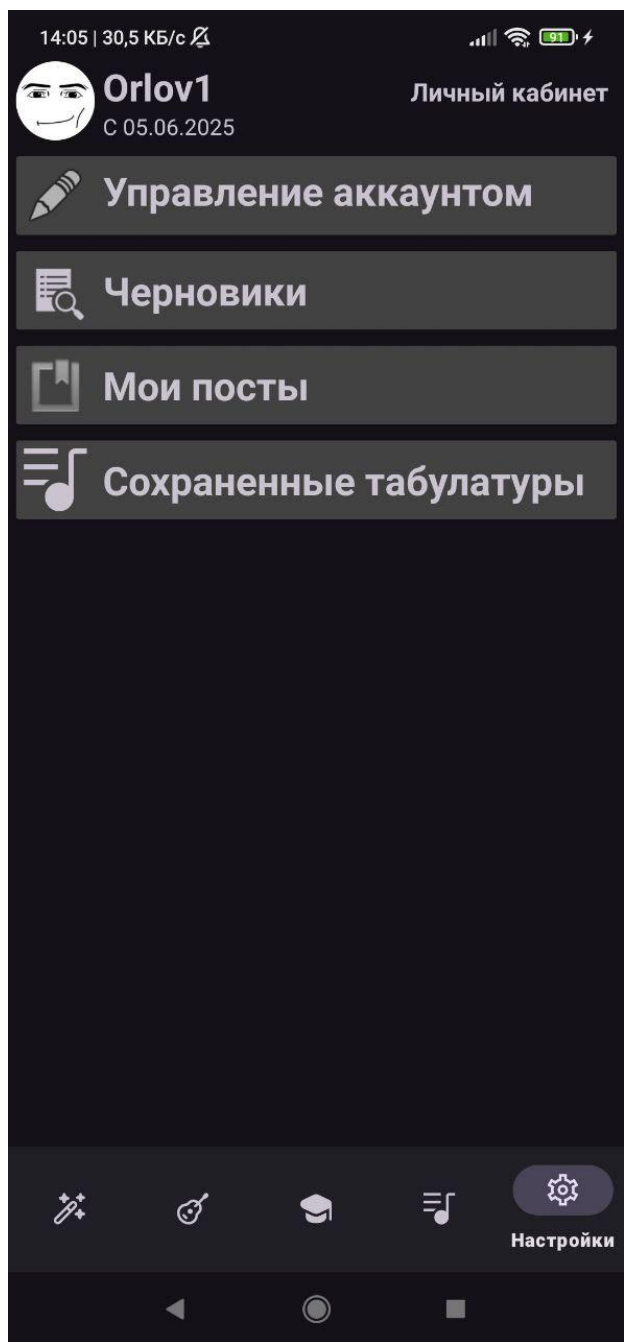


Рисунок 24 – Измененные данные в личном кабинете

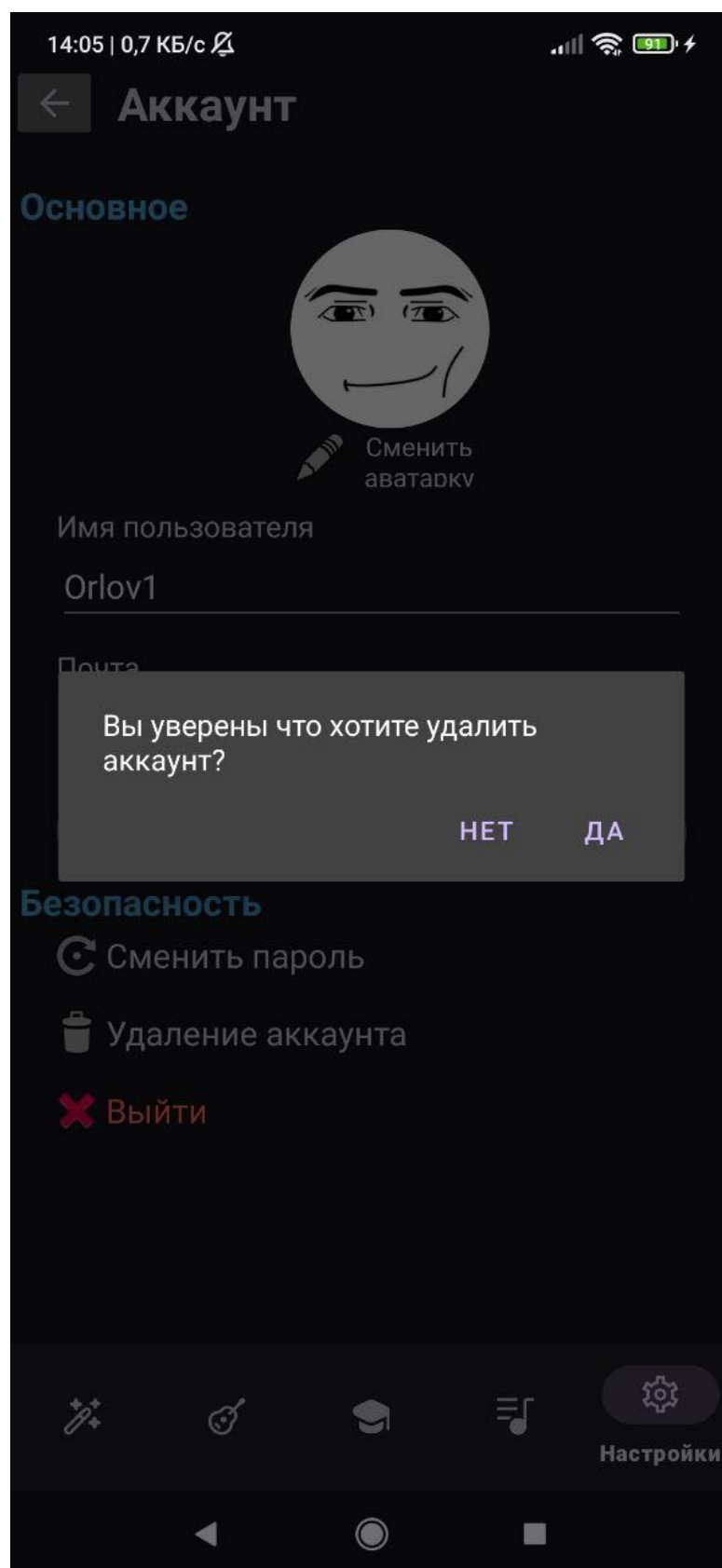


Рисунок 25 – Подтверждение удаления аккаунта

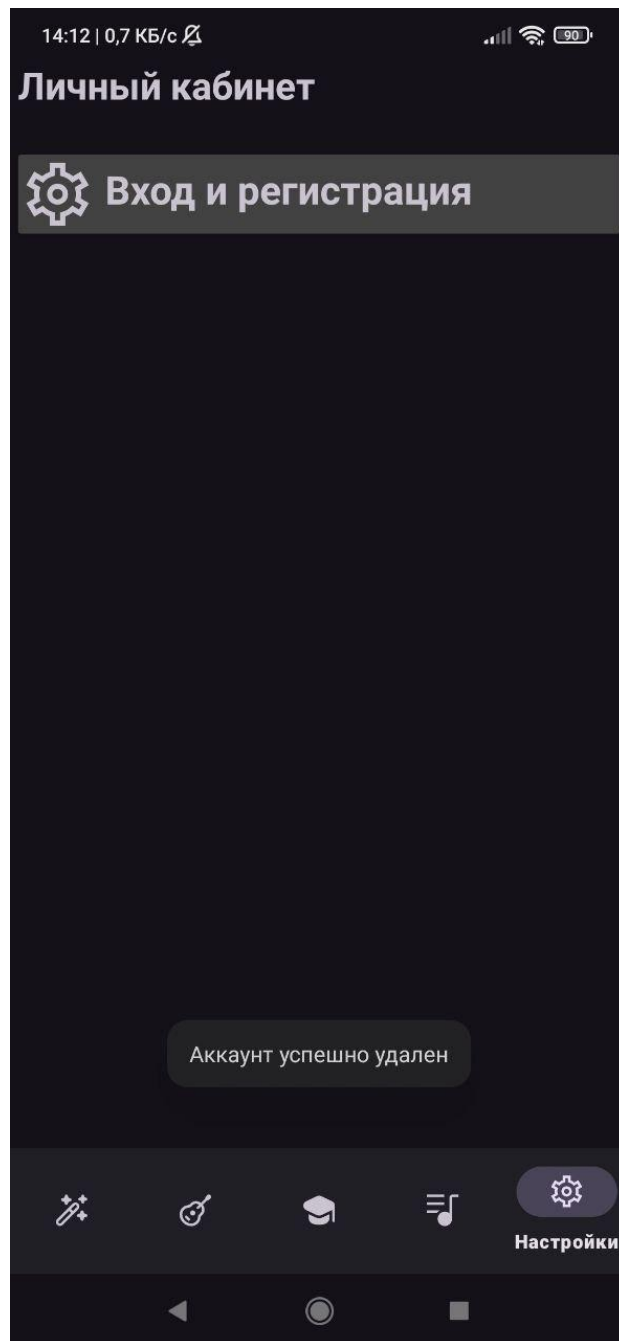


Рисунок 26 – Уведомление о удалении аккаунта

	user_id [PK] bigint	username character varying (255)	email character varying (255)
1	1	admin	vladimir.2000@inbox.ru
2	2	user2	sanya@inbox.ru
3	3	user	vasya@inbox.ru

Рисунок 27 – Демонстрация удаления записи о пользователе из БД

При нажатии из личного кабинета кнопки «Черновики» откроется фрагмент, представленный на рисунке 28.

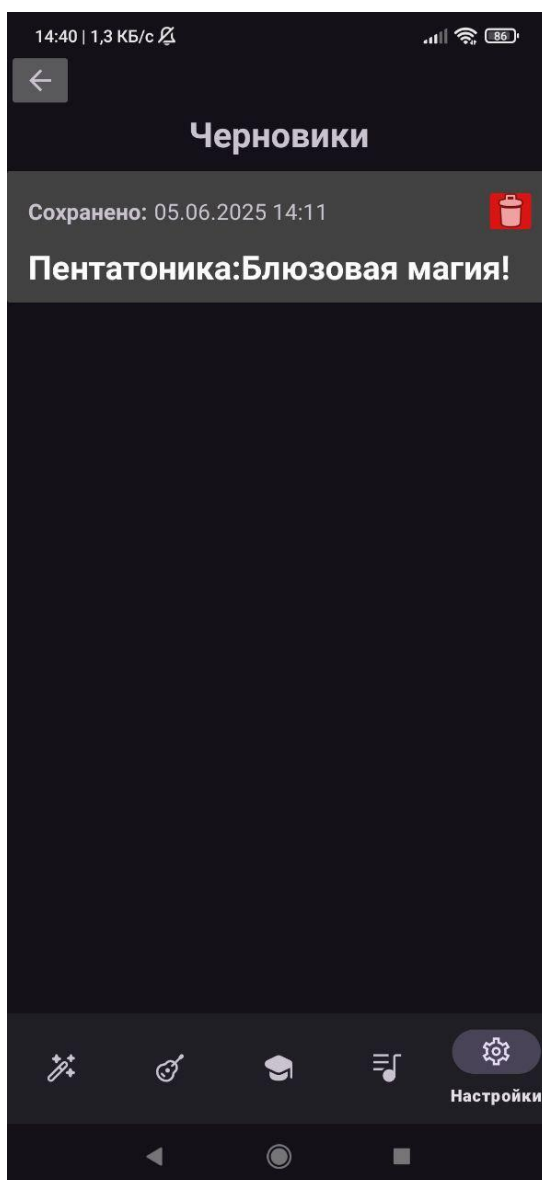


Рисунок 28 – Лента черновиков из личного кабинета

Здесь находятся все сохраненные черновики пользователя нажав на него откроется редактор, при нажатии на красную иконку с корзиной черновик будет удален(рисунки 29 – 30)

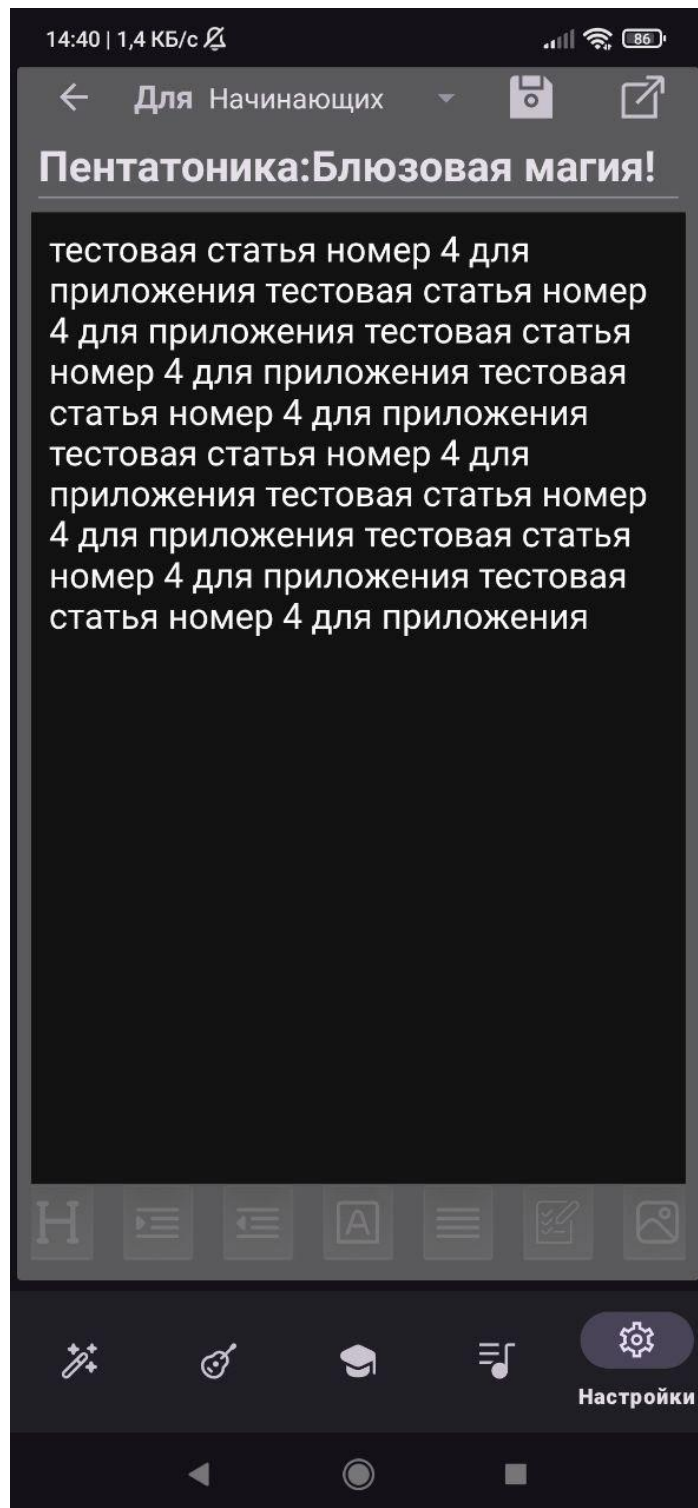


Рисунок 29 – Редактор с открытым черновиком

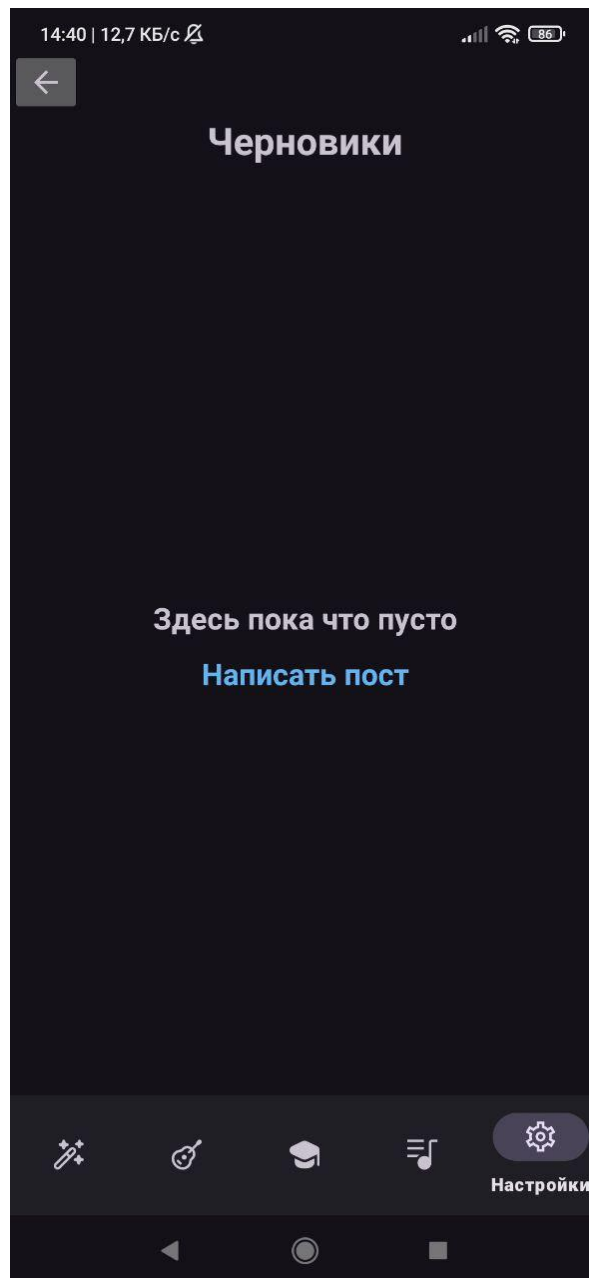


Рисунок 30 – Демонстрация удаления черновика

При нажатии кнопки «Мои посты» откроется аналогичный экран, но с опубликованными постами пользователя (Рисунки 31 -32).

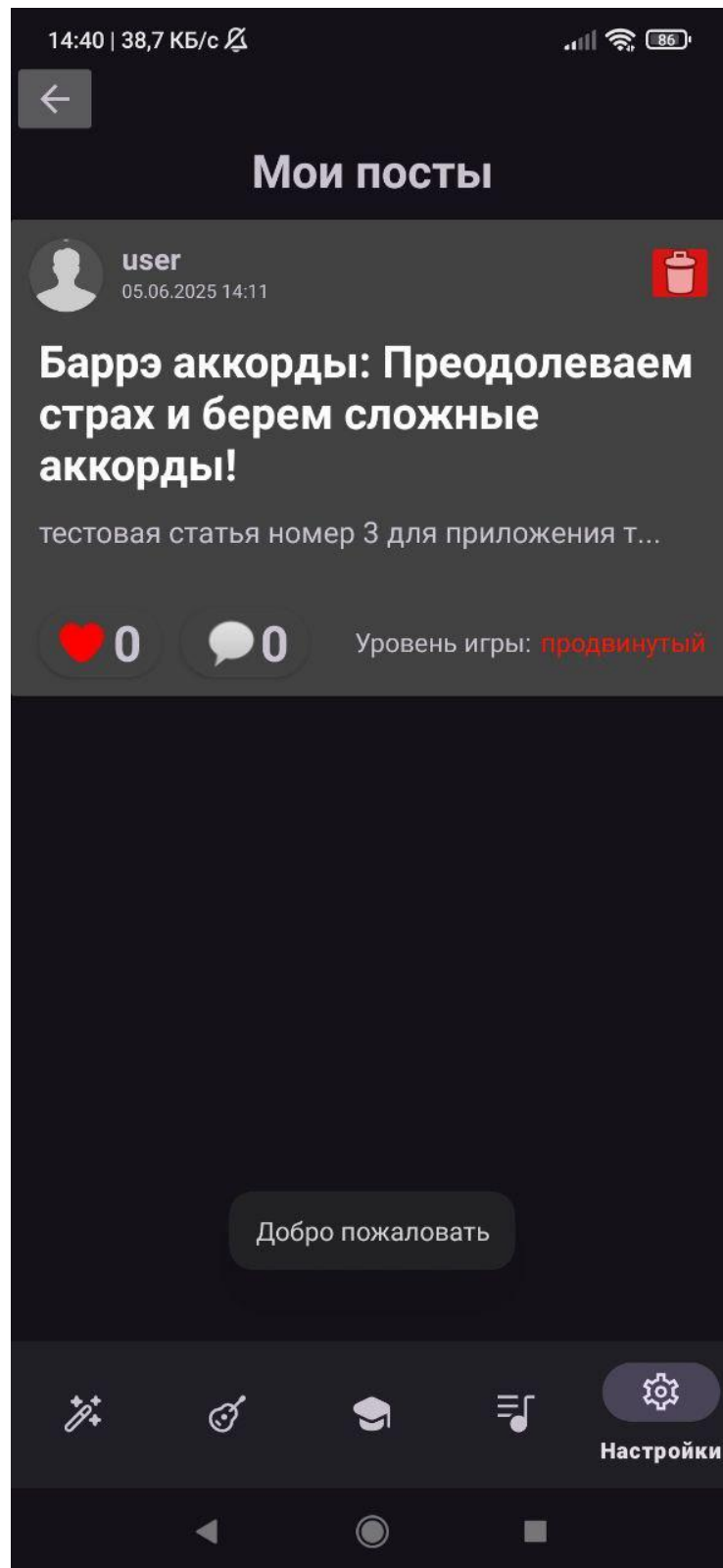


Рисунок 31 – Опубликованные посты пользователя

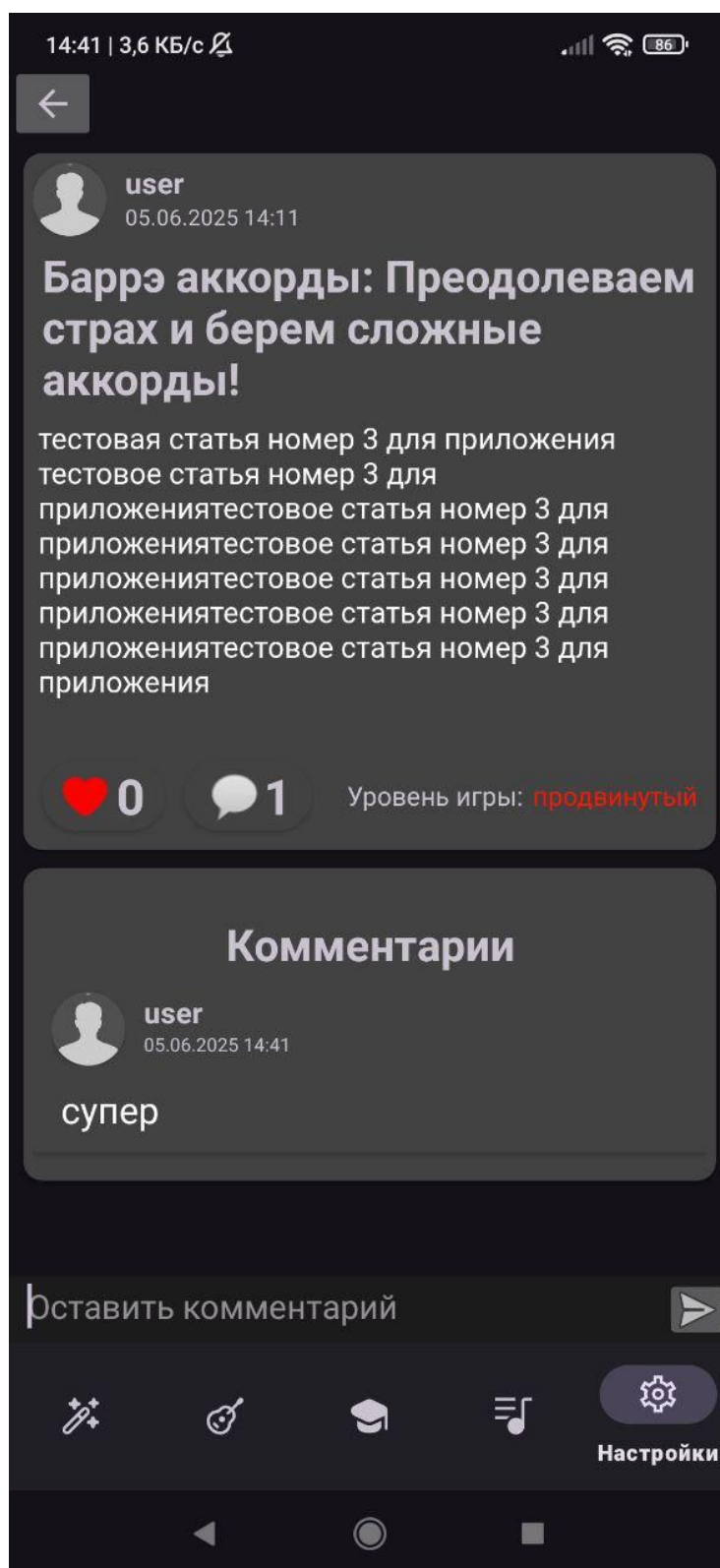


Рисунок 32 – переход к посту пользователя

При нажатии на кнопку «Сохраненные табулатуры» откроется окно, представленное на рисунке 33. В нем при нажатии на элемент списка раскроется конкретная табулатура. Так же нажав на кнопку со стрелкой можно

опубликовать или распубликовать табулатуру, которая будет отображаться всем пользователям во вкладке меню «Табулатуры». При нажатии на кнопку «PDF» табулатура сохранится на устройстве в виде pdf файла. При нажатии на красную кнопку с корзиной табулатура будет удалена (Рисунки 34 – 39).

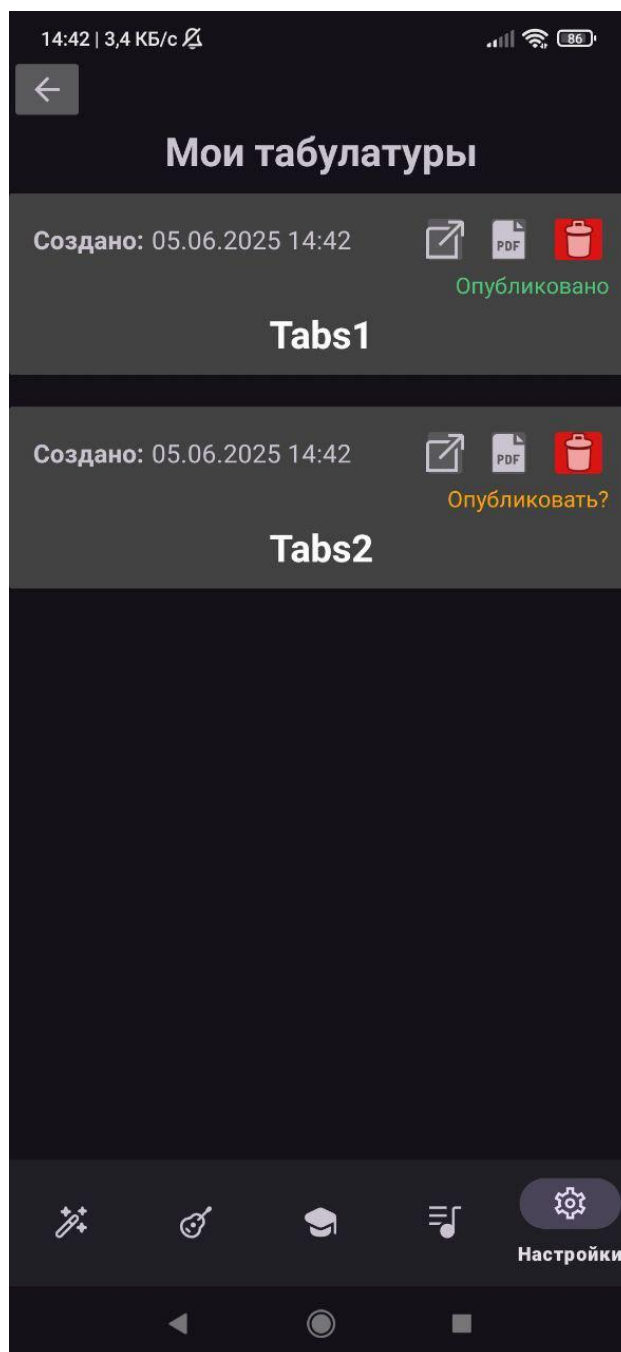


Рисунок 33 – Страница с табулатурами пользователя

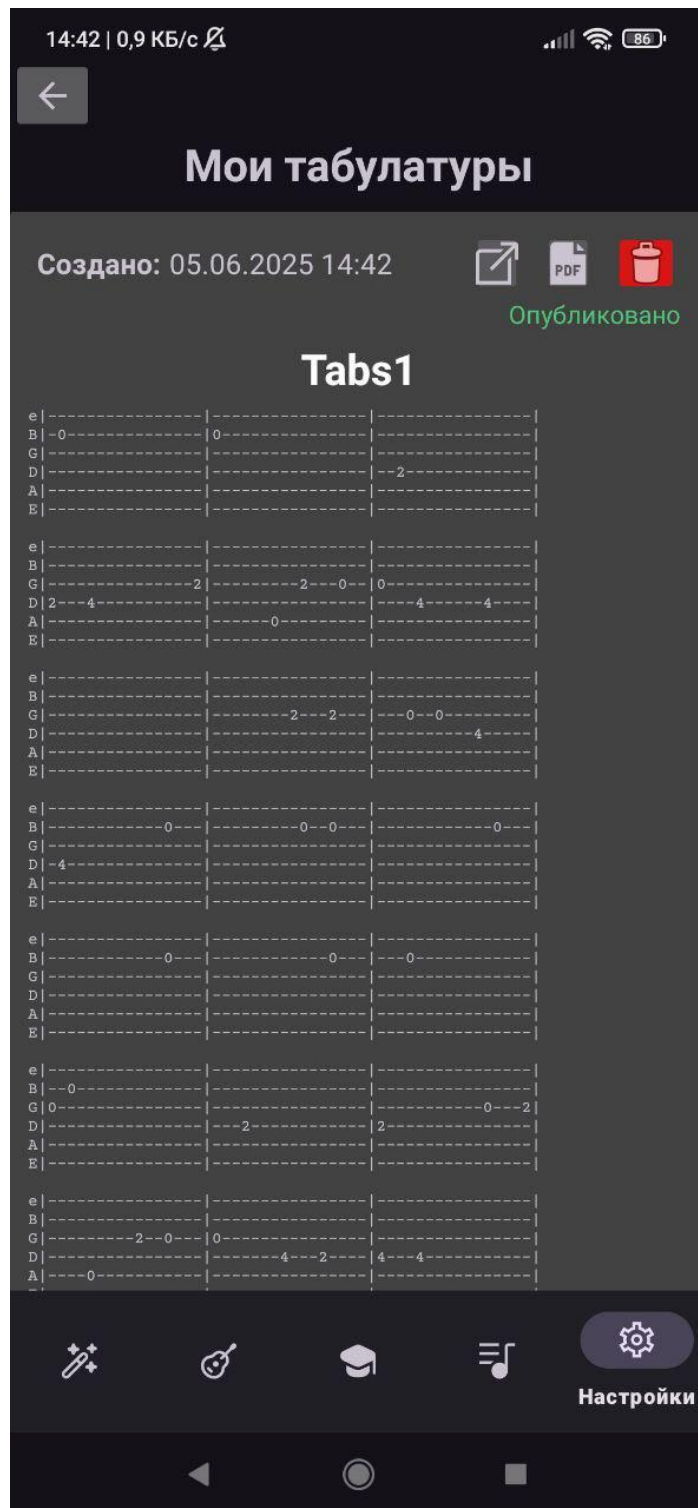


Рисунок 34 – Раскрывающийся элемент списка

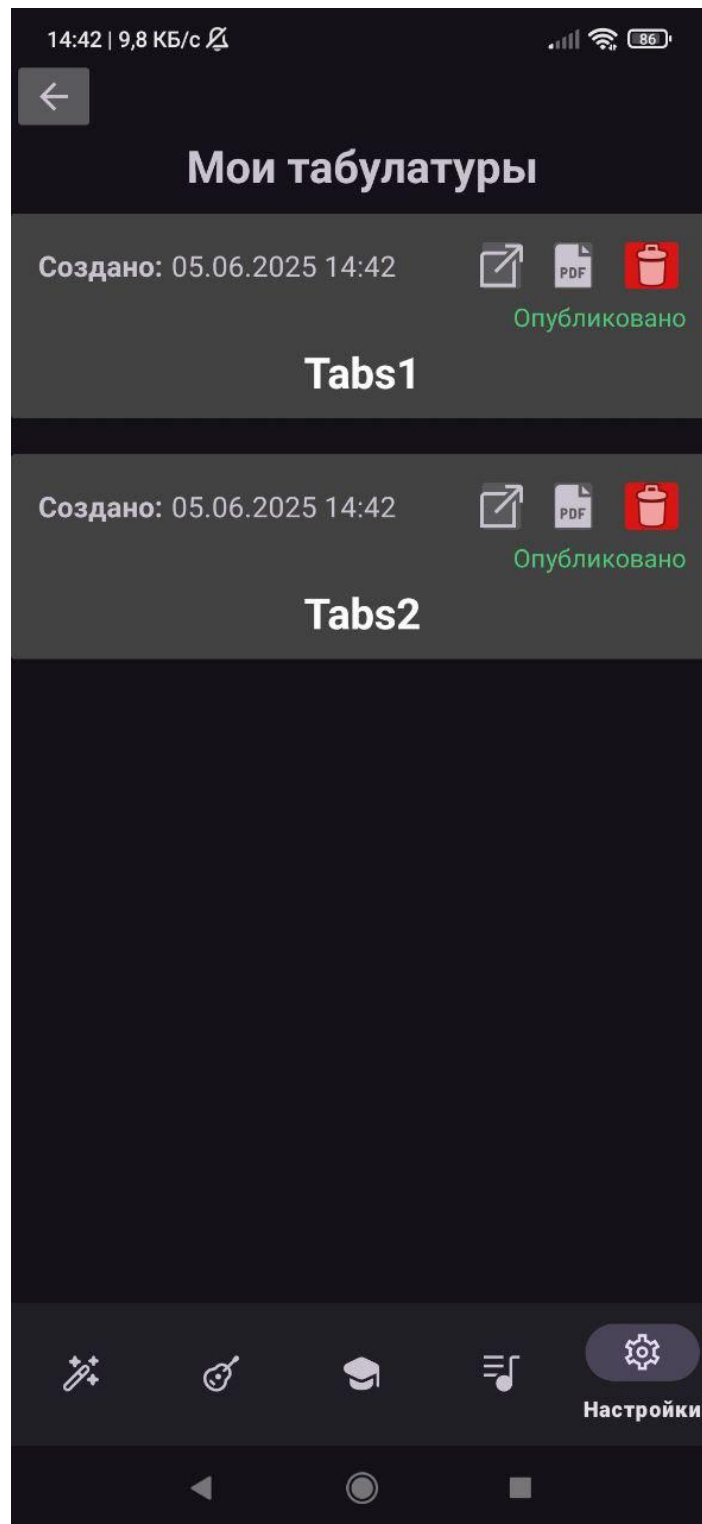


Рисунок 35 – Нажатие кнопки «опубликовать» у табулатуры

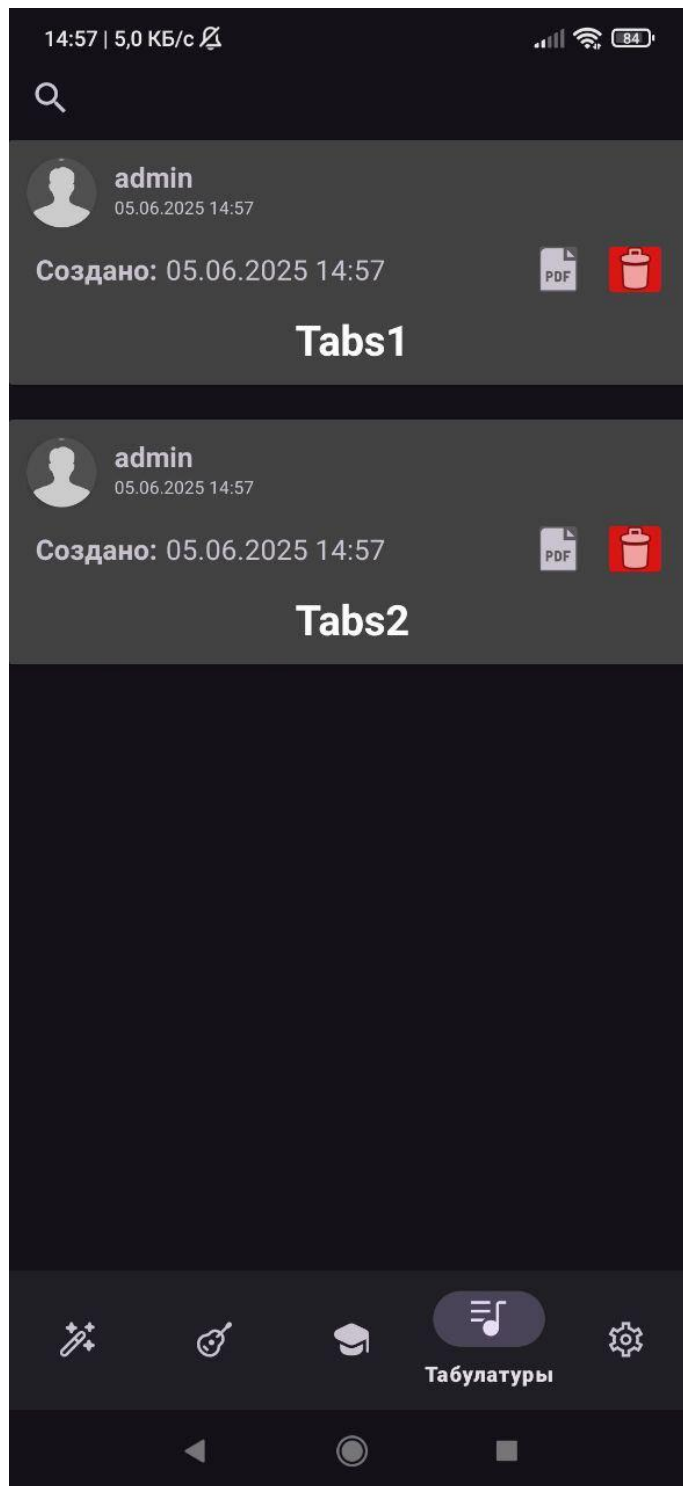


Рисунок 36 – Табулатура появилась в общей ленте

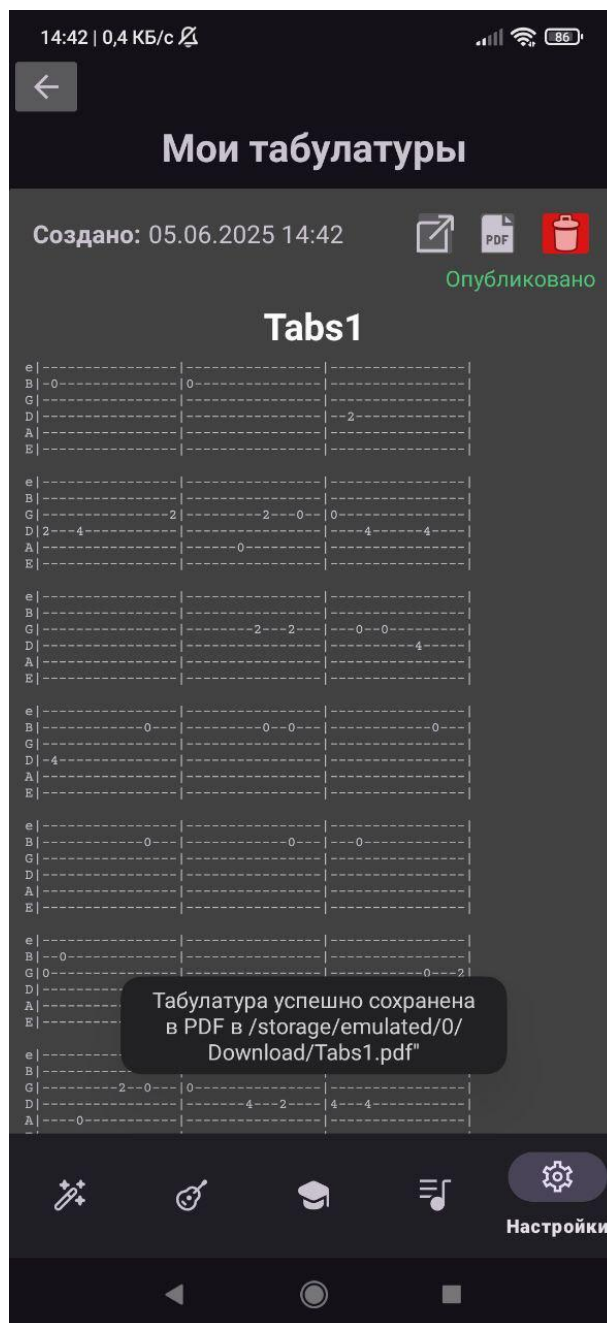


Рисунок 37 – Сохранение табулатуры на устройство

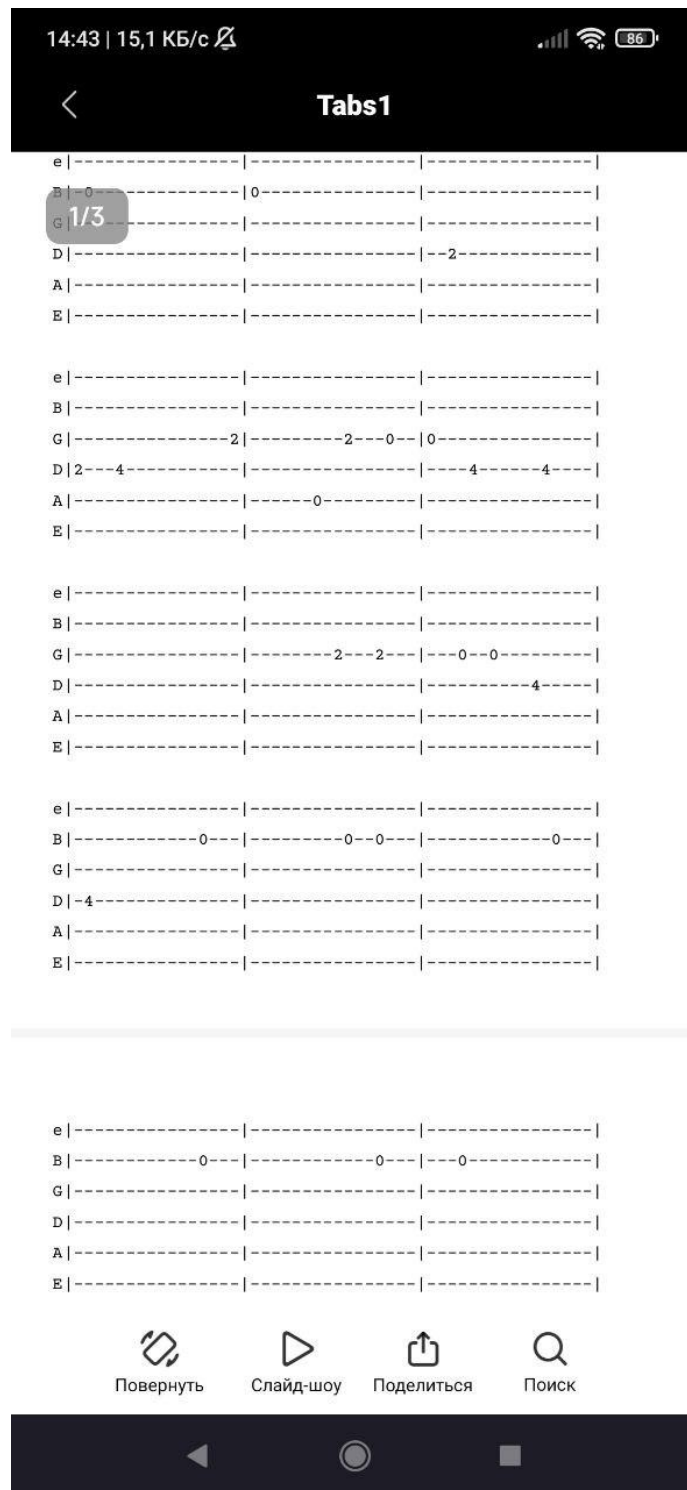


Рисунок 38 – Открытие pdf файла с табулатурой из файловой системы андроида

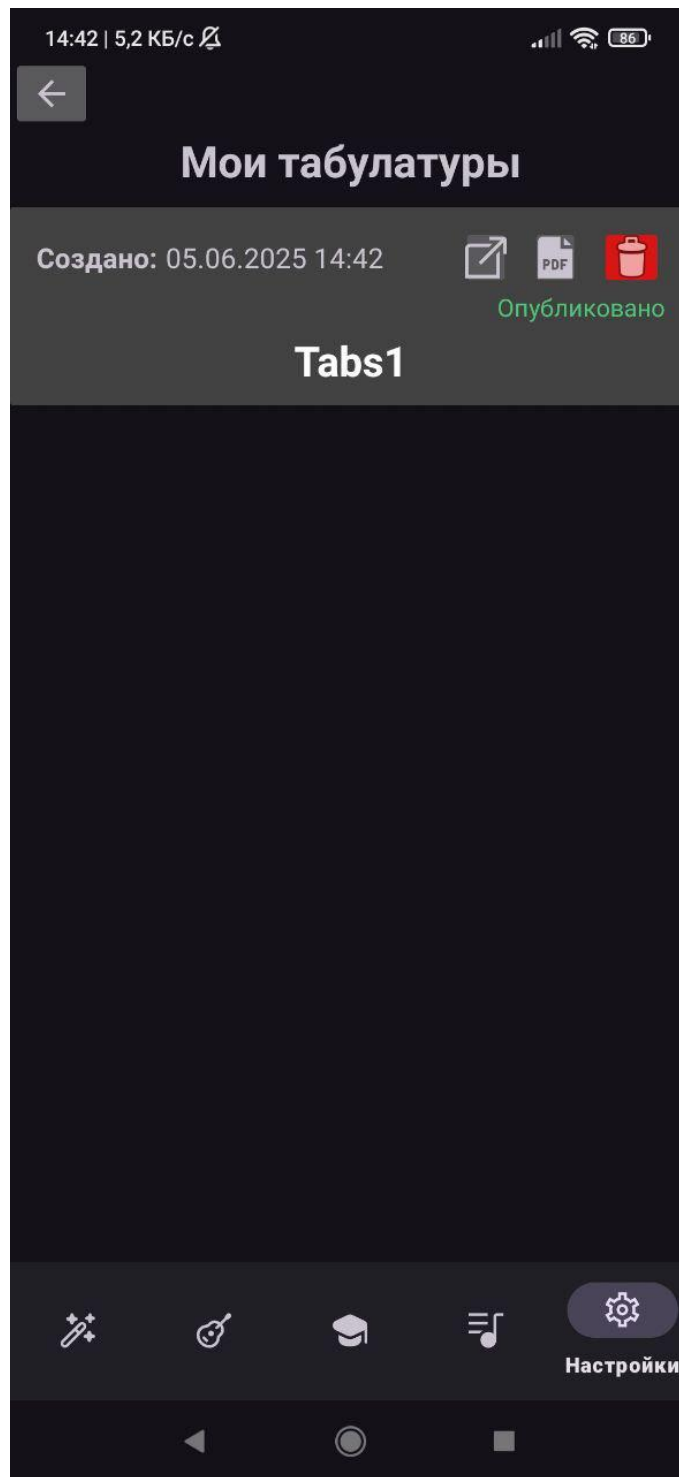


Рисунок 39 – Удаление табулатуры

4.3 Тестирование ленты и редактора уроков

При выборе на вкладке меню кнопку «Все уроки» или при запуске приложения откроется лента с уроками, представленная на рисунке 40. Здесь имеется возможность отсортировать уроки по уровню сложности, рейтингу или дате публикации, отыскать нужный урок по названию, так же можно

открыть посты пользователей. Если пользователь авторизован он может оценить урок нажав на кнопку с сердцем. При нажатии на конкретный урок он откроется и будет показан полный текст статьи, а также комментарии пользователей. (рисунки 41 – 43)

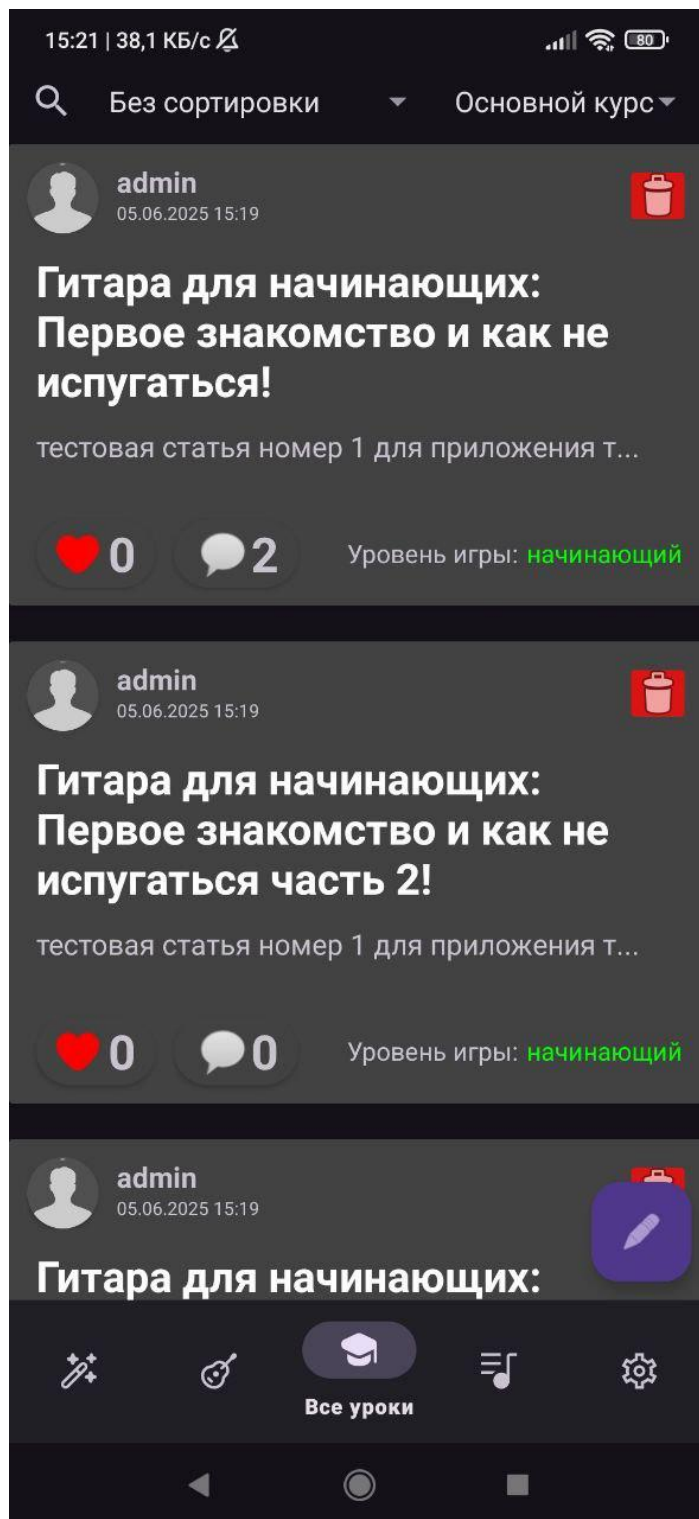


Рисунок 40 – Лента основных уроков от администрации

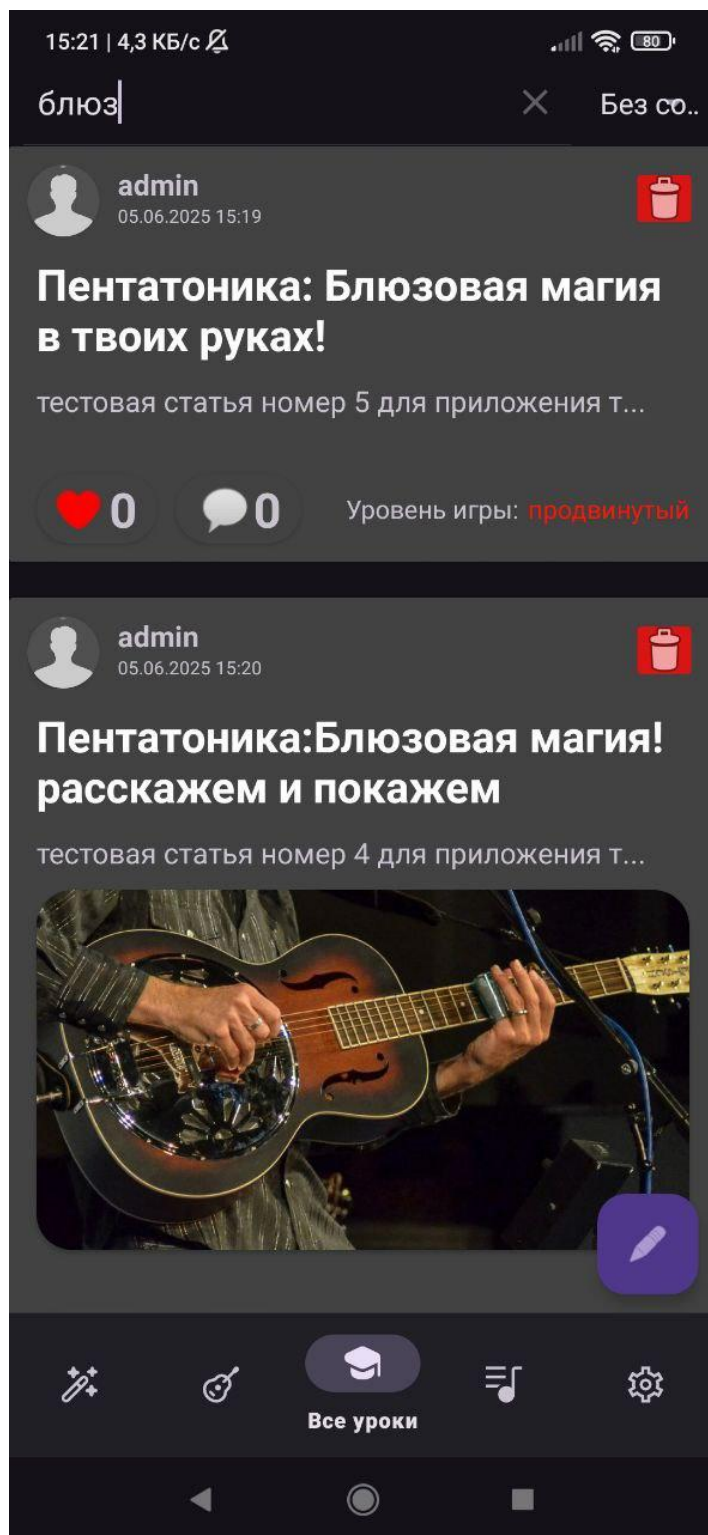


Рисунок 41 – Поиск по названию

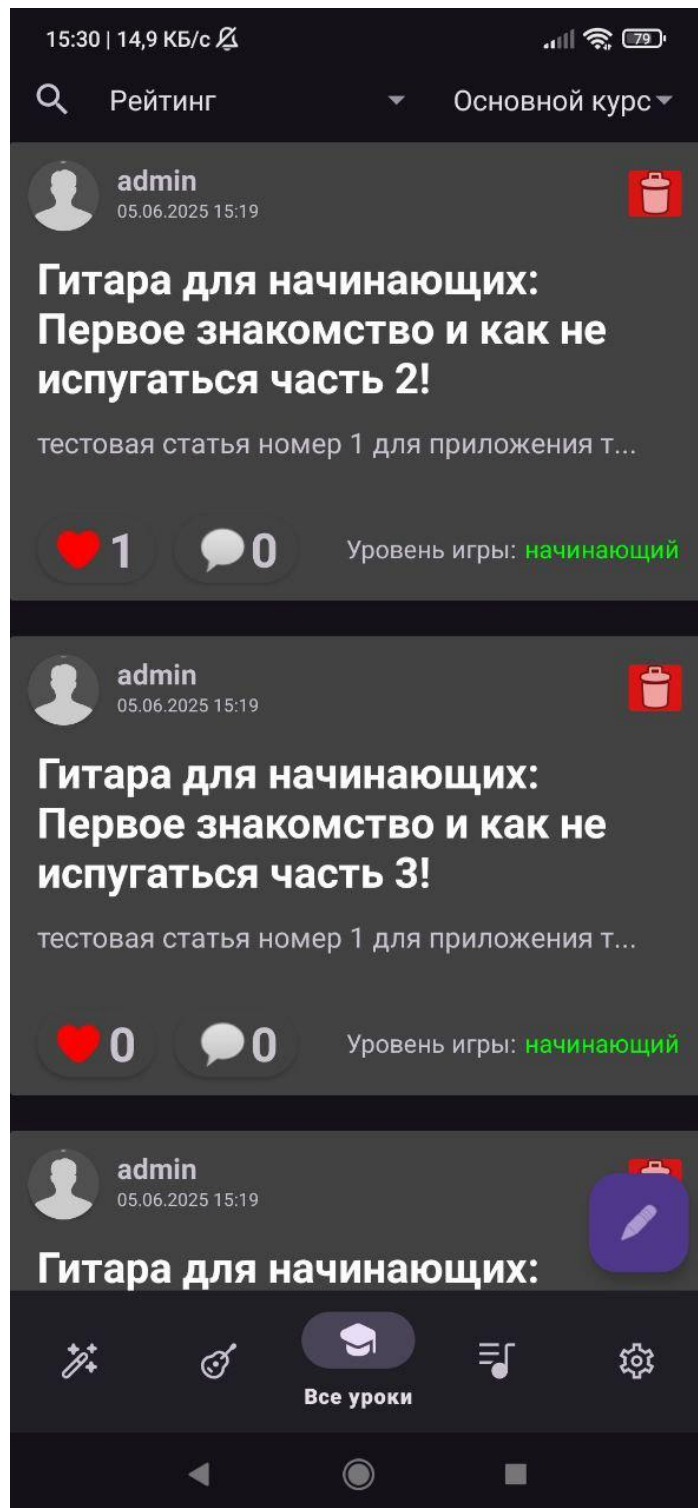


Рисунок 42 – Сортировка по рейтингу

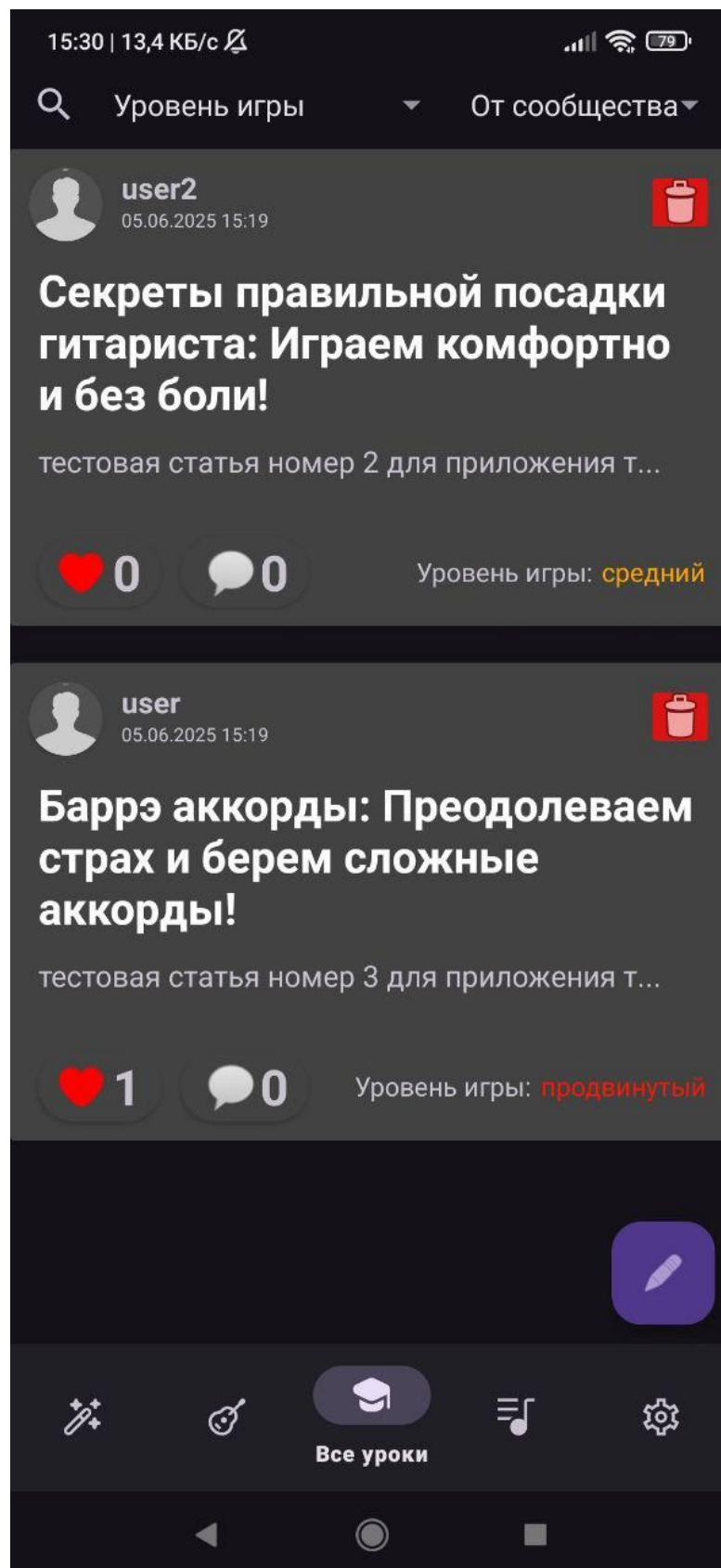


Рисунок 43 – посты от сообщества

Если нажать на фиолетовую кнопку с карандашом откроется редактор статей, если пользователь авторизован, в другом случае произойдет переход к форме авторизации (Рисунок 44-45).

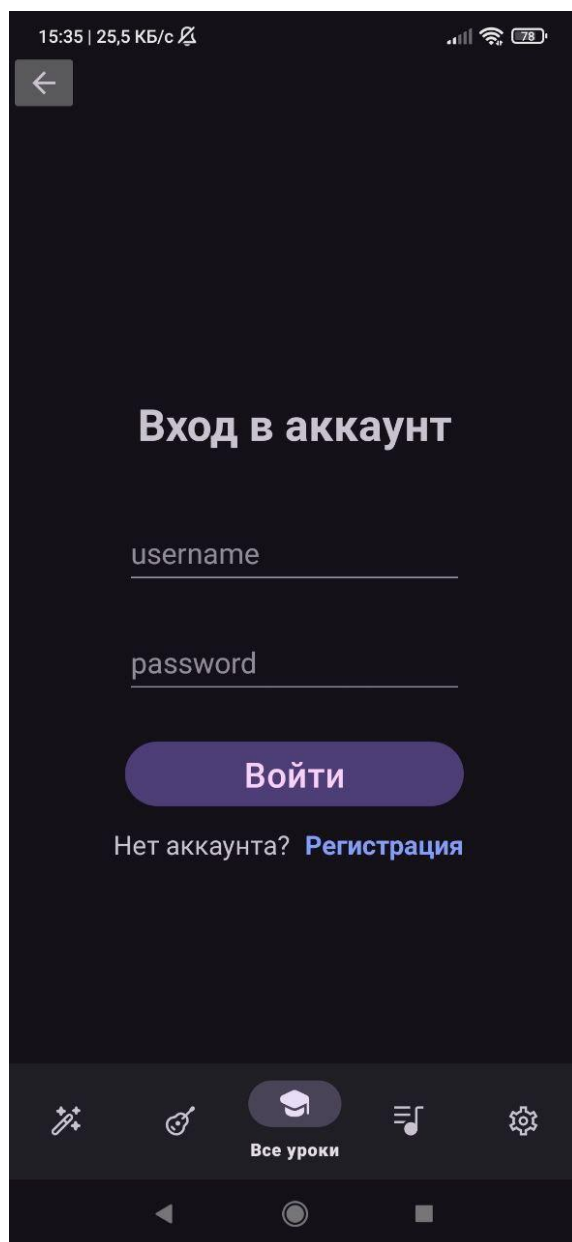


Рисунок 44 – Переход к форме входа при нажатии на кнопку неавторизованным пользователем

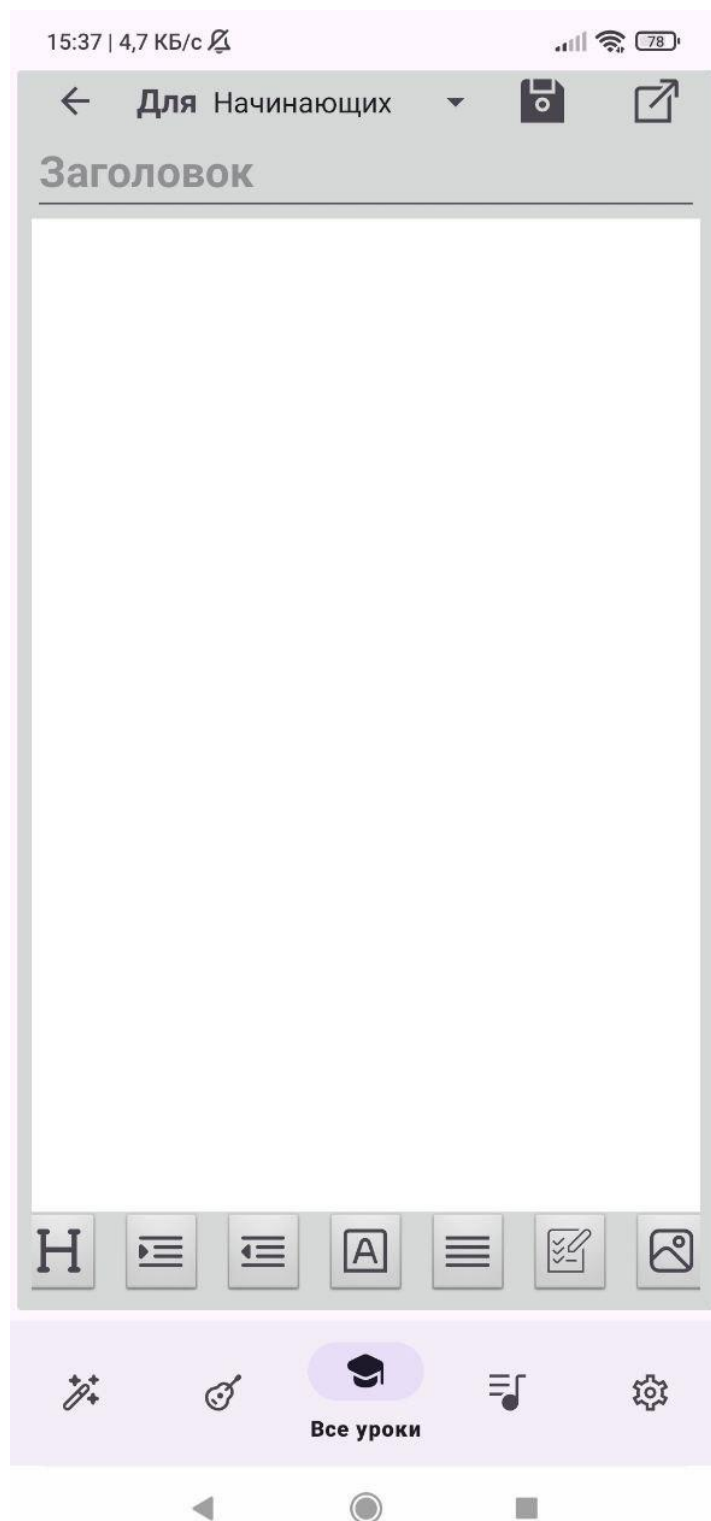


Рисунок 45 – Окно редактора

В верхней панели расположены кнопки «назад» для выхода из редактора при которой будет предложено сохранение черновика, выбор для кого

предназначена статья, кнопка сохранения статьи в черновик и кнопка публикации.

Ниже расположен заголовок статьи и основная часть.

Внизу расположены кнопки инструментов редактора: заголовки, отступы, выравнивание, списки и вставка изображения. Тестирование работы редактора представлено на рисунках 46 – 52.



Рисунок 46 – Тестирование возможностей редактора

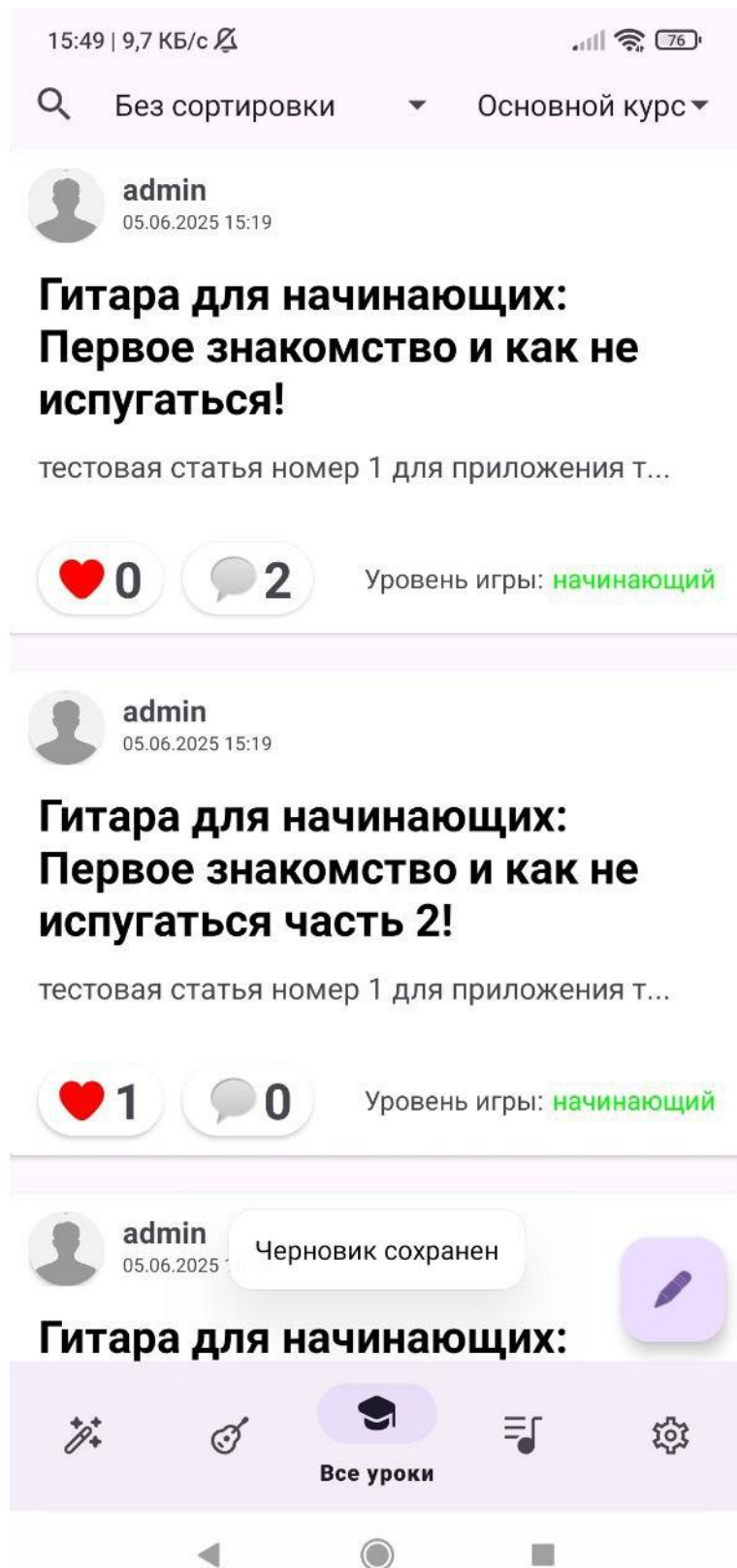


Рисунок 47 – Уведомление о сохранении черновика

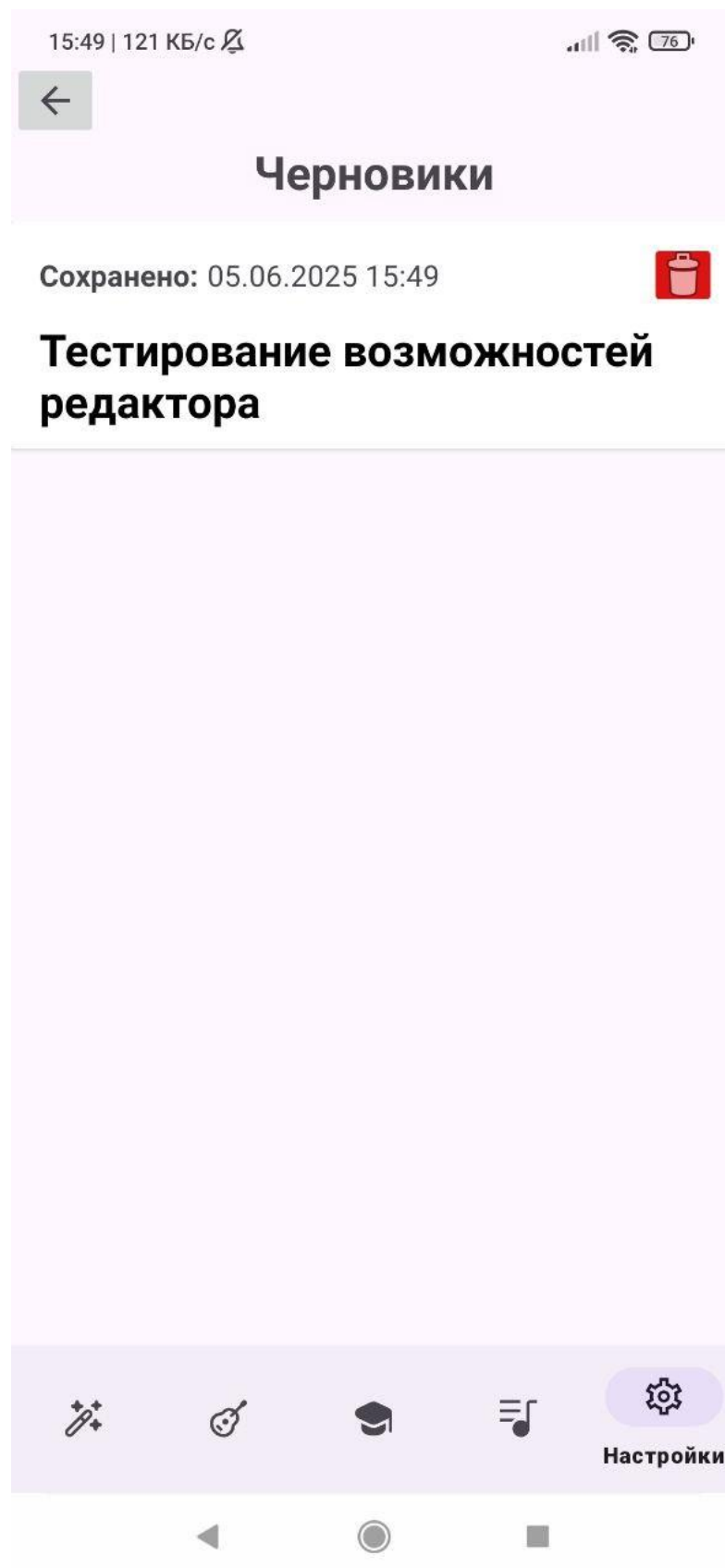


Рисунок 48 – Сохраненный черновик из личного кабинета пользователя

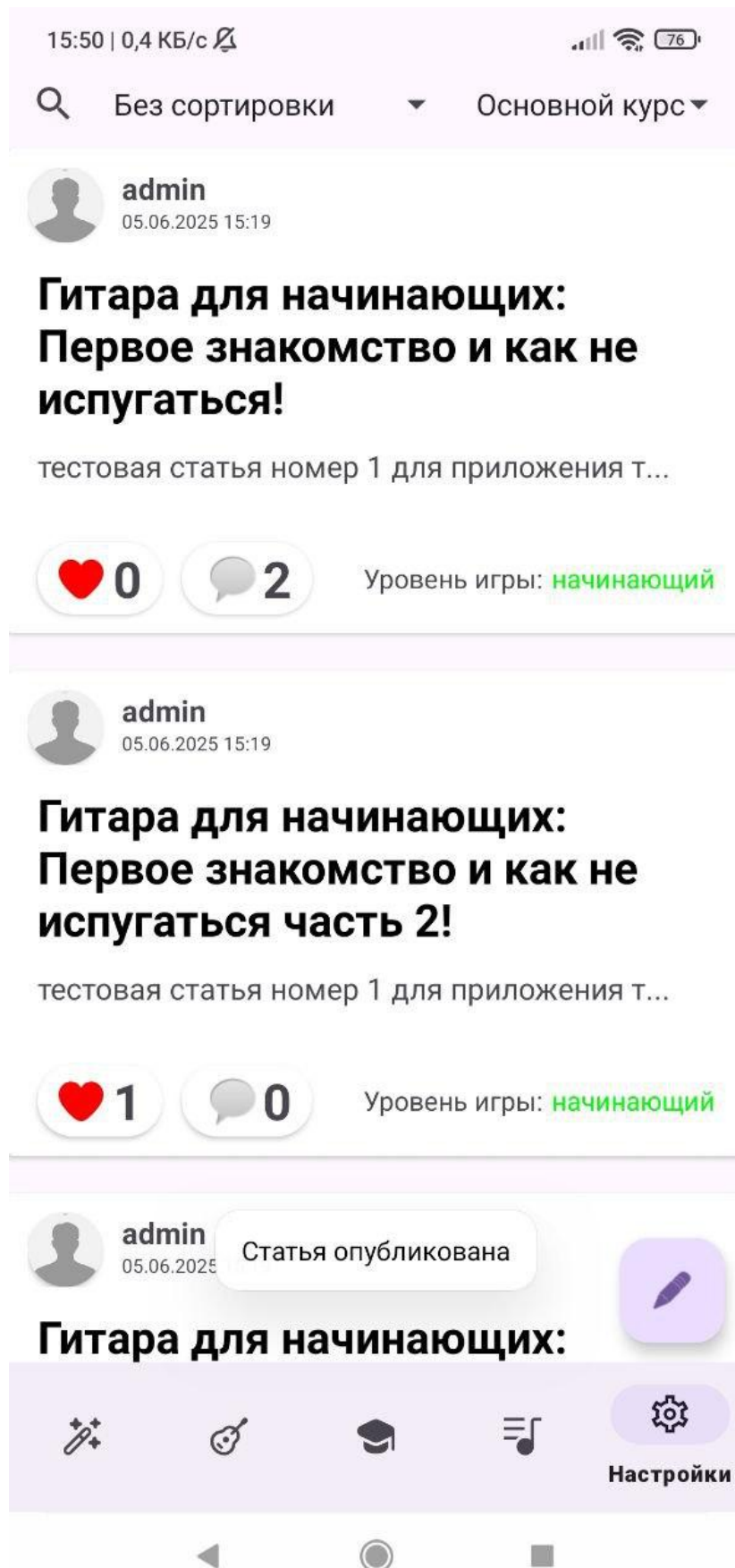


Рисунок 49 – Уведомление о публикации статьи



Рисунок 50 – Опубликованная статья в ленте от сообщества

4.4 Тестирование справочника аккордов

При нажатии в меню кнопки «Аккорды» откроется следующее окно представленное на рисунке 51. При вводе названия аккорда в поле отобразится его аппликатура, также есть возможность просмотра аппликатур этого аккорда в разных позициях (рисунки 52-53).

Аппликатура — порядок расположения и чередования пальцев при игре на музыкальном инструменте.



Рисунок 51 – Справочник аккордов



Рисунок 52 – Ввод аккорда «Е» для поиска аппликатуры



Рисунок 53 – Аппликатура аккорда «Е» в 6 позиции

4.5 Тестирование инструментов генерации табулатур и метронома

При нажатии на кнопку «Инструменты» откроется окно с выбором инструментов (рисунок 54), если это первое открытие этой панели, то устройство запросит разрешение на доступ к микрофону устройства (рисунок 55). При нажатии на кнопку «Генерация табулатуры» откроется следующее окно, представленное на рисунке 56.

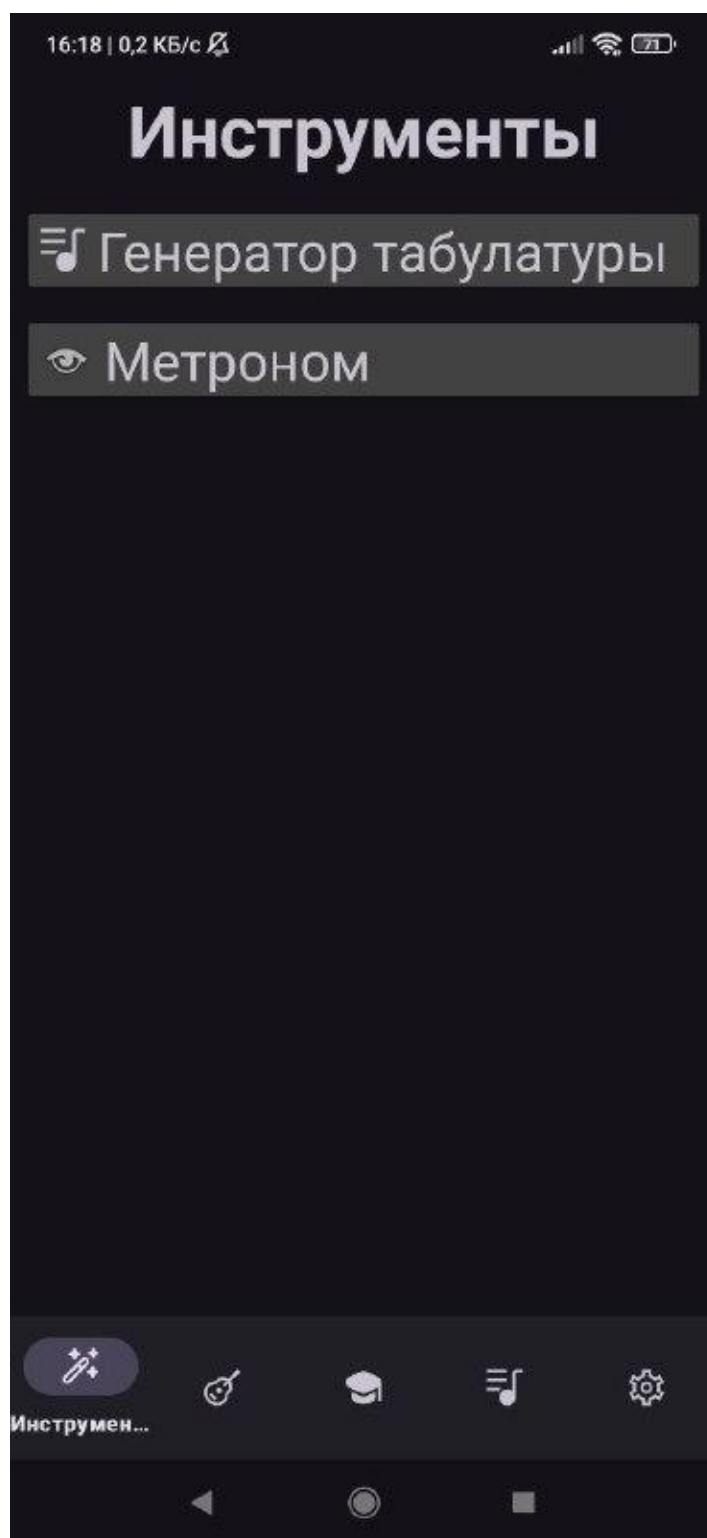


Рисунок 54 -Панель инструментов

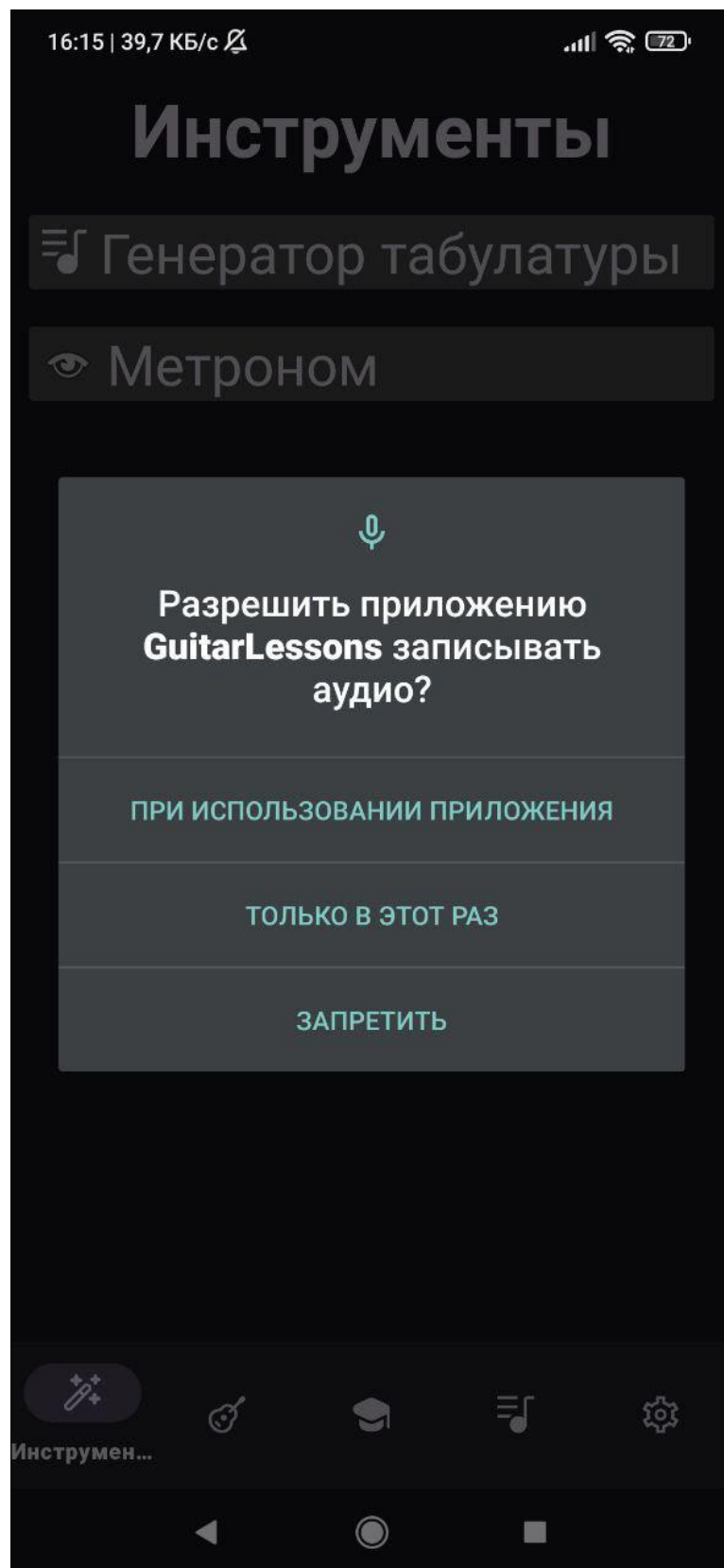


Рисунок 55 – Запрос разрешения на доступ к записи аудио с микрофона

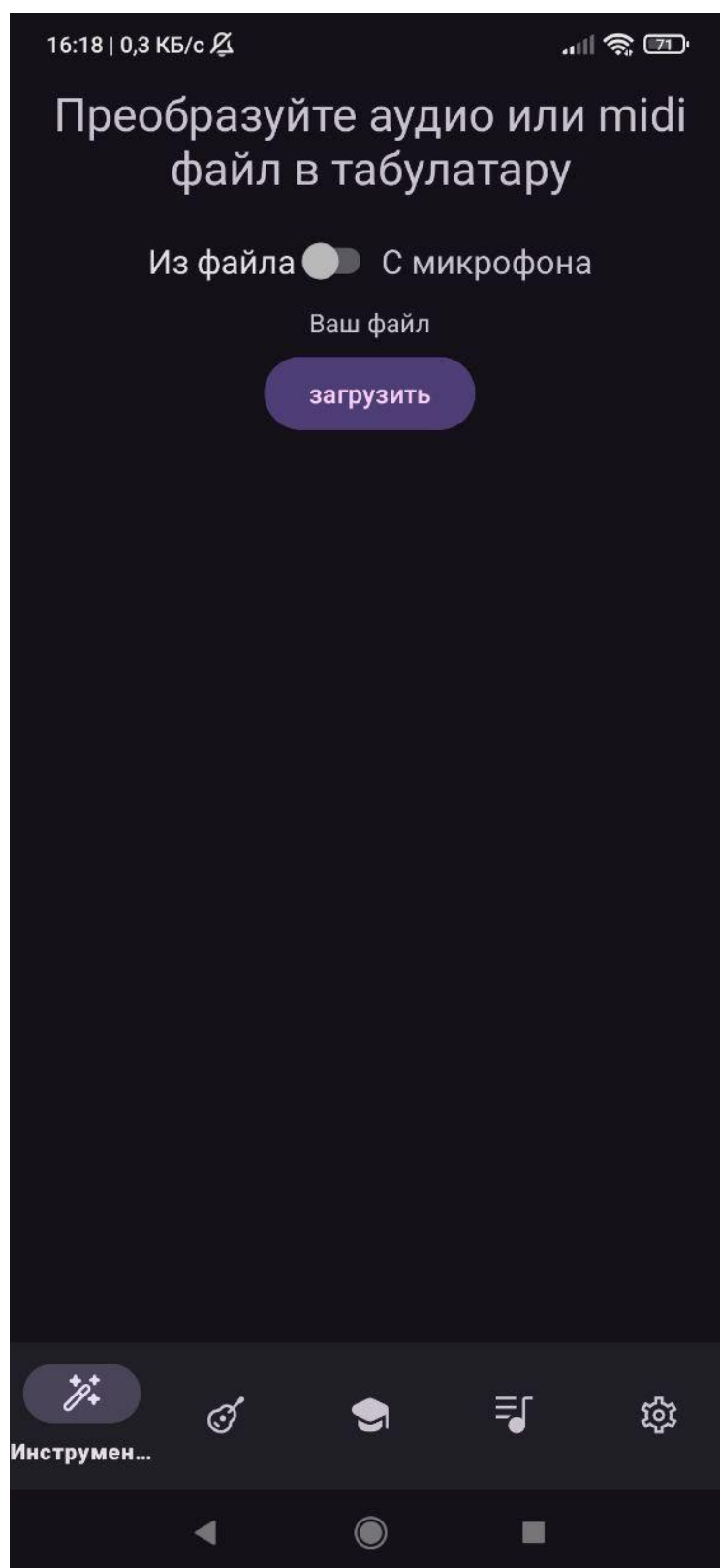


Рисунок 56 – Окно генерации табулатуры

Если переключатель установлен в состояние «из файла», то при нажатии на кнопку «загрузить» откроется файловая система устройства, где можно

выбрать файлы типа mid, midi или wav (рисунок 57). Если же переключатель установлен в состояние «с микрофона» то вместо кнопки «загрузить» появится кнопка «запись» (рисунок 58). После выбора соответствующего трека появится кнопка «преобразовать в табулатуру» (рисунок 59). При нажатии на которую начнется обработка файла. По окончании процесса на экран будет выведена табулатура, а внизу появится возможность её назвать. (рисунки 60-62) А также возможность сохранить в pdf или личный кабинет с выбором опубликовать табулатуру или нет (рисунок 63).

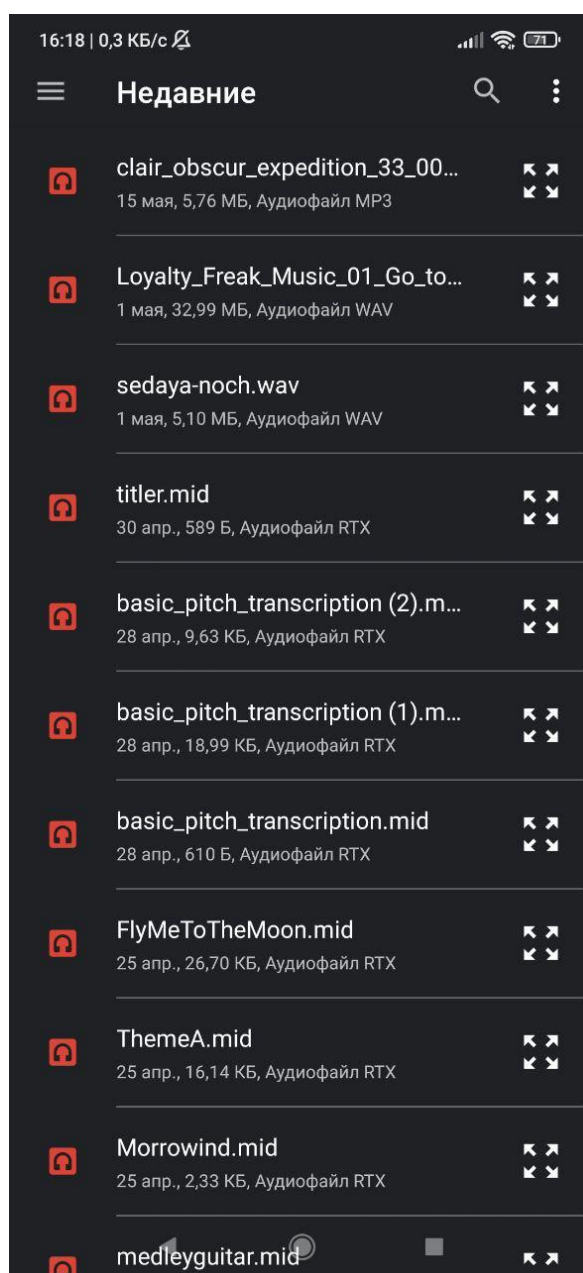


Рисунок 57 – Открытие файловой системы устройства

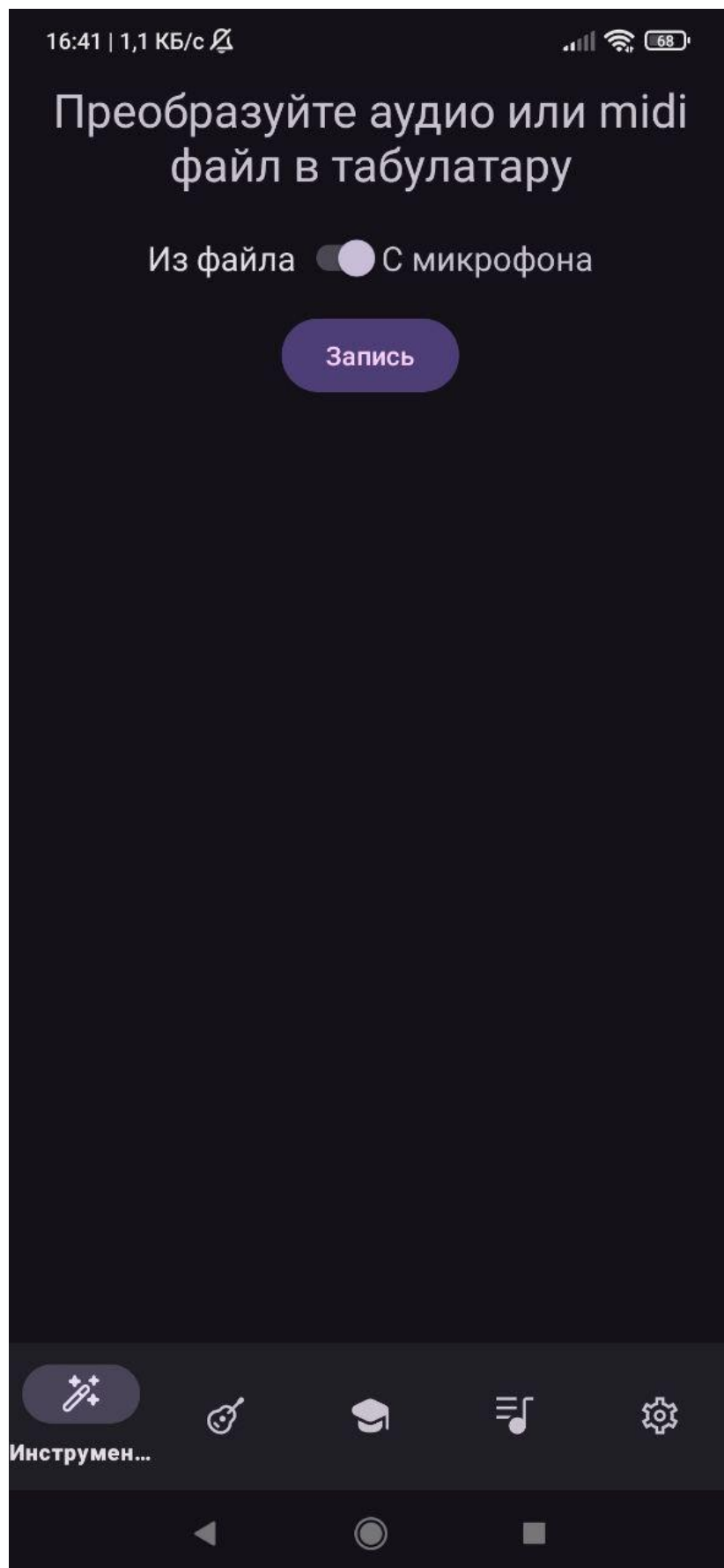


Рисунок 58– Запись с микрофона

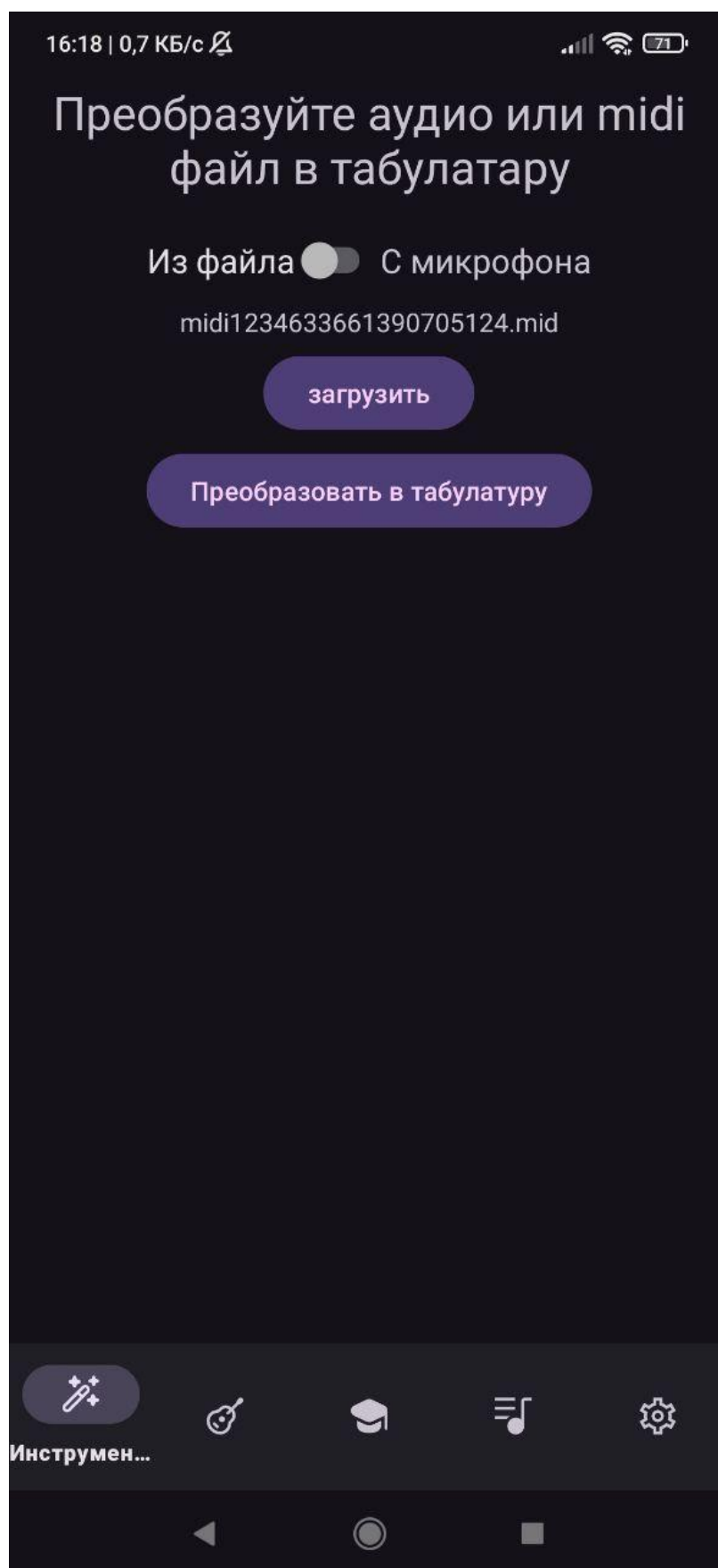


Рисунок 59 – Состояние после выбора файла

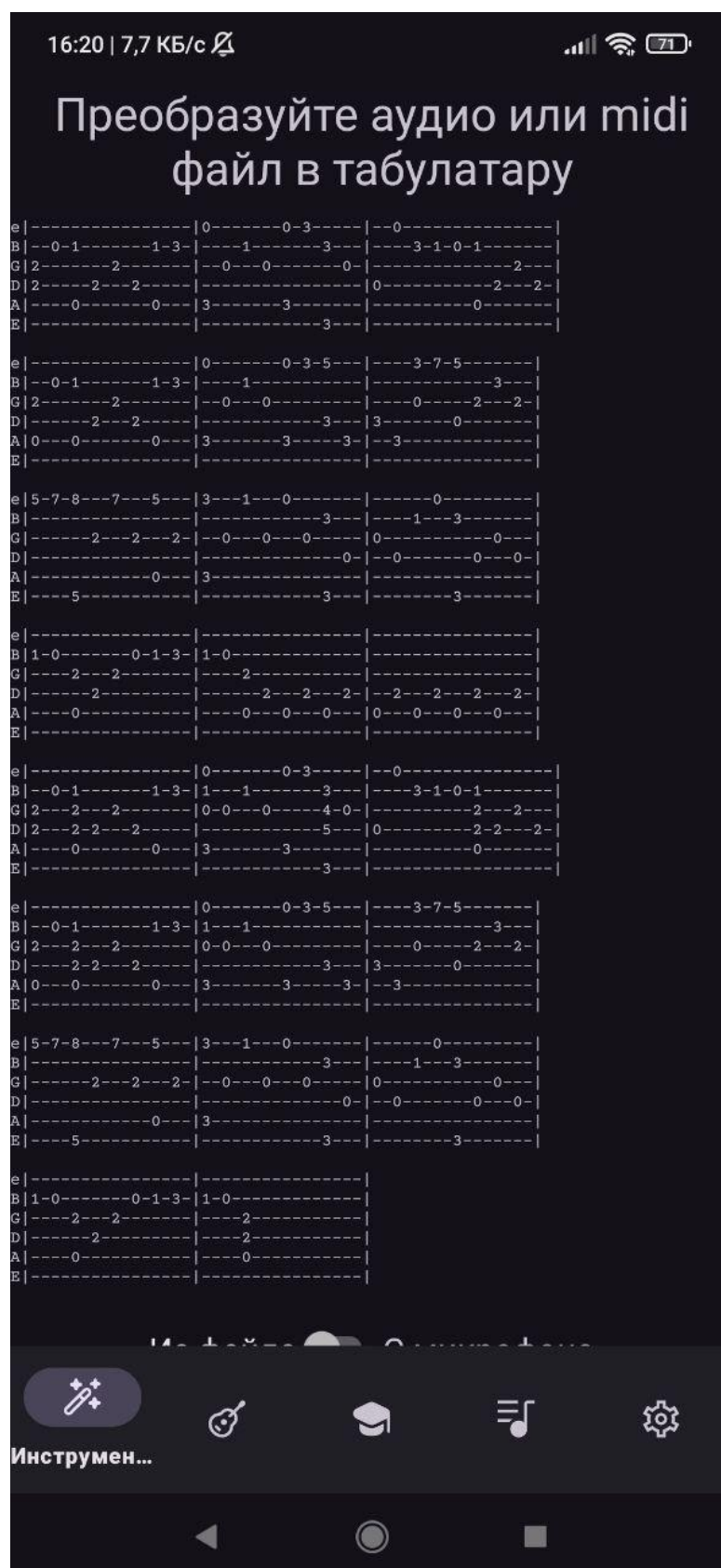


Рисунок 60 – Преобразование midi файла в табулатуру

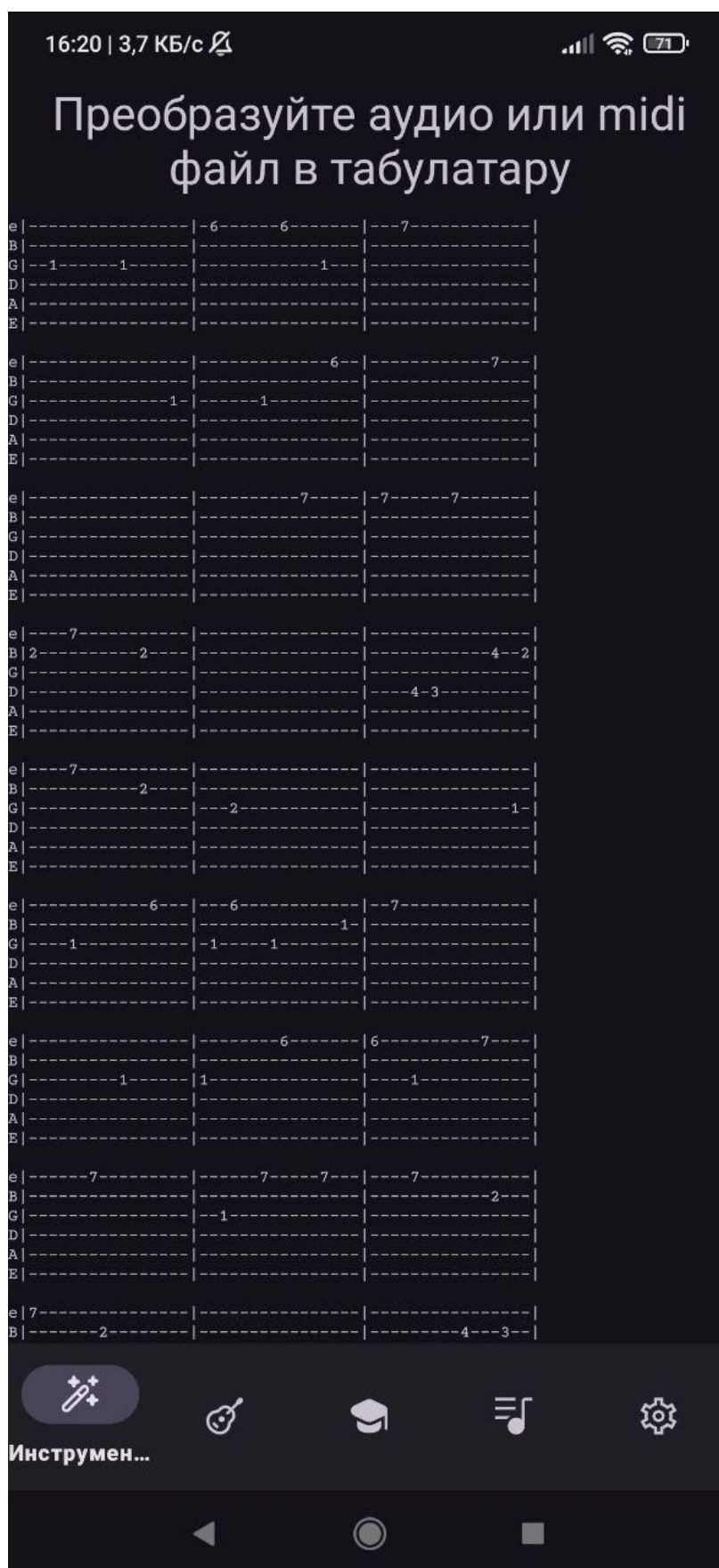


Рисунок 61 – Преобразование wav файла в табулатуру

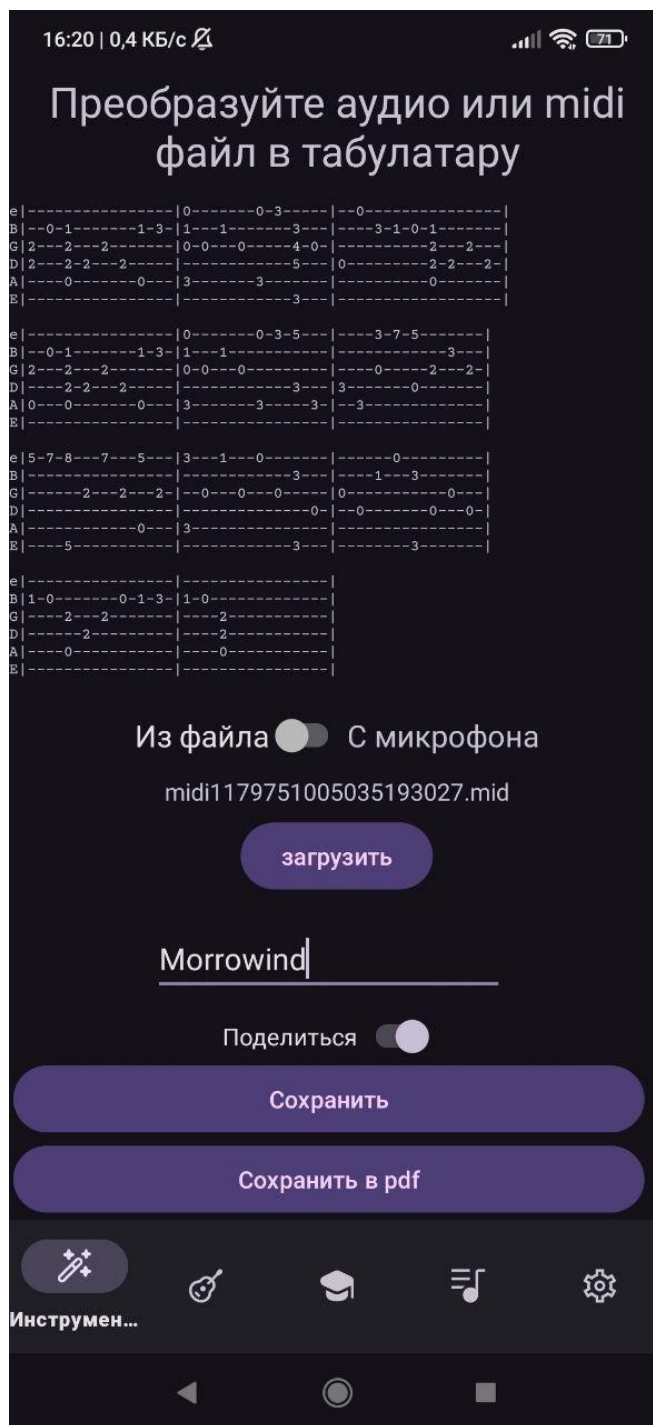


Рисунок 62 – Настройка табулатуры перед сохранением

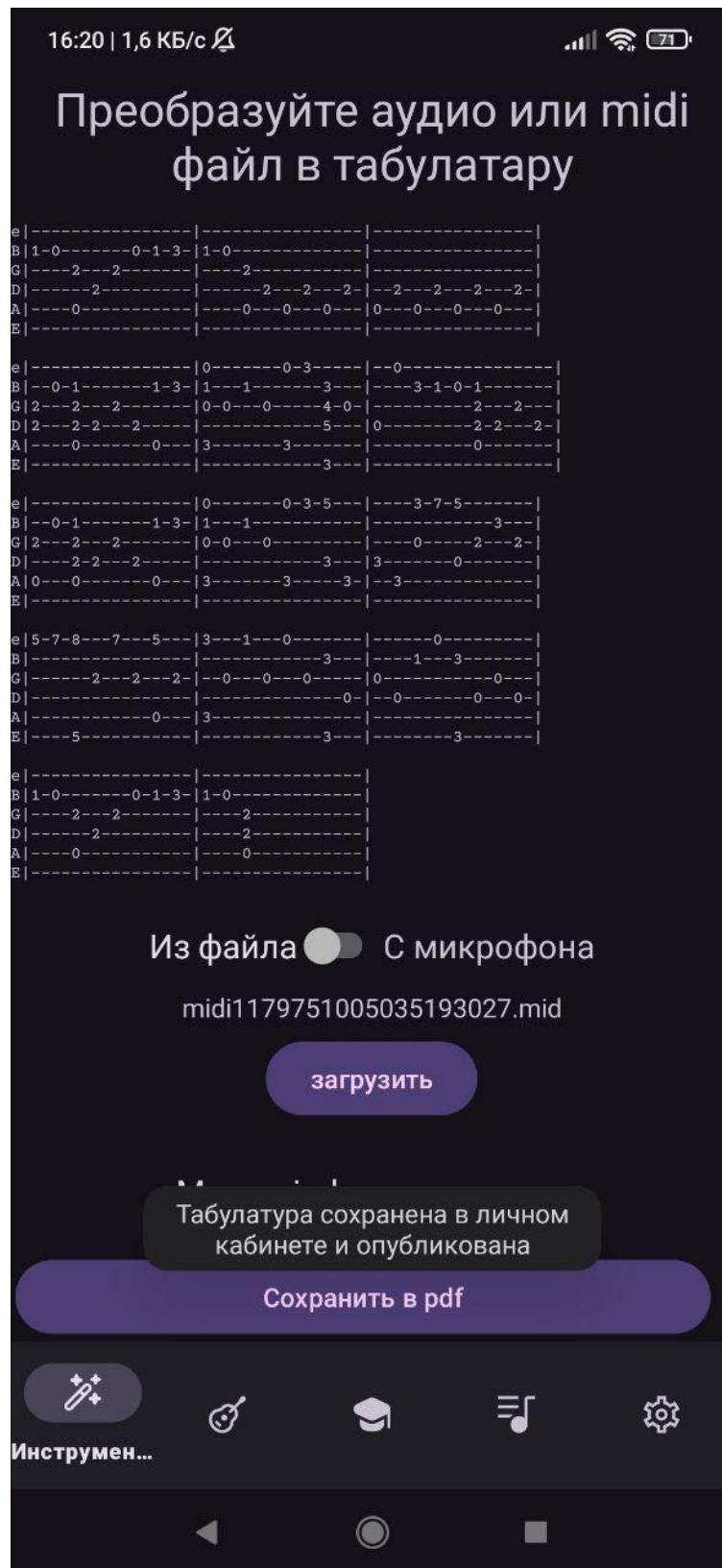


Рисунок 63 – сохранение табулатуры в личный кабинет

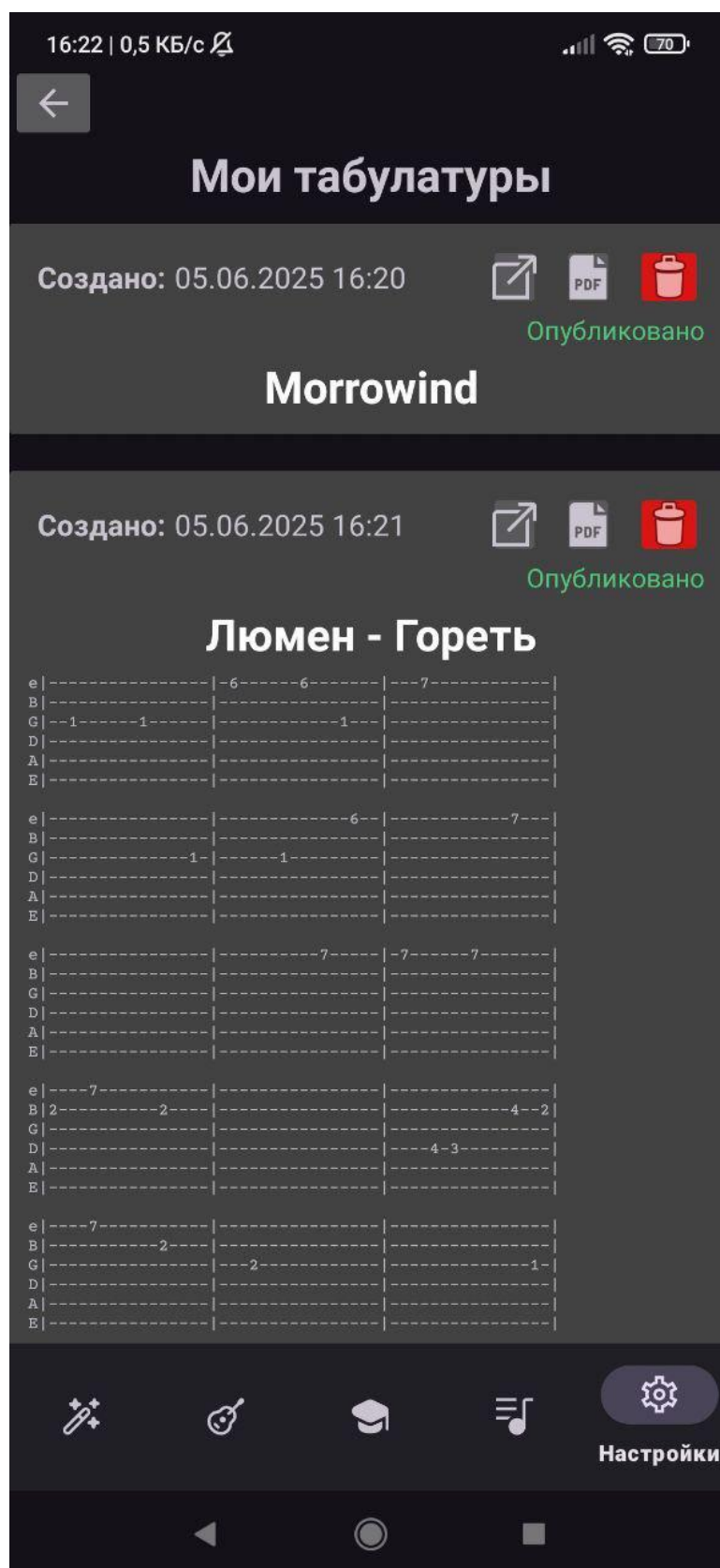


Рисунок 64 – Табулатура в личном кабинете

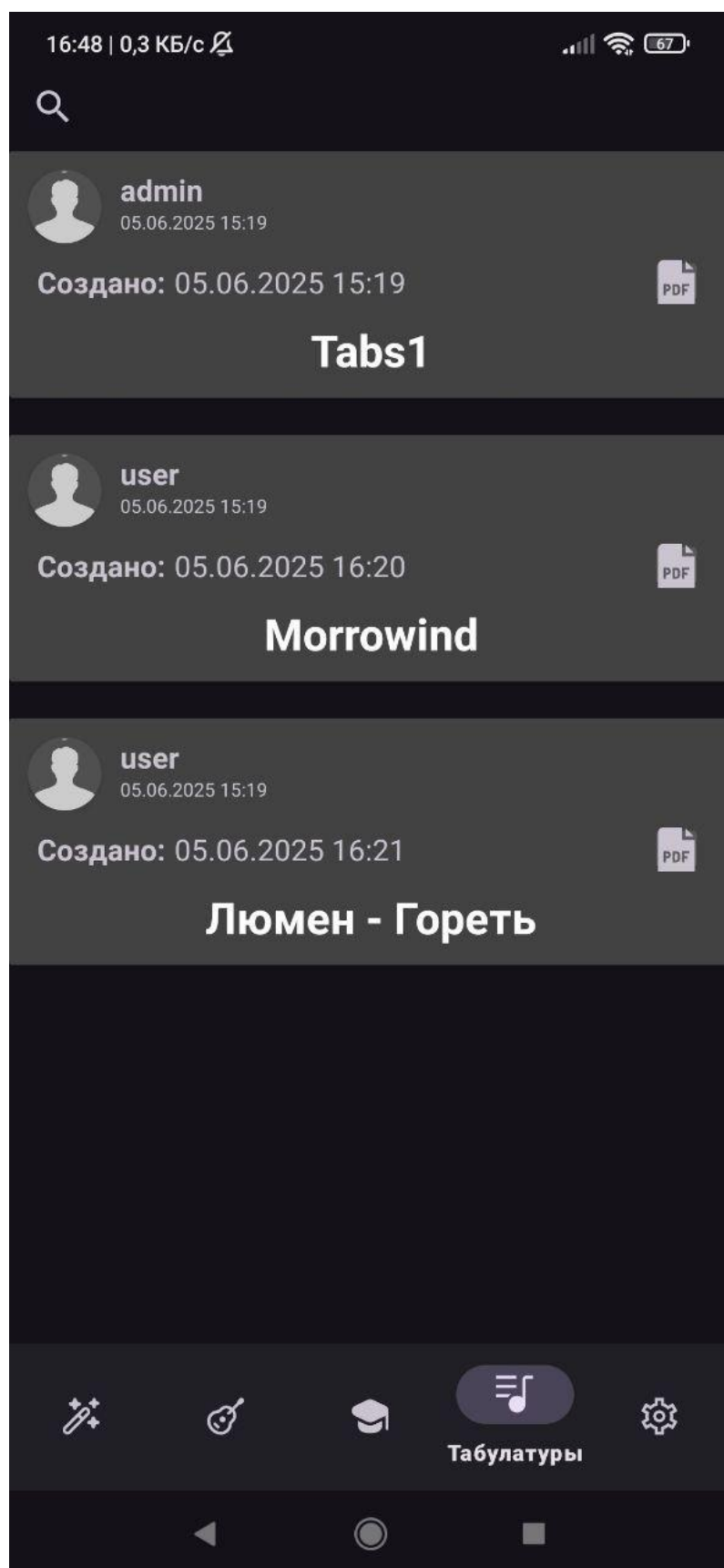


Рисунок 65 – Опубликованная табулатура в ленте

При нажатии на кнопку «Метроном» в панели инструментов откроется следующее окно, представленное на рисунке 66. Нажав на кнопку с иконкой треугольника метроном включится. С помощью ползунка или кнопок со стрелками можно изменять темп метронома, доступен диапазон от 40 до 210 BPM(beats per minute с английского удары в минуту).

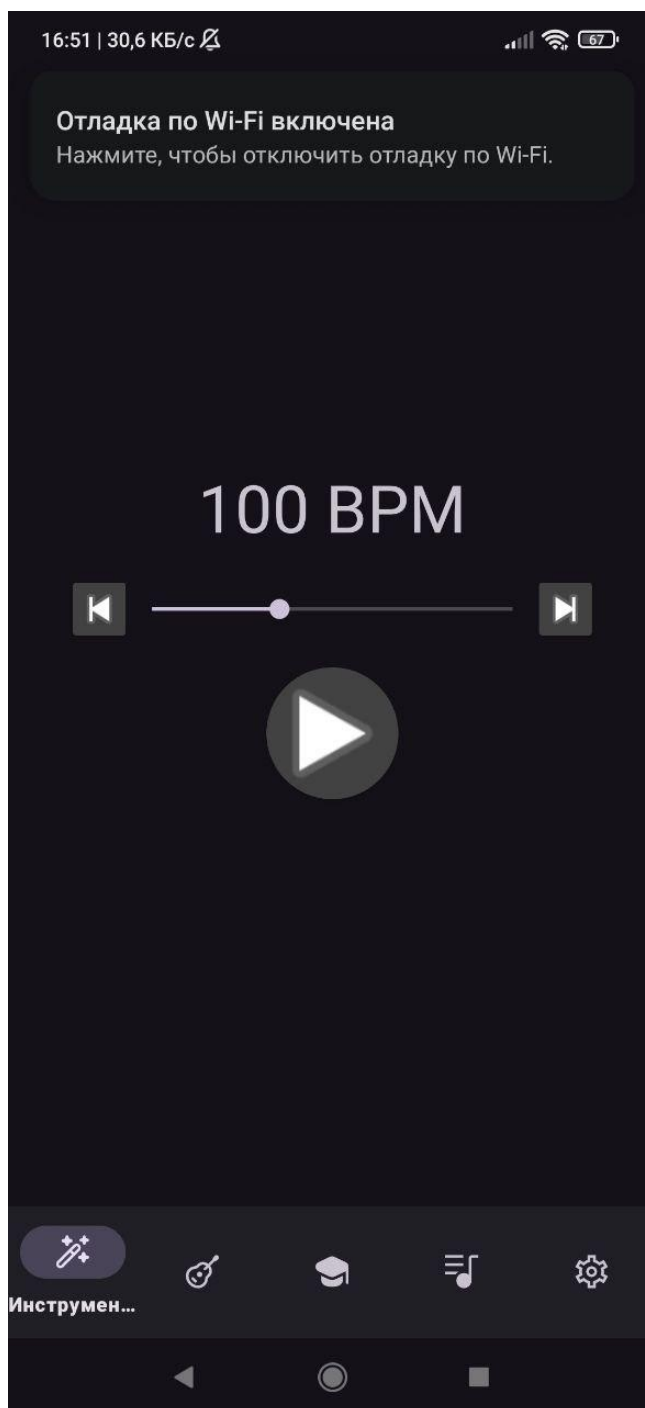


Рисунок 66 – Окно с метрономом

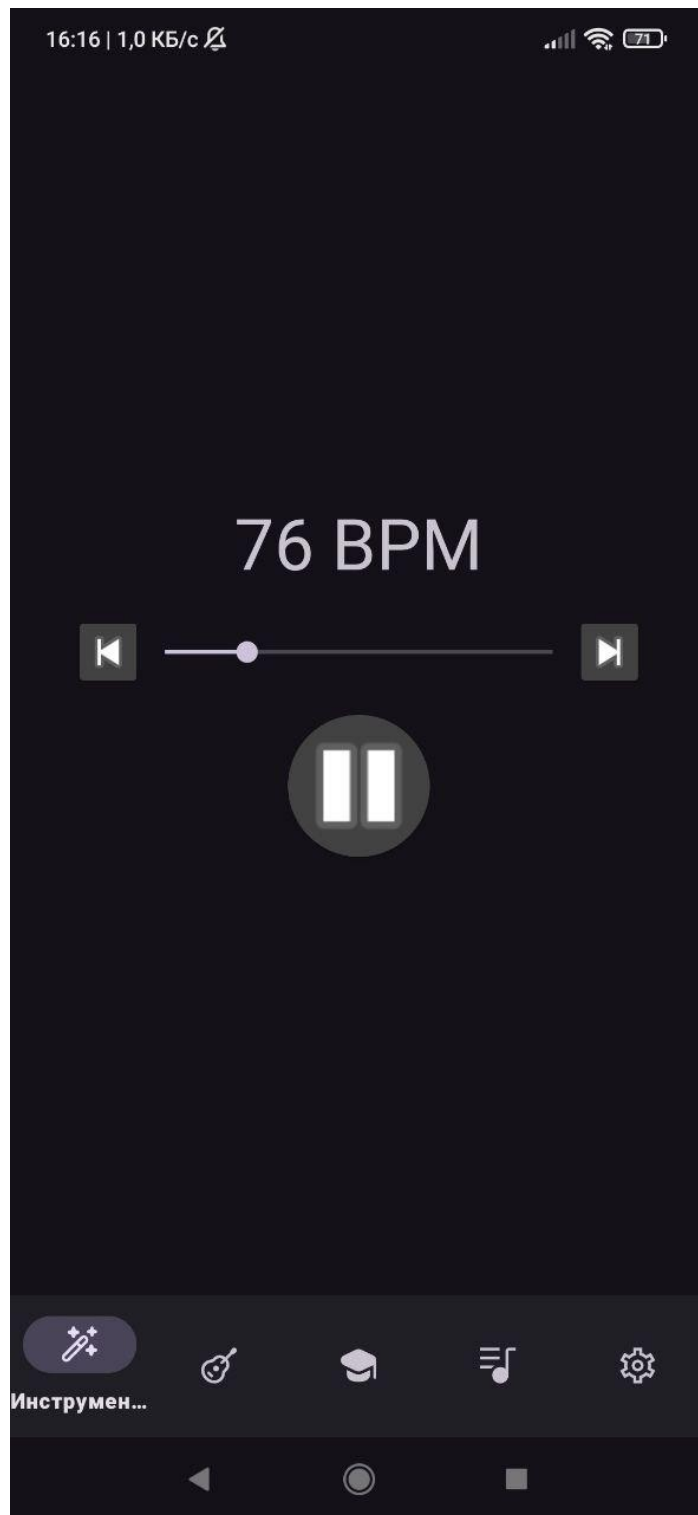


Рисунок 67 – Изменение BPM

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной квалификационной работы была достигнута поставленная цель — создание приложения для обучения игре на гитаре. Приложение позволяет генерировать табулатуру, создавать и просматривать структурированные уроки, а также предоставляет доступ к справочнику аккордов. Также были выполнены поставленные задачи: создание системы авторизации, личный кабинет, редактор для уроков, сортировка и поиск уроков, генерация табулатуры из аудиофайла, микрофонного входа или файла формата MIDI с сохранением в PDF или личный кабинет с возможностью поделиться с другими пользователями приложения, также был реализован метроном.

Подводя итоги выполнения ВКР, можно сделать следующие выводы:

- 1) был проведен анализ предметной области, на основе которого были сформулированы требования к системе,
- 2) реализовано приложение, соответствующее всем требованиям
- 3) результаты тестирования программы подтвердили выполнение поставленных задач

В будущем планируется расширение функционала созданной программы. В список улучшений будет включено обновление алгоритма выделения нот с дополнительным использованием нейронных сетей для повышения качества распознавания, создание удобного редактора табулатуры с возможностью прослушать результат. Так же планируется расширение инструментария с добавлением распознавания аккордов и тюнера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Как читать табы (табулатуры) для гитары. Полное руководство для начинающих гитаристов // Перебором URL: <https://pereborom.ru/kak-chitat-tabu-dlya-gitary/> (дата обращения: 19.04.2025).
2. Andrew Duncan Combinatorial Music Theory. - Journal of the Audio Engineering Society, 1991. - 448 с.
3. Masataka Goto A real-time music-scene-description system: predominant F0 estimation for detecting melody and bass lines in real-world audio signals. - 4-е изд. - Speech Communication, 2004. - 329 с.
4. Коберниченко В.Г. Основы цифровой обработки сигналов: учебное пособие. - Урал. федер. ун-т. — Екатеринбург: М-во науки и высш. образования Рос. Федерации, 2018. - 150 с.
5. Подборка лучших приложений для начинающих гитаристов // maxim URL: <https://www.maximonline.ru/lifestyle/8-luchshikh-prilozhenii-dlya-nachinayushikh-gitaristov-id500633/> (дата обращения: 01.05.2025).
6. Ричард Марк Фундаментальный подход к программной архитектуре: паттерны, свойства, проверенные методы. - СПб.: Питер, 2023. - 448 с.
7. Скин Джош Kotlin. Программирование для профессионалов. - СПб.: Питер, 2020. - 464 с.
8. Документация по Spring Framework // Spring URL: <https://spring.io/projects/spring-framework> (дата обращения: 23.04.2025).
9. Уоллс К. Spring в действии. - 6-е изд. - М.: ДМК Пресс, 2021. - 704 с.
10. Моргунов Е.П. PostgreSQL. Профессиональный SQL. - М.: ДМК Пресс, 2025. - 444 с.
11. Филлипс Билл. Android. Программирование для профессионалов. 4-е издание. - 4-е изд. - СПб.: Питер, 2021. - 704 с.
12. Документация по работе с библиотекой TarsosDSP // TarsosDSP URL: <https://0110.be/tags/TarsosDSP> (дата обращения: 10.03.2025).

13. GitHub репозиторий библиотеки RichEditor // GitHub URL: <https://github.com/wasabeef/richeditor-android> (дата обращения: 13.04.2025).
14. GitHub репозиторий библиотеки ChordDroid // GitHub URL: <https://github.com/trungdq88/ChordDroid> (дата обращения: 20.04.2025).
15. Комаров В.И. Путеводитель по базам данных. - М.: ДМК Пресс, 2024. - 520 с.

SQL скрипт инициализации

```
DROP TABLE IF EXISTS article_likes;
DROP TABLE IF EXISTS comments;
DROP TABLE IF EXISTS articles;
DROP TABLE IF EXISTS tabs;
DROP TABLE IF EXISTS status;
DROP TABLE IF EXISTS skill_levels;
DROP TABLE IF EXISTS users;
DROP TABLE IF EXISTS roles;

CREATE TABLE roles
(
    role_id    SERIAL PRIMARY KEY,
    role_name  VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE users
(
    user_id          SERIAL PRIMARY KEY,
    username          VARCHAR(50) UNIQUE NOT NULL,
    email             VARCHAR(255) UNIQUE NOT NULL,
    password          VARCHAR(255) NOT NULL,
    registration_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL ,
    avatar_picture    VARCHAR(255),
    role_id           INTEGER NOT NULL,
    CONSTRAINT fk_role_id FOREIGN KEY (role_id) REFERENCES roles (role_id)
);

CREATE TABLE status
(
    status_id SERIAL PRIMARY KEY,
    name VARCHAR(20) UNIQUE NOT NULL
);

CREATE TABLE skill_levels
(
    skill_level_id SERIAL PRIMARY KEY,
    name VARCHAR(24) UNIQUE NOT NULL
);

CREATE TABLE articles
(
    article_id SERIAL PRIMARY KEY,
    title       VARCHAR(255) NOT NULL,
    content     TEXT NOT NULL,
    publication_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    rating      INTEGER DEFAULT 0,
    author_id   INTEGER NOT NULL,
    skill_level_id INTEGER NOT NULL,
    status_id   INTEGER NOT NULL,
    CONSTRAINT fk_author_id FOREIGN KEY (author_id) REFERENCES users
(user_id),
    CONSTRAINT fk_skill_level_id FOREIGN KEY (skill_level_id) REFERENCES
skill_levels (skill_level_id),
    CONSTRAINT fk_status_id FOREIGN KEY (status_id) REFERENCES status
(status_id)
);
```

```

CREATE TABLE tabs
(
    tab_id SERIAL PRIMARY KEY,
    title    VARCHAR(255) NOT NULL,
    content  TEXT NOT NULL,
    creation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    author_id INTEGER NOT NULL,
    is_shared BOOLEAN,

    CONSTRAINT fk_author_id FOREIGN KEY (author_id) REFERENCES users
(user_id)
);

CREATE TABLE comments(
    comment_id SERIAL PRIMARY KEY,
    comment_text TEXT NOT NULL,
    comment_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    author_id INTEGER NOT NULL,
    article_id INTEGER NOT NULL,
    CONSTRAINT fk_user_id FOREIGN KEY (author_id) REFERENCES users(user_id),
    CONSTRAINT fk_article_id FOREIGN KEY (article_id) REFERENCES
articles(article_id)
)

INSERT INTO status(name)
VALUES
    ('draft'),
    ('published'),
    ('rejected');

INSERT INTO skill_levels(name)
VALUES
    ('beginner'),
    ('intermediate'),
    ('advanced');

INSERT INTO roles (role_name)
VALUES
    ('USER'),
    ('ADMIN');

```

Исходный код мобильного приложения

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <!-- Permission to record audio -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission
        android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission
        android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="32"
        tools:ignore="ScopedStorage" />
    <uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.GuitarLessons"
        android:networkSecurityConfig="@xml/network_security_config"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <meta-data
            android:name="preloaded_fonts"
            android:resource="@array/preloaded_fonts" />
    </application>
</manifest>
```

AccordsFragment.kt

```
package com.example.guitarlessons.fragments

import android.annotation.SuppressLint
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageButton
import androidx.core.widget.doOnTextChanged
import androidx.fragment.app.Fragment
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentAccordsBinding
```

```

class AccordsFragment : Fragment() {

    private lateinit var binding: FragmentAccordsBinding
    private var position = 0
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?,
    ): View {
        binding = FragmentAccordsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        binding.accordChord.drawChord("Am", 0)
        binding.positionView.text = position.toString()
        setIsEnable(false, 0.5f, binding.decPosBtn)
        binding.decPosBtn.setOnClickListener {
            if (position > 0) {
                position--
                binding.accordChord.toPosition(position)
                binding.positionView.text = position.toString()
            }

            when (position) {
                0 -> setIsEnable(false, 0.5f, binding.decPosBtn)
                8 -> setIsEnable(false, 0.5f, binding.insPosBtn)
                else -> {
                    setIsEnable(true, 1.0f, binding.decPosBtn)
                    setIsEnable(true, 1.0f, binding.insPosBtn)
                }
            }
        }

        binding.insPosBtn.setOnClickListener {
            if (position < 8) {
                position++
                binding.accordChord.toPosition(position)
                binding.positionView.text = position.toString()
                when (position) {
                    0 -> setIsEnable(false, 0.5f, binding.decPosBtn)
                    8 -> setIsEnable(false, 0.5f, binding.insPosBtn)
                    else -> {
                        setIsEnable(true, 1.0f, binding.decPosBtn)
                        setIsEnable(true, 1.0f, binding.insPosBtn)
                    }
                }
            }
        }

        binding.accordTextView.doOnTextChanged { text, start, before, count -
>
            if (!text.isNullOrEmpty())
                binding.accordChord.drawChord(text.toString(), position)
        }
    }

    private fun setIsEnable(isEnable: Boolean, alpha: Float, btn: ImageButton) {
        btn.isEnabled = isEnable
        btn.alpha = alpha
    }

    @SuppressLint("CommitTransaction")

```

```

    fun replaceFragmentOnView(f:Fragment) {
        parentFragmentManager
            .beginTransaction()
            .replace(R.id.fragmentContainerView,f)
            .commit()
    }

    companion object {
        @JvmStatic
        fun newInstance() = AccordsFragment()
    }
}

```

AccordManagerFragment.kt

```

package com.example.guitarlessons.fragments

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.net.Uri
import android.os.Bundle
import android.text.TextUtils
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.activity.result.ActivityResultLauncher
import androidx.activity.result.contract.ActivityResultContracts
import androidx.core.view.isGone
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.fragment.app.viewModels
import androidx.lifecycle.LifecycleOwner
import androidx.lifecycle.lifecycleScope
import com.bumptech.glide.Glide
import com.bumptech.glide.request.RequestOptions
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentAccountManagerBinding
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.upload.UploadApi
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.retrofit.user.UserApi
import com.example.guitarlessons.viewmodel.ImageViewModel
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.MediaType.Companion.toMediaTypeOrNull
import okhttp3.MultipartBody
import okhttp3.OkHttpClient
import okhttp3.RequestBody
import okhttp3.RequestBody.Companion.asRequestBody
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.converter.scalars.ScalarsConverterFactory
import java.io.File
import java.io.FileOutputStream

```

```

import java.io.IOException

class AccountManagerFragment : Fragment() {
    private val tokensViewModel: TokensViewModel by activityViewModels()

    private var accessToken: String? = null

    private var refreshToken: String? = null

    private var username: String? = null

    private lateinit var binding: FragmentAccountManagerBinding

    private lateinit var getContent: ActivityResultLauncher<String>

    private val imageViewModel: ImageViewModel by viewModels()

    private var imgUri: Uri? = null

    private var imgUrl:String? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        getContent =
registerForActivityResult(ActivityResultContracts.GetContent()) { uri: Uri? -
>
        imageViewModel.uri.value = uri
        if (uri != null) {
            context?.let {
                Glide.with(it)
                    .load(uri)
                    .apply(
                        RequestOptions().override(
                            binding.avatarChangeHolder.width,
                            binding.avatarChangeHolder.height
                        )
                    )
                    .centerCrop()
                    .into(binding.avatarChangeImage)
            }
        }
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentAccountManagerBinding.inflate(inflater, container,
false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
    }
}

```

```

val interceptor = HttpLoggingInterceptor()
interceptor.level = HttpLoggingInterceptor.Level.BODY

val client = OkHttpClient.Builder()
    .addInterceptor(interceptor)
    .build()

val retrofit = Retrofit
    .Builder()
    .baseUrl(BuildConfig.API_URL).client(client)
    .addConverterFactory(ScalarsConverterFactory.create())
    .addConverterFactory(GsonConverterFactory.create())
    .build()

imgViewModel.uri.observe(activity as LifecycleOwner, {
    imgUri = it
})

binding.backAccountBtn.setOnClickListener {
    replaceFragmentOnView(SettingsFragment.newInstance())
}

binding.DeleteAccountBtn.setOnClickListener {
    val builder = AlertDialog.Builder(context)
    builder.setMessage("Вы уверены что хотите удалить аккаунт?")
        .setCancelable(false)
        .setPositiveButton("Да") { dialog, id ->
            lifecycleScope.launch {
                deleteMe(retrofit)
            }
        }
        .setNegativeButton("Нет") { dialog, id ->
            dialog.dismiss()
        }
    val alert = builder.create()
    alert.show()
}

binding.ExitAccountBtn.setOnClickListener {
    lifecycleScope.launch {

        TokenManager.clearTokens(requireContext())
        tokensViewModel.accessToken.value = null
        tokensViewModel.refreshToken.value = null
        replaceFragmentOnView(SettingsFragment.newInstance())
    }
}

binding.changePasswordBtn.setOnClickListener {
    if (binding.changePasswordForm.isGone)
        binding.changePasswordForm.visibility = View.VISIBLE
    else
        binding.changePasswordForm.visibility = View.GONE
}

if ((activity as MainActivity).isConnected()) {
    if (accessToken != null) {
        lifecycleScope.launch {
            val user: User? = getMe(retrofit)

```

```

        activity?.runOnUiThread {
            if (user != null) {
                username = user.username
                authorized(user)
            } else {
                unauthorized()
            }
        }
    } else {
        unauthorized()
    }
} else {
    unauthorized()
}

binding.confirmChangePassword.setOnClickListener {
    if ((activity as MainActivity).isConnected()) {
        lifecycleScope.launch {
            changePassword(retrofit)
        }
    } else {
        Toast.makeText(context, "Отсутствует подключение к интернету", Toast.LENGTH_SHORT)
            .show()
    }
}

binding.changeDataBtn.setOnClickListener {
    if ((activity as MainActivity).isConnected()) {
        lifecycleScope.launch {
            uploadImage(retrofit)
            changeData(retrofit)
        }
    } else {
        Toast.makeText(context, "Отсутствует подключение к интернету", Toast.LENGTH_SHORT)
            .show()
    }
}

binding.avatarChangeBtn.setOnClickListener {
    chooseAndProcessImage()
}

}

private suspend fun deleteMe(retrofit: Retrofit){
    try {
        val userApi: UserApi = retrofit.create(UserApi::class.java)

        return withContext(Dispatchers.IO) {
            val response = userApi.deleteMe("Bearer_${accessToken}")
            if (response.code() == 200) {
                TokenManager.clearTokens(requireContext())
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = null
                    tokensViewModel.refreshToken.value = null
                    Toast.makeText(context, "Аккаунт успешно удален", Toast.LENGTH_LONG) .show()
                }
            }
        }
    }
}

```

```

        }
        replaceFragmentOnView(SettingsFragment.newInstance())
    }
    else{
        activity?.runOnUiThread {
            Toast.makeText(context, "Произошла ошибка при удалении
учетной записи", Toast.LENGTH_LONG).show()
        }
    }
}
} catch (e:Exception){
    activity?.runOnUiThread {
        Toast.makeText(context, "Произошла ошибка попробуйте
позже", Toast.LENGTH_LONG).show()
    }
}

}

private fun chooseAndProcessImage() {
    getContent().launch("image/*")
}

private fun convertUriToFile(imgUri: Uri): File? {
    try {
        val inputStream =
requireContext().contentResolver.openInputStream(imgUri)

        val file = File.createTempFile("image", ".jpg",
requireContext().cacheDir)

        val outputStream = FileOutputStream(file)
        val buffer = ByteArray(4096) // Размер буфера
        var bytesRead: Int

        while (inputStream?.read(buffer).also { bytesRead = it!! } != -1)
        {
            outputStream.write(buffer, 0, bytesRead)
        }

        return file
    } catch (e: IOException) {
        e.printStackTrace()
        return null // Или выбросьте исключение
    }
}

private suspend fun uploadImage(retrofit: Retrofit) {
    if (imgUri != null) {
        val uploadApi: UploadApi = retrofit.create(UploadApi::class.java)
        val file:File? = convertUriToFile(imgUri!!)
        if(file != null){
            val mediaType = "image/*".toMediaTypeOrNull()
            val requestImage: RequestBody = file.asRequestBody(mediaType)
            val body: MultipartBody.Part =
MultipartBody.Part.createFormData(
                "image",
                file.name,
                requestImage
            )

```

```

        return withContext(Dispatchers.IO) {
            val responseImage: Response<String> =
uploadApi.uploadImage(body)
            if (responseImage.code() == 200) {
                activity?.runOnUiThread {
                    imgUrl = responseImage.body().toString()
                    Log.e("URL", imgUrl.toString())
                }
            }
        }
    }
}

private suspend fun changeData(retrofit: Retrofit) {
    if (!isEmailValid()) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Введена некорректная почта",
Toast.LENGTH_SHORT).show()
        }
        return
    }
    if (isNotValidUsername()) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Введено некорректное имя
пользователя", Toast.LENGTH_SHORT)
                .show()
        }
        return
    }
    val authApi: AuthApi = retrofit.create(AuthApi::class.java)
    val userApi: UserApi = retrofit.create(UserApi::class.java)

    return withContext(Dispatchers.IO) {
        val responseUser: Response<User>? = username?.let {
            userApi.updateUser(
                it,
                User(
                    username = binding.usernameChangeET.text.toString(),
                    email = binding.emailChangeET.text.toString(),
                    avatarPicture = imgUrl
                ),
                "Bearer_${accessToken}"
            )
        }
        if (responseUser?.code() == 200) {
            activity?.runOnUiThread {
                Toast.makeText(context, "Данные успешно обновлены",
Toast.LENGTH_LONG).show()
            }
        } else {

            if (responseUser?.code() == 409) {
                activity?.runOnUiThread {
                    Toast.makeText(
                        context,
                        "Имя пользователя или почта уже заняты",
                        Toast.LENGTH_LONG
                    ).show()
                }
            }
        }
    }
}

```



```

        }
        getMe(retrofit)
    } else {
        TokenManager.clearTokens(requireContext())
        tokensViewModel.accessToken.value = null
        tokensViewModel.refreshToken.value = null
        null
    }
} else
    null
} else
    null

}
}
catch (e:Exception){
    unauthorized()
    return null
}
}

private fun authorized(user: User) {
    binding.apply {
        if (user.avatarPicture != null) {
            context?.let {
                Glide.with(it)
                    .load(user.avatarPicture)
                    .apply(
                        RequestOptions().override(
                            binding.avatarChangeHolder.width,
                            binding.avatarChangeHolder.height
                        )
                    )
                    .centerCrop()
                    .into(avatarChangeImage)
            }
        }
        usernameChangeET.setText(user.username)
        emailChangeET.setText(user.email)
    }
}

private fun unauthorized() {
    binding.apply {
    }
}

private fun isValidPassword(): Boolean {
    return binding.changePasswordET.text.isEmpty() ||
        binding.changePasswordET.text.length > 255 ||
        binding.changePasswordET.text.length < 8
}

private fun isMatchPasswords(): Boolean {
    return binding.changePasswordET.text.toString() ==
        binding.reenterPasswordET.text.toString()
}

private fun isValidUsername(): Boolean {
    return binding.usernameChangeET.text.isEmpty() ||
        binding.usernameChangeET.text.length > 50 ||

```

```

binding.usernameChangeET.text.length < 4
    }

    private fun isEmailValid(): Boolean {
        return !TextUtils.isEmpty(binding.emailChangeET.text) &&
        android.util.Patterns.EMAIL_ADDRESS.matcher(
            binding.emailChangeET.text
        ).matches()
    }

    @SuppressWarnings("CommitTransaction")
    fun replaceFragmentOnView(f: Fragment) {
        parentFragmentManager
            .beginTransaction()
            .replace(R.id.fragmentContainerView, f)
            .commit()
    }

    companion object {
        @JvmStatic
        fun newInstance() = AccountManagerFragment()
    }
}

```

ArticleFragment.kt

```

package com.example.guitarlessons.fragments

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.graphics.Color
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.recyclerview.widget.LinearLayoutManager
import com.bumptech.glide.Glide
import com.bumptech.glide.request.RequestOptions
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.adapter.CommentAdapter
import com.example.guitarlessons.databinding.FragmentArticleBinding
import com.example.guitarlessons.retrofit.admin.AdminApi
import com.example.guitarlessons.retrofit.article.Article
import com.example.guitarlessons.retrofit.article.ArticleApi
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.comment.Comment
import com.example.guitarlessons.retrofit.comment.CommentApi
import com.example.guitarlessons.retrofit.comment.RequestComment
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.viewmodel.ArticleViewModel
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

```

```

import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class ArticleFragment : Fragment(), CommentAdapter.ClickListener {
    private lateinit var binding: FragmentArticleBinding
    private lateinit var commentAdapter: CommentAdapter
    private val articleViewModel: ArticleViewModel by activityViewModels()
    private var isAdmin: Boolean = false
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private var retrofit: Retrofit? = null
    private var accessToken: String? = null
    private var me: User? = null
    private var refreshToken: String? = null
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?,
    ): View {
        binding = FragmentArticleBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        binding.backBtn.setOnClickListener {
            replaceFragmentOnView(LessonsFragment.newInstance())
        }
        commentAdapter = CommentAdapter(this)
        binding.commentsView.layoutManager = LinearLayoutManager(context)
        binding.commentsView.adapter = commentAdapter
        if ((activity as MainActivity).isConnected()) {
            val interceptor = HttpLoggingInterceptor()
            interceptor.level = HttpLoggingInterceptor.Level.BODY

            val client = OkHttpClient.Builder()
                .addInterceptor(interceptor)
                .connectTimeout(50, TimeUnit.MILLISECONDS)
                .build()

            val rf = Retrofit
                .Builder()
                .baseUrl(BuildConfig.API_URL).client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build()
            retrofit = rf
            CoroutineScope(Dispatchers.IO).launch {
                val user = getMe(rf)
                if (user != null) {
                    me = user
                    if (user.role!!.roleName == "ADMIN") {
                        isAdmin = true
                    }
                }
            }
        }
    }
}

```

```

        commentAdapter.changeVisibility(isAdmin)
    }
    CoroutineScope(Dispatchers.IO).launch {
        getComments(rf)
    }

    articleViewModel.id.observe(viewLifecycleOwner) {
        CoroutineScope(Dispatchers.IO).launch {
            getArticle(rf, it)
        }
    }

    binding.likeBtn.setOnClickListener {
        val articleApi = rf.create(ArticleApi::class.java)
        CoroutineScope(Dispatchers.IO).launch {
            if (me != null) {
                val response = articleApi.like(
                    me!!.id!!,
                    articleViewModel.id.value!!,
                    "Bearer_${accessToken}"
                )
                if (response.code() == 200) {
                    activity?.runOnUiThread {
                        binding.likeCountView.text =
response.body()!!.count.toString()
                    }
                }
            } else {
                activity?.runOnUiThread {
                    Toast.makeText(
                        context,
                        "Для этого необходимо авторизоваться",
                        Toast.LENGTH_SHORT
                    ).show()
                }
            }
        }
    }

    binding.sendCommentBtn.setOnClickListener {
        CoroutineScope(Dispatchers.IO).launch {
            createComment(rf)
            activity?.runOnUiThread {
                binding.sendComment.text.clear()
            }
        }
    }
} else {
    replaceFragmentOnView(LessonsFragment.newInstance())
}

private suspend fun createComment(retrofit: Retrofit) {
    if (binding.sendComment.text.isEmpty())
        return
    try {
        val commentApi = retrofit.create(CommentApi::class.java)
        val authApi = retrofit.create(AuthApi::class.java)
        val authResponse =
authApi.refreshAccessToken(RefreshTokenRequest(refreshToken!!))
        if (authResponse.code() == 200) {
            TokenManager.saveTokens(
                requireContext(), accessToken!!,
                refreshToken!!
            )
        }
    }
}

```

```

    )
    activity?.runOnUiThread {
        tokensViewModel.accessToken.value = accessToken
        tokensViewModel.refreshToken.value = refreshToken
    }
    val responseComment = commentApi.addComment(
        RequestComment(
            binding.sendComment.text.toString(),
            articleViewModel.id.value!!
        ), "Bearer_$accessToken"
    )
    if (responseComment.code() == 200) {
        getComments(retrofit)
        activity?.runOnUiThread {
            binding.commentCountView.text =
                (binding.commentCountView.text.toString().toInt() + 1).toString()
            Toast.makeText(
                requireContext(),
                "Комментарий отправлен!",
                Toast.LENGTH_SHORT
            )
                .show()
        }
    }
}
} catch (e: Exception) {
    activity?.runOnUiThread {
        Toast.makeText(context, "Ошибка отправки комментария",
            Toast.LENGTH_SHORT).show()
    }
}

}

private suspend fun getComments(retrofit: Retrofit) {
    try {
        val commentApi = retrofit.create(CommentApi::class.java)
        val responseComments = articleViewModel.id.value?.let {
            commentApi.getByArticlesId(it)
        }
        if (responseComments?.code() == 200) {
            activity?.runOnUiThread {
                commentAdapter.submitList(responseComments.body())
            }
        }
    } catch (e: Exception) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Ошибка загрузки комментариев",
                Toast.LENGTH_SHORT).show()
        }
    }
}

private suspend fun getArticle(retrofit: Retrofit, id: Long) {
    val articleApi = retrofit.create(ArticleApi::class.java)
    val responseArticle: Response<Article> = articleApi.getById(id)
    if (responseArticle.code() == 200) {
        val article = responseArticle.body()
        activity?.runOnUiThread {
            binding.apply {

```

```

        if (article!!.author?.avatarPicture != null) {
            context?.let {
                Glide.with(it)
                    .load(article.author?.avatarPicture)
                    .apply(
                        RequestOptions().override(
                            avatarAuthorHolder.width,
                            avatarAuthorHolder.height
                        )
                    )
                    .centerCrop()
                    .into(avatarAuthorImage)
            }
        }
        authorNameView.text = article.author?.username ?:
"Пользователь"

        publicDateView.text = article.publicationDate
        titleView.text = article.title
        contentView.loadData(
            article.content,
            "text/html",
            null
        )
        contentView.setBackgroundColor(Color.TRANSPARENT)
        likeCountView.text = article.rating.toString()
        commentCountView.text = article.comments.size.toString()
        when (article.skillLevel?.levelName) {
            "beginner" -> {
                levelNameView.setTextColor(Color.GREEN)
                levelNameView.text = "начинающий"
            }

            "intermediate" -> {
                levelNameView.setTextColor(Color.rgb(255, 165,
0))
                levelNameView.text = "средний"
            }

            "advanced" -> {
                levelNameView.setTextColor(Color.RED)
                levelNameView.text = "продвинутый"
            }
        }
        articleBodyLayout.visibility = View.VISIBLE
    }

}

}

}

private suspend fun getMe(retrofit: Retrofit): User? {
    try {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        return withContext(Dispatchers.IO) {
            val responseUser: Response<User> =
authApi.getMe("Bearer_${accessToken}")
            if (responseUser.isSuccessful) {
                // Проверка статуса ответа
                responseUser.body()
            } else if (responseUser.code() == 500) {
                if (refreshToken != null) {

```

```

        val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
            RefreshTokenRequest(
                refreshToken!!
            )
        )
        if (responseToken.code() == 200) {
            accessToken =
responseToken.body()?.accessToken.toString()
            TokenManager.saveTokens(
                requireContext(), accessToken!!,
                refreshToken!!
            )
            activity?.runOnUiThread {
                tokensViewModel.accessToken.value =
accessToken
                tokensViewModel.refreshToken.value =
refreshToken
            }
            getMe(retrofit)
        } else {
            TokenManager.clearTokens(requireContext())
            activity?.runOnUiThread {
                tokensViewModel.accessToken.value = null
                tokensViewModel.refreshToken.value = null
            }
            null
        }
    }
    } else
        null
    } else
        null

    }
} catch (e: Exception) {
    return null
}
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = ArticleFragment()
}

override fun onDeleteBtnClick(comment: Comment) {
    if (retrofit == null)
        return
    val builder = AlertDialog.Builder(context)
    builder.setMessage("Вы уверены что хотите удалить комментарий?")
    builder.setCancelable(false)
    builder.setPositiveButton("Удалить") { dialog, id ->

        CoroutineScope(Dispatchers.IO).launch {
            delComment(comment, retrofit!!)
        }
    }
}

```

```

        }
        .setNegativeButton("Отмена") { dialog, id ->
            dialog.dismiss()
        }
        val alert = builder.create()
        alert.show()
    }

    private suspend fun delComment(comment: Comment, retrofit: Retrofit) {
        try {
            val adminApi = retrofit.create(AdminApi::class.java)
            val authApi = retrofit.create(AuthApi::class.java)
            val authResponse =
authApi.refreshAccessToken(RefreshTokenRequest(refreshToken!!))
            if (authResponse.code() == 200) {
                TokenManager.saveTokens(
                    requireContext(), accessToken!!,
                    refreshToken!!
                )
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = accessToken
                    tokensViewModel.refreshToken.value = refreshToken
                }
                val responseCode = adminApi.deleteCommentById(
                    comment.id!!, "Bearer_$accessToken"
                ).code()
                if (responseCode == 200) {
                    getComments(retrofit)
                    activity?.runOnUiThread {
                        Toast.makeText(
                            requireContext(),
                            "Комментарий удален!",
                            Toast.LENGTH_SHORT
                        ).show()
                        binding.commentCountView.text =
(binding.commentCountView.text.toString().toInt() - 1).toString()
                    }
                }
            }
        } catch (e: Exception) {
            activity?.runOnUiThread {
                Toast.makeText(context, "Ошибка удаления комментария",
                    Toast.LENGTH_SHORT).show()
            }
        }
    }
}

```

DraftsFragment.kt

```
package com.example.guitarlessons.fragments
```

```

import android.annotation.SuppressLint
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.adapter.DraftAdapter

```

```

import com.example.guitarlessons.databinding.FragmentDraftsBinding
import com.example.guitarlessons.retrofit.article.Article
import com.example.guitarlessons.retrofit.article.ArticleApi
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.viewmodel.DraftViewModel
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class DraftsFragment : Fragment(), DraftAdapter.ClickListener {
    private lateinit var binding: FragmentDraftsBinding
    private lateinit var draftAdapter: DraftAdapter
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private var accessToken: String? = null
    private var refreshToken: String? = null

    private var _retrofit: Retrofit? = null

    private val draftViewModel: DraftViewModel by activityViewModels()
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentDraftsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        draftAdapter = DraftAdapter(this)
        binding.draftRecyclerView.layoutManager = LinearLayoutManager(context)
        binding.draftRecyclerView.adapter = draftAdapter
        if ((activity as MainActivity).isConnected()) {
            val interceptor = HttpLoggingInterceptor()
            interceptor.level = HttpLoggingInterceptor.Level.BODY

            val client = OkHttpClient.Builder()
                .addInterceptor(interceptor)
                .connectTimeout(50, TimeUnit.MILLISECONDS)
                .build()

            val retrofit = Retrofit
                .Builder()
                .baseUrl(BuildConfig.API_URL).client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build()
            _retrofit = retrofit
        }
    }

```

```

        getDrafts()
    }
    binding.backSettingsBtn.setOnClickListener {
        replaceFragmentOnView(SettingsFragment.newInstance())
    }

    binding.messageEmptyBtn.setOnClickListener {
        replaceFragmentOnView(EditorFragment.newInstance())
    }
}

private fun getDrafts() {
    if (_retrofit == null)
        return
    val articleApi = _retrofit!!.create(ArticleApi::class.java)
    CoroutineScope(Dispatchers.IO).launch {
        val responseArticles =
            articleApi.getAllDrafts("Bearer ${accessToken}")
        if (responseArticles.code() == 200) {
            activity?.runOnUiThread {
                if (responseArticles.body()?.articles?.isEmpty() == true)
                    binding.MessageEmptyHolder.visibility = View.VISIBLE
                else
                    binding.draftRecyclerView.visibility = View.VISIBLE
            }
        }
        draftAdapter.submitList(responseArticles.body()?.articles)
    }
}

}

}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = DraftsFragment()
}

override fun onClick(article: Article) {
    draftViewModel.id.value = article.id
    replaceFragmentOnView(EditorFragment.newInstance())
}

override fun onDeleteBtnClick(article: Article) {
    CoroutineScope(Dispatchers.IO).launch {
        delDraft(article)
    }
}

private suspend fun delDraft(article: Article) {
    if (_retrofit == null)
        return

```

```

        val authApi: AuthApi = _retrofit!!.create(AuthApi::class.java)
        val articleApi: ArticleApi =
            _retrofit!!.create(ArticleApi::class.java)
        val responseCode: Int =
            articleApi.deleteArticleById(article.id!!,
                "Bearer_${accessToken}").code()
        if (responseCode == 200) {
            getDrafts()
        } else if (responseCode == 500) {
            if (refreshToken != null) {
                val responseToken: Response<TokenResponse> =
                    authApi.refreshAccessToken(
                        RefreshTokenRequest(
                            refreshToken!!
                        )
                    )
                if (responseToken.code() == 200) {
                    accessToken =
                        responseToken.body()?.accessToken.toString()
                    TokenManager.saveTokens(
                        requireContext(), accessToken!!,
                        refreshToken!!
                    )
                    activity?.runOnUiThread {
                        tokensViewModel.accessToken.value = accessToken
                        tokensViewModel.refreshToken.value = refreshToken
                    }
                    delDraft(article)
                } else {
                    TokenManager.clearTokens(requireContext())
                    activity?.runOnUiThread {
                        tokensViewModel.accessToken.value = null
                        tokensViewModel.refreshToken.value = null
                    }
                }
            }
        }
    }
}

```

EditorFragment.kt

```
package com.example.guitarlessons.fragments
```

```

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.content.res.Resources
import android.net.Uri
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.AdapterView
import android.widget.AdapterView.Adapter
import android.widget.ArrayAdapter
import android.widget.Toast
import androidx.activity.result.ActivityResultLauncher
import androidx.activity.result.contract.ActivityResultContracts
import androidx.core.view.forEach
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R

```

```

import com.example.guitarlessons.databinding.FragmentEditorBinding
import com.example.guitarlessons.retrofit.article.ArticleApi
import com.example.guitarlessons.retrofit.article.ArticleDto
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.upload.UploadApi
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.viewmodel.DraftViewModel
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.MediaType.Companion.toMediaTypeOrNull
import okhttp3.MultipartBody
import okhttp3.OkHttpClient
import okhttp3.RequestBody
import okhttp3.RequestBody.Companion.asRequestBody
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.io.File
import java.io.FileOutputStream
import java.io.IOException
import java.util.concurrent.TimeUnit

class EditorFragment : Fragment() {

    private val tokensViewModel: TokensViewModel by activityViewModels()

    private val draftViewModel: DraftViewModel by activityViewModels()

    private var accessToken: String? = null

    private var refreshToken: String? = null

    private lateinit var binding: FragmentEditorBinding

    private lateinit var images: MutableList<Uri>

    private lateinit var getContent: ActivityResultLauncher<String>

    private var skillLevel: Long = 0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        getContent =
            registerForActivityResult(ActivityResultContracts.GetContent()) { uri: Uri? -
>
                val size = Resources.getSystem().displayMetrics.widthPixels / 3.2
                uri?.let { images.add(it) }
                binding.editor.insertImage(uri.toString(), "img", size.toInt());
            }
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentEditorBinding.inflate(inflater, container, false)
    }

```

```

        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        images = mutableListOf()

        startHide()
        initEditor()
        try {
            val retrofit = initConnection()
            if ((activity as MainActivity).isConnected()) {
                if (accessToken != null) {
                    CoroutineScope(Dispatchers.IO).launch {
                        val user: User? = getMe(retrofit)
                        activity?.runOnUiThread {
                            if (user != null)
                                autherized(user)
                        }
                    }
                }
            }
            initBtn(retrofit)

            draftViewModel.id.observe(viewLifecycleOwner, {
                loadDraft(retrofit)
            })
        } catch (_: Exception) {
        }
    }

    private fun loadDraft(retrofit: Retrofit) {
        if (draftViewModel.id.value == null)
            return
        val articleApi = retrofit.create(ArticleApi::class.java)
        CoroutineScope(Dispatchers.IO).launch {
            val responseArticle =
                articleApi.getDraftById(draftViewModel.id.value!!, "Bearer_${accessToken}")
            if (responseArticle.code() == 200) {

                activity?.runOnUiThread {

                    binding.titleEditView.setText(responseArticle.body()!!.title)
                    binding.editor.html = responseArticle.body()!!.content
                }
            }
        }
    }

    private fun setupSpinner() {
        ArrayAdapter.createFromResource(
            requireContext(),
            R.array.skill_levels_array,
            android.R.layout.simple_spinner_item
        ).also { adapter ->
            // Specify the layout to use when the list of choices appears.
        }

        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
    }

```

```

)

        // Apply the adapter to the spinner.
        binding.skillLevelSpinner.adapter = adapter
    }
    binding.skillLevelSpinner.onItemSelectedListener =
        object : AdapterView.OnItemSelectedListener {
            override fun onItemSelected(
                parent: AdapterView<*>,
                view: View?,
                position: Int,
                id: Long
            ) {
                skillLevel = position.toLong()
            }

            override fun onNothingSelected(parent: AdapterView<*>) {
                skillLevel = 0
            }
        }
    }

    private fun authorized(user: User) {
        binding.apply {
            setupSpinner()
            skillLevelSpinner.visibility = View.VISIBLE
            skillLevelView.visibility = View.VISIBLE
        }
    }

    private fun initEditor() {
        binding.apply {

            setIsEnableAll(false, 0.5f)

            editor.setOnFocusChangeListener { _, b ->
                if (b)
                    setIsEnableAll(true, 1.0f)
                else
                    setIsEnableAll(false, 0.5f)
            }
            editor.setEditorHeight(200);
            editor.setEditorFontSize(20);
            editor.setPadding(10, 10, 10, 10);
            editor.setPlaceholder("Insert text here...");
        }
    }

    private fun initConnection(): Retrofit {
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })

        val interceptor = HttpLoggingInterceptor()
        interceptor.level = HttpLoggingInterceptor.Level.BODY

        val client = OkHttpClient.Builder()
            .addInterceptor(interceptor)
            .connectTimeout(50, TimeUnit.MILLISECONDS)
    }

```

```

        .build()

    val retrofit = Retrofit
        .Builder()
        .baseUrl(BuildConfig.API_URL) .client(client)
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    return retrofit
}

private fun initBtn(retrofit: Retrofit) {
    binding.apply {
        backBtn.setOnClickListener {
            val builder = AlertDialog.Builder(context)
            builder.setMessage("Сохранить в черновик?")
            .setCancelable(false)
            .setPositiveButton("Сохранить") { dialog, id ->
                CoroutineScope(Dispatchers.IO).launch {
                    if (uploadDraft(retrofit)) {
                        if (draftViewModel.id.value != null)

replaceFragmentOnView(DraftsFragment.newInstance())
                            else

replaceFragmentOnView(LessonsFragment.newInstance())
                                }
                            }
                        .setNegativeButton("Выйти") { dialog, id ->
                            if (draftViewModel.id.value != null)

replaceFragmentOnView(DraftsFragment.newInstance())
                                else

replaceFragmentOnView(LessonsFragment.newInstance())
                                    }
                                val alert = builder.create()
                                alert.show()
                            }
                        revert.setOnClickListener {
                            startHide()
                        }
                        H.setOnClickListener {
                            layoutBtn.forEach { child ->
                                child.visibility = View.GONE
                            }
                            revert.visibility = View.VISIBLE
                            H2.visibility = View.VISIBLE
                            H3.visibility = View.VISIBLE
                            T.visibility = View.VISIBLE
                        }
                        draftBtn.setOnClickListener {
                            CoroutineScope(Dispatchers.IO).launch {
                                uploadDraft(retrofit)
                            }
                        }
                        publishBtn.setOnClickListener {
                            CoroutineScope(Dispatchers.IO).launch {
                                if (replacePathImage(retrofit))
                                    uploadArticle(retrofit)
                                else
                                    {

```

```

        activity?.runOnUiThread {
            Toast.makeText(context, "Не удалось опубликовать
статью", Toast.LENGTH_SHORT).show()
        }
    }

    H3.setOnClickListener {
        editor.setHeading(3)
        startHide()
    }
    H2.setOnClickListener {
        editor.setHeading(2)
        startHide()
    }
    T.setOnClickListener {
        editor.removeFormat()
        startHide()
    }
    indentBtn.setOnClickListener {
        editor.setIndent()
    }
    outdentBtn.setOnClickListener {
        editor.setOutdent()
    }
    textStyleBtn.setOnClickListener {
        layoutBtn.forEach { child ->
            child.visibility = View.GONE
        }
        revert.visibility = View.VISIBLE
        bold.visibility = View.VISIBLE
        italic.visibility = View.VISIBLE
        underline.visibility = View.VISIBLE
        crossed.visibility = View.VISIBLE
    }
    bold.setOnClickListener {
        editor.setBold()
        startHide()
    }
    italic.setOnClickListener {
        editor.setItalic()
        startHide()
    }
    underline.setOnClickListener {
        editor.setUnderline()
        startHide()
    }
    crossed.setOnClickListener {
        editor.setStrikeThrough()
        startHide()
    }

    alignGroup.setOnClickListener {
        layoutBtn.forEach { child ->
            child.visibility = View.GONE
        }
        revert.visibility = View.VISIBLE
        alignLeft.visibility = View.VISIBLE
        alignCenter.visibility = View.VISIBLE
        alignRight.visibility = View.VISIBLE
    }

    alignLeft.setOnClickListener {

```

```

        editor.setAlignLeft()
        startHide()
    }
    alignCenter.setOnClickListener {
        editor.setAlignCenter()
        startHide()
    }
    alignRight.setOnClickListener {
        editor.setAlignRight()
        startHide()
    }
    listGroup.setOnClickListener {
        layoutBtn.forEach { child ->
            child.visibility = View.GONE
        }
        revert.visibility = View.VISIBLE
        bullets.visibility = View.VISIBLE
        numbers.visibility = View.VISIBLE
    }
    bullets.setOnClickListener {
        editor.setBullets()
        startHide()
    }
    numbers.setOnClickListener {
        editor.setNumbers()
        startHide()
    }
    H.setOnClickListener {
        layoutBtn.forEach { child ->
            child.visibility = View.GONE
        }
        revert.visibility = View.VISIBLE
        H2.visibility = View.VISIBLE
        H3.visibility = View.VISIBLE
        T.visibility = View.VISIBLE
    }
    loadImageBtn.setOnClickListener {
        getContent.launch("image/")
    }
}

}

private fun startHide() {
    binding.apply {
        layoutBtn.forEach { child ->
            child.visibility = View.GONE
        }
        H.visibility = View.VISIBLE
        indentBtn.visibility = View.VISIBLE
        outdentBtn.visibility = View.VISIBLE
        textStyleBtn.visibility = View.VISIBLE
        alignGroup.visibility = View.VISIBLE
        listGroup.visibility = View.VISIBLE
        loadImageBtn.visibility = View.VISIBLE
    }
}

private fun convertUriToFile(imgUri: Uri): File? {
    try {
        val inputStream =
requireContext().contentResolver.openInputStream(imgUri)

```

```

        val file = File.createTempFile("image", ".jpg",
requireContext().cacheDir)

        val outputStream = FileOutputStream(file)
        val buffer = ByteArray(4096)
        var bytesRead: Int

        while (inputStream?.read(buffer).also { bytesRead = it!! } != -1)
        {
            outputStream.write(buffer, 0, bytesRead)
        }

        return file
    } catch (e: IOException) {
        e.printStackTrace()
        return null
    }
}

private suspend fun replacePathImage(retrofit: Retrofit): Boolean {
    try {
        if (images.isEmpty()) {
            return true
        }
        var content = binding.editor.html
        val multiParts: MutableList<MultipartBody.Part> = mutableListOf()
        images.removeAll { image ->
            !content.contains(image.toString(), ignoreCase = true)
        }
        val uploadApi: UploadApi = retrofit.create(UploadApi::class.java)
        images.forEach { imgUri ->
            val file: File? = convertUriToFile(imgUri)
            if (file != null) {
                val mediaType = "image/*".toMediaTypeOrNull()
                val requestImage: RequestBody =
file.asRequestBody(mediaType)
                val body: MultipartBody.Part =
MultipartBody.Part.createFormData(
                    "images",
                    file.name,
                    requestImage
                )
                multiParts.add(body)
            }
        }

        val responseImage: Response<List<String>> =
uploadApi.uploadManyImage(multiParts)
        if (responseImage.code() == 200) {
            val urls = responseImage.body()
            var i = 0
            images.forEach { localImage ->
                content =
                    content.replace(localImage.toString(),
urls?.get(i)!!, ignoreCase = true)
                i++
            }
            withContext(Dispatchers.Main) {
                binding.editor.html = content
            }
        }
    }
}

```

```

        return true
    }

    } catch (e: Exception) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Ошибка загрузки изображений",
Toast.LENGTH_SHORT).show()

        }
        return false
    }

    return false
}

private fun isArticleValid(): Boolean {
    var isValid = true
    if (binding.editor.html == null ||
        binding.editor.html.length < 200
    ) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Слишком короткая статья",
Toast.LENGTH_LONG).show()
        }
        isValid = false
    } else if (binding.editor.html.length > 20000) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Слишком длинная статья",
Toast.LENGTH_LONG).show()
        }
        isValid = false
    } else if (binding.titleEditView.text.length < 30) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Слишком короткий заголовок",
Toast.LENGTH_LONG).show()
        }
        isValid = false
    } else if (binding.titleEditView.text.length > 255) {
        activity?.runOnUiThread {
            Toast.makeText(context, "Слишком длинный заголовок",
Toast.LENGTH_LONG).show()
        }
        isValid = false
    }

    return isValid
}

private suspend fun uploadArticle(retrofit: Retrofit) {
    if (!isArticleValid())
        return
    insertArticle(retrofit, false)
}

private suspend fun insertArticle(retrofit: Retrofit, isDraft: Boolean) {
    val articleApi: ArticleApi = retrofit.create(ArticleApi::class.java)
    val authApi: AuthApi = retrofit.create(AuthApi::class.java)
    val responseNewArticle = articleApi.addOrUpdate(
        ArticleDto(

```

```

        id = draftViewModel.id.value,
        title = binding.titleEditView.text.toString(),
        content = binding.editor.html,
        skillLevel = skillLevel + 1,
        isDraft = isDraft
    ), "Bearer_${accessToken}"
)

if (responseNewArticle.code() == 200) {
    activity?.runOnUiThread {
        if (isDraft)
            Toast.makeText(context, "Черновик сохранен",
Toast.LENGTH_LONG).show()
        else
            Toast.makeText(context, "Статья опубликована",
Toast.LENGTH_LONG).show()
    }
    replaceFragmentOnView(LessonsFragment.newInstance())
} else {

    if (responseNewArticle.code() == 409) {
        activity?.runOnUiThread {
            Toast.makeText(
                context,
                "Урок с таким заголовком уже существует",
                Toast.LENGTH_LONG
            ).show()
        }

    } else if (responseNewArticle.code() == 500) {
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                RefreshTokenRequest(
                    refreshToken!!
                )
            )
            if (responseToken.code() == 200) {
                accessToken =
responseToken.body()?.accessToken.toString()
                TokenManager.saveTokens(
                    requireContext(), accessToken!!,
                    refreshToken!!
                )
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = accessToken
                    tokensViewModel.refreshToken.value = refreshToken
                }
                insertArticle(retrofit, isDraft)
            } else {
                TokenManager.clearTokens(requireContext())
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = null
                    tokensViewModel.refreshToken.value = null
                }
            }
        }
    }
}

}

private suspend fun uploadDraft(retrofit: Retrofit): Boolean {

```

```

        if (binding.titleEditView.text.isEmpty()) {
            activity?.runOnUiThread {
                Toast.makeText(context, "Заголовок не должен быть пустым",
                    Toast.LENGTH_SHORT).show()
            }
            return false
        }
        if (binding.editor.html == null) {
            activity?.runOnUiThread {
                Toast.makeText(context, "Статья не должна быть пустой",
                    Toast.LENGTH_SHORT).show()
            }
            return false
        }
        insertArticle(retrofit, true)
        return true
    }

    private suspend fun getMe(retrofit: Retrofit): User? {
        try {
            val authApi: AuthApi = retrofit.create(AuthApi::class.java)
            return withContext(Dispatchers.IO) {
                val responseUser: Response<User> =
                    authApi.getMe("Bearer_${accessToken}")
                if (responseUser.isSuccessful) { // Проверка статуса ответа
                    responseUser.body()
                } else if (responseUser.code() == 500) {
                    if (refreshToken != null) {
                        val responseToken: Response<TokenResponse> =
                            authApi.refreshAccessToken(
                                RefreshTokenRequest(
                                    refreshToken!!
                                )
                            )
                        if (responseToken.code() == 200) {
                            accessToken =
                                responseToken.body()?.accessToken.toString()
                            TokenManager.saveTokens(
                                requireContext(), accessToken!!,
                                refreshToken!!
                            )
                            activity?.runOnUiThread {
                                tokensViewModel.accessToken.value =
                                    accessToken
                                tokensViewModel.refreshToken.value =
                                    refreshToken
                            }
                            getMe(retrofit)
                        } else {
                            TokenManager.clearTokens(requireContext())
                            activity?.runOnUiThread {
                                tokensViewModel.accessToken.value = null
                                tokensViewModel.refreshToken.value = null
                            }
                            null
                        }
                    }
                } else {
                    null
                }
            }
        } catch (e: Exception) {

```

```

        return null
    }
}

private fun setIsEnableAll(state: Boolean, a: Float) {
    binding.apply {
        layoutBtn.forEach { child ->
            child.isEnabled = state
            child.alpha = a
        }
    }
}

override fun onDestroy() {
    super.onDestroy()
    if (draftViewModel.id.value != null)
        draftViewModel.id.value = null
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = EditorFragment()
}
}

```

LessonsFragment.kt

```
package com.example.guitarlessons.fragments
```

```

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.AdapterView
import android.widget.AdapterView.AdapterView
import android.widget.ArrayAdapter
import android.widget.SearchView
import android.widget.TextView
import android.widget.Toast
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.lifecycle.LifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.adapter.ArticleAdapter
import com.example.guitarlessons.databinding.FragmentLessonsBinding
import com.example.guitarlessons.retrofit.admin.AdminApi
import com.example.guitarlessons.retrofit.article.Article
import com.example.guitarlessons.retrofit.article.ArticleApi

```

```

import com.example.guitarlessons.retrofit.article.PageableArticles
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.utils.addOnScrollListenerToEnd
import com.example.guitarlessons.utils.removeOnScrollListenerToEnd
import com.example.guitarlessons.viewmodel.ArticleViewModel
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class LessonsFragment : Fragment(), ArticleAdapter.ClickListener {
    private val startLimit: Int = 10
    private lateinit var articleAdapter: ArticleAdapter
    private lateinit var binding: FragmentLessonsBinding
    private val articleViewModel: ArticleViewModel by activityViewModels()
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private var accessToken: String? = null
    private var refreshToken: String? = null
    private var offset: Int = 0
    private var limit: Int = startLimit
    private var sortField: String = "id"
    private var isThisLoading = false
    private var me: User? = null
    private var isAuth: Boolean = false
    private var isAdmin: Boolean = false
    private var roleId: Long = 2
    private var retrofit: Retrofit? = null

    private lateinit var likeTextView: TextView

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?,
    ): View {
        binding = FragmentLessonsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        setupSpinner()
        articleAdapter = ArticleAdapter(this)
        binding.apply {
            rcView.layoutManager = LinearLayoutManager(context)
            rcView.adapter = articleAdapter

```

```

        reloadBtn.setOnClickListener {
            replaceFragmentOnView(newInstance())
        }

    }

    binding.rcView.addOnScrollListenerToEnd(binding.loadMoreBtn)

    if((activity as MainActivity).isConnected()){
        try {
            //Если есть интернет соединение
            authorized()

            val interceptor = HttpLoggingInterceptor()
            interceptor.level = HttpLoggingInterceptor.Level.BODY

            val client = OkHttpClient.Builder()
                .addInterceptor(interceptor)
                .connectTimeout(50, TimeUnit.MILLISECONDS)
                .build()

            val rf = Retrofit
                .Builder()
                .baseUrl(BuildConfig.API_URL).client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build()
            retrofit = rf
            lifecycleScope.launch {
                val user = getMe(retrofit!!)
                if (user != null) {

                    me = user
                    isAuth = true
                    if (user.role!!.roleName == "ADMIN"){
                        isAdmin = true
                    }
                }
                articleAdapter.changeVisibility(isAdmin)
            }
            //Вывод всех продуктов в RecyclerView
            setupSpinnerListener(retrofit!!)

            binding.editorBtn.setOnClickListener{
                CoroutineScope(Dispatchers.IO).launch {
                    if (isAuth)

replaceFragmentOnView(EditorFragment.newInstance())
                    else

replaceFragmentOnView(LoginFragment.newInstance())
                }
            }
            binding.loadMoreBtn.setOnClickListener {
                loadMoreData(retrofit!!)
                binding.loadMoreBtn.visibility = View.GONE
                binding.rcView.smoothScrollBy(0,100)
            }
        }
    }

```

```

        }

        binding.sv.setOnQueryTextListener(object :
SearchView.OnQueryTextListener {
            override fun onQueryTextSubmit(text: String?): Boolean {
                CoroutineScope(Dispatchers.IO).launch {
                    val articleApi =
retrofit!!.create(ArticleApi::class.java)
                    val responseArticles = text?.let {
articleApi.searchAllPublishedByTitle(it, roleId) }
                    if (responseArticles!!.code() == 200) {
                        activity?.runOnUiThread {

articleAdapter.submitList(responseArticles.body())
                        }
                    }
                }
                return true
            }

            override fun onQueryTextChange(p0: String?): Boolean {
                return true
            }
        })
    } catch (e: Exception) {
        unauthorized()
    }

}
else{
    //Если нет интернет соединения
    unauthorized()
}

}

private fun chooseTypeArticles(sort:String){
    CoroutineScope(Dispatchers.IO).launch {
        sortField = sort
        when(binding.contentSpin.selectedItemPosition){
            0 -> {
                limit = startLimit
                getAllMainArticles(retrofit!!)
            }
            1 -> {
                limit = startLimit
                getAllCommunityArticles(retrofit!!)
            }
        }
    }
}

private fun loadMoreData(retrofit: Retrofit){
    CoroutineScope(Dispatchers.IO).launch {
        when(binding.contentSpin.selectedItemPosition){
            0 -> getAllMainArticles(retrofit)
            1 -> getAllCommunityArticles(retrofit)
        }
    }
}

private fun setupSpinner(){
    ArrayAdapter.createFromResource(

```

```

        requireContext(),
        R.array.type_lesson_array,
        android.R.layout.simple_spinner_item
    ).also { adapter ->

adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item
)
        binding.contentSpin.adapter = adapter
    }

    ArrayAdapter.createFromResource(
        requireContext(),
        R.array.sort_article_array,
        android.R.layout.simple_spinner_item
    ).also { adapter ->

adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item
)
        binding.sortSpinner.adapter = adapter
    }

}

private fun setupSpinnerListener(retrofit: Retrofit){
    binding.sortSpinner.onItemSelectedListener = object :
AdapterView.OnItemSelectedListener {
        override fun onItemSelected(parent: AdapterView<*>, view: View?,
position: Int, id: Long) {
            when(position){
                0 -> chooseTypeArticles("id")
                1 -> chooseTypeArticles("publicationDate")
                2 -> chooseTypeArticles("rating")
                3 -> chooseTypeArticles("skillLevel.id")
                else -> chooseTypeArticles("id")
            }
        }

        override fun onNothingSelected(parent: AdapterView<*>) {
            chooseTypeArticles("id")
        }
    }

    binding.contentSpin.onItemSelectedListener = object :
AdapterView.OnItemSelectedListener {
        override fun onItemSelected(parent: AdapterView<*>, view: View?,
position: Int, id: Long) {
            limit = startLimit

            CoroutineScope(Dispatchers.IO).launch {
                when(position){
                    0 -> {
                        roleId = 2
                        getAllMainArticles(retrofit)
                    }
                    1 -> {
                        roleId = 1
                        getAllCommunityArticles(retrofit)
                    }
                }
            }
        }
    }
}

```

```

        override fun onNothingSelected(parent: AdapterView<*>) {

            CoroutineScope(Dispatchers.IO).launch {
                getAllMainArticles(retrofit)
            }
        }
    }

    private fun authorized(){
        binding.apply {
            rcView.visibility = View.VISIBLE
            ConnectionLostMessage.visibility = View.GONE
        }
    }

    private fun unauthorized(){
        binding.apply {
            rcView.visibility = View.GONE
            ConnectionLostMessage.visibility = View.VISIBLE
        }
    }

    private suspend fun getMe(retrofit: Retrofit): User? {
        try {
            val authApi: AuthApi = retrofit.create(AuthApi::class.java)
            return withContext(Dispatchers.IO) {
                val responseUser: Response<User> =
authApi.getMe("Bearer_${accessToken}")
                if (responseUser.isSuccessful) {
                    // Проверка статуса ответа
                    responseUser.body()
                } else if (responseUser.code() == 500) {
                    if (refreshToken != null) {
                        val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                            RefreshTokenRequest(
                                refreshToken!!
                            )
                        )
                        if (responseToken.code() == 200) {
                            accessToken =
responseToken.body()?.accessToken.toString()
                            TokenManager.saveTokens(
                                requireContext(), accessToken!!,
                                refreshToken!!
                            )
                            activity?.runOnUiThread {
                                tokensViewModel.accessToken.value =
accessToken
                                tokensViewModel.refreshToken.value =
refreshToken
                            }
                        }
                        getMe(retrofit)
                    } else {
                        TokenManager.clearTokens(requireContext())
                        activity?.runOnUiThread {
                            tokensViewModel.accessToken.value = null
                            tokensViewModel.refreshToken.value = null
                        }
                        null
                    }
                }
            }
        }
    }

```

```

        } else
            null
    } else
        null
    }

} catch (e: Exception) {
    return null
}
}

private suspend fun getAllMainArticles(retrofit: Retrofit){
    try {
        val articleApi = retrofit.create(ArticleApi::class.java)
        Log.e("sortfield", sortField)
        val responseArticle: Response<PageableArticles> =
            articleApi.getAllMain(offset, limit, sortField)
        if (responseArticle.code() == 200){
            activity?.runOnUiThread {

                articleAdapter.submitList(responseArticle.body()!!.content)
                //articleAdapter.notifyItemRangeChanged(0,
                articleAdapter.itemCount);
                limit+=startLimit
                isThisLoading = false
            }
        }

    } catch (e:Exception){
        activity?.runOnUiThread {
            isThisLoading = false
            Toast.makeText(context, "Ошибка подключения к серверу",
                Toast.LENGTH_SHORT).show()
            unauthorized()
        }
    }

}

private suspend fun getAllCommunityArticles(retrofit: Retrofit){
    try {
        val articleApi = retrofit.create(ArticleApi::class.java)
        val responseArticle: Response<PageableArticles> =
            articleApi.getAllCommunity(offset, limit, sortField)
        if (responseArticle.code() == 200){
            activity?.runOnUiThread {

                articleAdapter.submitList(responseArticle.body()!!.content)
                limit+=startLimit
                isThisLoading = false
            }
        }

    } catch (e:Exception){
        activity?.runOnUiThread {
            isThisLoading = false
            Toast.makeText(context, "Ошибка подключения к серверу",
                Toast.LENGTH_SHORT).show()
            unauthorized()
        }
    }

}

```

```

    }
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = LessonsFragment()
}

override fun onClick(article: Article) {

    articleViewModel.id.value = article.id
    replaceFragmentOnView(ArticleFragment.newInstance())
}

override fun onLikeClick(article: Article, likeCount: TextView) {

    val articleApi = retrofit!!.create(ArticleApi::class.java)
    CoroutineScope(Dispatchers.IO).launch {
        if (me != null) {
            val response = articleApi.like(me!!.id!!, article.id!!,
"Bearer_${accessToken}")
            if (response.code() == 200) {
                activity?.runOnUiThread {
                    likeCount.text = response.body()!!.count.toString()
                }
            }
        }
        else{
            activity?.runOnUiThread {
                Toast.makeText(
                    context,
                    "Для этого необходимо авторизоваться",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }
    }
}

}

override fun onAdminDelClick(article: Article) {
    val builder = AlertDialog.Builder(context)
    builder.setMessage("Вы уверены что хотите удалить урок?")
        .setCancelable(false)
        .setPositiveButton("Удалить") { dialog, id ->

            CoroutineScope(Dispatchers.IO).launch {
                delPost(article)
            }
        }
        .setNegativeButton("Отмена") { dialog, id ->
            dialog.dismiss()
        }
    val alert = builder.create()
    alert.show()
}

```

```

    }

    private suspend fun delPost(article: Article) {
        if (retrofit == null)
            return
        val authApi: AuthApi = retrofit!!.create(AuthApi::class.java)
        val adminApi: AdminApi = retrofit!!.create(AdminApi::class.java)
        val responseCode: Int =
            adminApi.deleteArticleById(article.id!!,
"Bearer_${accessToken}").code()
        if (responseCode == 200) {
            chooseTypeArticles(sortField)
        } else if (responseCode == 500) {
            if (refreshToken != null) {
                val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                    RefreshTokenRequest(
                        refreshToken!!
                    )
                )
                if (responseToken.code() == 200) {
                    accessToken =
responseToken.body()?.accessToken.toString()
                    TokenManager.saveTokens(
                        requireContext(), accessToken!!,
                        refreshToken!!
                    )
                    activity?.runOnUiThread {
                        tokensViewModel.accessToken.value = accessToken
                        tokensViewModel.refreshToken.value = refreshToken
                    }
                    delPost(article)
                } else {
                    TokenManager.clearTokens(requireContext())
                    activity?.runOnUiThread {
                        tokensViewModel.accessToken.value = null
                        tokensViewModel.refreshToken.value = null
                    }
                }
            }
        }
    }
}

}

}

override fun onDestroyView() {
    super.onDestroyView()
    binding.rcView.removeOnScrollListenerToEnd()
}
}

```

LoginFragment.kt

```

package com.example.guitarlessons.fragments

import android.annotation.SuppressLint
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.core.widget.doAfterTextChanged
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import com.example.guitarlessons.BuildConfig

```

```

import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentLoginBinding
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.auth.AuthRequest
import com.example.guitarlessons.retrofit.auth.AuthResponse
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

class LoginFragment : Fragment() {

    private lateinit var binding: FragmentLoginBinding

    private val tokensViewModel: TokensViewModel by activityViewModels()

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentLoginBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val interceptor = HttpLoggingInterceptor()
        interceptor.level = HttpLoggingInterceptor.Level.BODY

        val client = OkHttpClient.Builder()
            .addInterceptor(interceptor)
            .build()

        val retrofit = Retrofit
            .Builder()
            .baseUrl(BuildConfig.API_URL).client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        binding.apply {

            passwordET.doAfterTextChanged {
                if (!passwordIsValid())
                    passwordErrTV.visibility = View.GONE
            }

            usernameET.doAfterTextChanged {
                if (!usernameIsValid())
                    usernameErrTV.visibility = View.GONE
            }

            regBtn.setOnClickListener {

```

```

        if ((activity as MainActivity).isConnected()) {
            replaceFragmentOnView(RegisterFragment.newInstance())
        } else
            Toast.makeText(
                activity, "Требуется интернет соединение",
                Toast.LENGTH_LONG
            ).show()
    }
    backButton.setOnClickListener {
        replaceFragmentOnView(SettingsFragment.newInstance())
    }
    //Авторизация пользователя
    loginBtn.setOnClickListener {

        if ((activity as MainActivity).isConnected()) {
            authorization(retrofit)
        } else {
            Toast.makeText(
                activity, "Требуется интернет соединение",
                Toast.LENGTH_LONG
            ).show()
        }
    }
}

}

private fun authorization(retrofit: Retrofit) {
    if (isValid())
        return

    val authApi: AuthApi = retrofit.create(AuthApi::class.java)
    CoroutineScope(Dispatchers.IO).launch {
        try {
            val response = authApi.auth(
                AuthRequest(
                    binding.usernameET.text.toString(),
                    binding.passwordET.text.toString()
                )
            )
            if (response.code() == 200) {
                val authResponse: AuthResponse = response.body()!!
                context?.let {
                    activity?.runOnUiThread {
                        tokensViewModel.accessToken.value =
authResponse.accessToken
                        tokensViewModel.refreshToken.value =
authResponse.refreshToken
                    }
                    TokenManager.saveTokens(
                        it,
                        authResponse.accessToken,
                        authResponse.refreshToken
                    )
                }
                activity?.runOnUiThread {
                    Toast.makeText(context, "Добро пожаловать",
Toast.LENGTH_LONG).show()
                    replaceFragmentOnView(SettingsFragment.newInstance())
                }
            }
            if (response.code() == 403) {
                activity?.runOnUiThread {
                    Toast.makeText(

```

```

        activity,
        "Неверный логин или пароль",
        Toast.LENGTH_LONG
    ).show()
    }
}
} catch (e: Exception) {
    activity?.runOnUiThread {
        Toast.makeText(
            activity,
            "Ошибка подключения к серверу",
            Toast.LENGTH_LONG
        ).show()
    }
    e.printStackTrace()
}
}

private fun isValid(): Boolean {
    var isValid = true
    binding.apply {
        if (passwordIsValid()) {
            passwordErrTV.visibility = View.VISIBLE
            isValid = false
        }
        if (usernameIsValid()) {
            usernameErrTV.visibility = View.VISIBLE
            isValid = false
        }
    }
    return isValid
}

private fun passwordIsValid(): Boolean {
    return binding.passwordET.text.isEmpty() ||
        binding.passwordET.text.length > 255 || binding.passwordET.text.length < 8
}

private fun usernameIsValid(): Boolean {
    return binding.usernameET.text.isEmpty() ||
        binding.usernameET.text.length > 50 || binding.usernameET.text.length < 4
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = LoginFragment()
}
}

```

MetronomeFragment.kt

```
package com.example.guitarlessons.fragments
```

```
import android.annotation.SuppressLint
```

```

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.SeekBar
import androidx.fragment.app.Fragment
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentMetronomeBinding
import com.example.guitarlessons.utils.Metronom

// TODO: Rename parameter arguments, choose names that match
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private const val ARG_PARAM1 = "param1"
private const val ARG_PARAM2 = "param2"

/**
 * A simple [Fragment] subclass.
 * Use the [MetronomeFragment.newInstance] factory method to
 * create an instance of this fragment.
 */
class MetronomeFragment : Fragment() {

    private var bpm: Long = 100
    private lateinit var binding: FragmentMetronomeBinding
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentMetronomeBinding.inflate(inflater, container,
false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        binding.apply {
            bpmValueSB.min = Metronom.BPM_LOWER_THRESHOLD.toInt()
            bpmValueSB.max = Metronom.BPM_UPPER_THRESHOLD.toInt()
            bpmValueSB.progress = bpm.toInt()
            bpmValueSB.setOnSeekBarChangeListener(object :
SeekBar.OnSeekBarChangeListener{
                override fun onProgressChanged(seekBar: SeekBar, progress:
Int, fromUser: Boolean) {
                    bpm = progress.toLong()
                    bpmView.text = "${bpm} BPM"
                }

                override fun onStartTrackingTouch(seekBar: SeekBar) {
                }

                override fun onStopTrackingTouch(seekBar: SeekBar) {
                    if (Metronom.isOn()) {
                        Metronom.stop()
                        Metronom.start(bpm)
                    }
                    bpmView.text = "${bpm} BPM"
                }
            })
            playBtn.setOnClickListener {
                Metronom.start(bpm)
                playBtn.visibility = View.GONE
                stopBtn.visibility = View.VISIBLE
            }
        }
    }
}

```

```

        stopBtn.setOnClickListener {
            Metronom.stop()
            playBtn.visibility = View.VISIBLE
            stopBtn.visibility = View.GONE
        }

        decrementBPMBtn.setOnClickListener {
            if (bpm > Metronom.BPM_LOWER_THRESHOLD) {
                bpm--
                bpmView.text = "${bpm} BPM"
                Metronom.stop()
                Metronom.start(bpm)
            }
        }

        incrementBPMBtn.setOnClickListener {
            if (bpm < Metronom.BPM_UPPER_THRESHOLD) {
                bpm++
                bpmView.text = "${bpm} BPM"
                Metronom.stop()
                Metronom.start(bpm)
            }
        }
    }
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = MetronomeFragment()
}
}

```

MidiFragment.kt

```

package com.example.guitarlessons.fragments

import android.annotation.SuppressLint
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.activity.result.ActivityResultLauncher
import androidx.activity.result.contract.ActivityResultContracts
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.lifecycle.LifecycleScope
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentMidiBinding
import com.example.guitarlessons.guitarPitch.Note

```

```

import com.example.guitarlessons.guitarPitch.NoteDurationDetector
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.tab.NoteRequest
import com.example.guitarlessons.retrofit.tab.TabApi
import com.example.guitarlessons.retrofit.tab.TabRequest
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.utils.PdfCreator
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.MediaType.Companion.toMediaTypeOrNull
import okhttp3.MultipartBody
import okhttp3.OkHttpClient
import okhttp3.RequestBody
import okhttp3.RequestBody.Companion.asRequestBody
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.io.File
import java.io.FileOutputStream
import java.io.IOException
import java.util.concurrent.TimeUnit

class MidiFragment : Fragment() {
    private lateinit var binding: FragmentMidiBinding

    private lateinit var getContent: ActivityResultLauncher<Array<String>>
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private var accessToken: String? = null
    private var refreshToken: String? = null
    private var fileUri: Uri? = null
    private var audioFile: File? = null
    private var fileName: String? = null
    private var isRecording: Boolean = false
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        getContent =
            registerForActivityResult(ActivityResultContracts.OpenDocument()) {
                { uri: Uri? ->
                    // Обработка выбранного URI (файла)
                    if (uri != null) {
                        fileUri = uri
                        fileName = getFileName(uri)
                        if (fileName != null) {
                            if (fileName!!.endsWith(".mid", ignoreCase = true) ||
                                fileName!!.endsWith(".midi", ignoreCase = true)
                            ) {
                                binding.sendHolder.visibility = View.VISIBLE
                                audioFile = convertUriToFile(fileUri!!, true)
                                binding.midiNameView.text = audioFile?.name ?:
                                    "Попробуйте ещё раз"

                            } else if (fileName!!.endsWith(".wav", ignoreCase =
true)) {
                                binding.sendHolder.visibility = View.VISIBLE
                                audioFile = convertUriToFile(fileUri!!, false)
                                binding.midiNameView.text = audioFile?.name ?:
                                    "Попробуйте ещё раз"

```

```

        }
    } else {
        println("Не удалось получить имя файла")
    }
}

}

private fun getFileName(uri: Uri): String? {
    var result: String? = null
    if (uri.scheme == "content") {
        requireContext().contentResolver.query(uri, null, null, null,
null)?.use { cursor ->
            if (cursor.moveToFirst()) {
                val displayNameIndex =
cursor.getColumnIndex(android.provider.OpenableColumns.DISPLAY_NAME)
                if (displayNameIndex != -1) {
                    result = cursor.getString(displayNameIndex)
                }
            }
        }
    }
    if (result == null) {
        result = uri.path
        val cut = result!!.lastIndexOf('/')
        if (cut != -1) {
            result = result!!.substring(cut + 1)
        }
    }
    return result
}

override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?,
    savedInstanceState: Bundle?,
): View {
    binding = FragmentMidiBinding.inflate(inflater, container, false)
    return binding.root
}

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
        accessToken = it
    })
    tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
        refreshToken = it
    })
    binding.switchFrom.setOnClickListener {
        binding.sendHolder.visibility = View.GONE
        binding.midiNameView.text = "Ваш файл"
        binding.tabsView.text = ""
        binding.saveHolder.visibility = View.GONE
        if (binding.switchFrom.isChecked) {
            binding.recordHolder.visibility = View.VISIBLE
            binding.loadFileHolder.visibility = View.GONE
            binding.saveTabBtn.visibility = View.VISIBLE
            binding.switchShare.visibility = View.VISIBLE
        } else {
            binding.recordHolder.visibility = View.GONE
            binding.loadFileHolder.visibility = View.VISIBLE
        }
    }
}

```

```

        binding.saveTabBtn.visibility = View.VISIBLE
        binding.switchShare.visibility = View.VISIBLE
    }
}
val interceptor = HttpLoggingInterceptor()
interceptor.level = HttpLoggingInterceptor.Level.BODY

val client = OkHttpClient.Builder()
    .addInterceptor(interceptor)
    .connectTimeout(50, TimeUnit.MILLISECONDS)
    .build()

val retrofit = Retrofit
    .Builder()
    .baseUrl(BuildConfig.API_URL).client(client)
    .addConverterFactory(GsonConverterFactory.create())
    .build()
binding.loadMidiBtn.setOnClickListener {
    val intent = Intent(Intent.ACTION_OPEN_DOCUMENT)
    intent.addCategory(Intent.CATEGORY_OPENABLE)
    intent.type = "*/*" // Указываем MIME-тип (любой)
    intent.putExtra(Intent.EXTRA_ALLOW_MULTIPLE, false)
    getContent().launch(arrayOf("*/*"))
}

val detector = NoteDurationDetector(48000f, 4096, 0, 0.01f)
binding.recordBtn.setOnClickListener {
    if (!isRecording) {
        detector.detectNoteDurationsFromMicrophone()
        isRecording = true
        binding.recordBtn.text = "СТОП"
    } else {
        val notes = detector.stopDetection()
        notes.forEach { Log.e("note", it.toString()) }
        lifecycleScope.launch {
            sendNotes(retrofit, notes)
        }
        isRecording = false
        binding.recordBtn.text = "запись"
    }
}

binding.toTabBtn.setOnClickListener {
    if (fileName!!.endsWith(".mid", ignoreCase = true) ||
        fileName!!.endsWith(".midi", ignoreCase = true)) {
        lifecycleScope.launch {
            uploadMidi(retrofit)
        }
    } else if (fileName!!.endsWith(".wav", ignoreCase = true)) {
        val file = convertUriToFile(fileUri!!, false)
        val notes: List<Note> =
detector.detectNoteDurationsFromFile(file!!)
        lifecycleScope.launch {
            sendNotes(retrofit, notes)
        }
    }
}

binding.saveTabBtn.setOnClickListener {
    lifecycleScope.launch {
        saveTabs(retrofit)
    }
}

```

```

    }

    binding.tabToPdfBtn.setOnClickListener {
        generatePdfForTabs()
    }
}

private fun generatePdfForTabs() {
    if (binding.titleTextView.text.toString().isEmpty() ||
binding.tabsView.text.toString().isEmpty()) {
        Toast.makeText(
            context,
            "Введите название табулатуры",
            Toast.LENGTH_SHORT
        ).show()
        return
    }

    PdfCreator.createPdf(binding.titleTextView.text.toString(), binding.tabsView.t
ext.toString(), requireContext())
}

private suspend fun saveTabs(retrofit: Retrofit) {
    if (binding.titleTextView.text.isEmpty()) {
        activity?.runOnUiThread {
            Toast.makeText(
                requireContext(),
                "Введите название табулатуры",
                Toast.LENGTH_SHORT
            ).show()
        }
        return
    }
    val tabApi = retrofit.create(TabApi::class.java)

    val response =
tabApi.save(TabRequest(binding.titleTextView.text.toString(),
binding.switchShare.isChecked, binding.tabsView.text.toString()),
"Bearer_${accessToken}")

    if (response.code() == 200) {
        if (binding.switchShare.isChecked) {
            activity?.runOnUiThread {
                Toast.makeText(
                    requireContext(),
                    "Табулатура сохранена в личном кабинете и
опубликована",
                    Toast.LENGTH_SHORT
                ).show()
                binding.saveTabBtn.visibility = View.GONE
                binding.switchShare.visibility = View.GONE
            }
        } else {
            activity?.runOnUiThread {
                Toast.makeText(
                    requireContext(),
                    "Табулатура сохранена в личном кабинете",
                    Toast.LENGTH_SHORT
                ).show()
                binding.saveTabBtn.visibility = View.GONE
                binding.switchShare.visibility = View.GONE
            }
        }
    }
}

```

```

    }

    }else if (response.code() == 500) {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
                authApi.refreshAccessToken(
                    RefreshTokenRequest(
                        refreshToken!!
                    )
                )
            if (responseToken.code() == 200) {
                accessToken =
responseToken.body()?.accessToken.toString()
                TokenManager.saveTokens(
                    requireContext(), accessToken!!,
                    refreshToken!!
                )
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = accessToken
                    tokensViewModel.refreshToken.value = refreshToken
                }
                saveTabs(retrofit)
            } else {
                TokenManager.clearTokens(requireContext())
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = null
                    tokensViewModel.refreshToken.value = null
                }
            }
        }
    }
    } else {
        activity?.runOnUiThread {
            Toast.makeText(
                requireContext(),
                "Не удалось сохранить табулатуру",
                Toast.LENGTH_SHORT
            ).show()
        }
    }
}

}

private fun convertUriToFile(imgUri: Uri, isMidi: Boolean): File? {
    try {
        val inputStream =
requireContext().contentResolver.openInputStream(imgUri)
        val file = if (isMidi)
            File.createTempFile("midi", ".mid",
requireContext().cacheDir)
        else
            File.createTempFile("wav", ".wav", requireContext().cacheDir)
        val outputStream = FileOutputStream(file)
        val buffer = ByteArray(4096) // Размер буфера
        var bytesRead: Int

        while (inputStream?.read(buffer).also { bytesRead = it!! } != -1)
        {
            outputStream.write(buffer, 0, bytesRead)
        }

        return file
    }
}

```

```

    } catch (e: IOException) {
        e.printStackTrace()
        return null // Или выбросьте исключение
    }
}

private suspend fun sendNotes(retrofit: Retrofit, notes: List<Note>) {
    val tabApi: TabApi = retrofit.create(TabApi::class.java)
    val response =
        tabApi.audio2midi(NoteRequest(binding.titleTextView.text.toString(), false,
            notes, 120), "Bearer_$accessToken")
    if (response.code() == 200) {
        activity?.runOnUiThread {
            val tabs = response.body()?.content
            activity?.runOnUiThread {
                binding.tabsView.text = tabs
                binding.saveHolder.visibility = View.VISIBLE
                binding.sendHolder.visibility = View.GONE
            }
        }
    } else if (response.code() == 500) {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
                authApi.refreshAccessToken(
                    RefreshTokenRequest(
                        refreshToken!!
                    )
                )
            if (responseToken.code() == 200) {
                accessToken =
                    responseToken.body()?.accessToken.toString()
                TokenManager.saveTokens(
                    requireContext(), accessToken!!,
                    refreshToken!!
                )
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = accessToken
                    tokensViewModel.refreshToken.value = refreshToken
                }
                sendNotes(retrofit, notes)
            } else {
                TokenManager.clearTokens(requireContext())
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = null
                    tokensViewModel.refreshToken.value = null
                }
            }
        }
    } else {
        activity?.runOnUiThread {
            Toast.makeText(
                requireContext(),
                "Не удалось получить табы",
                Toast.LENGTH_SHORT
            ).show()
        }
    }
}

private suspend fun uploadMidi(retrofit: Retrofit) {
    try {

```

```

        if (fileUri != null) {
            val tabApi: TabApi = retrofit.create(TabApi::class.java)
            if (audioFile != null) {
                val mediaType = "audio/mid".toMediaTypeOrNull()
                val requestMidi: RequestBody =
                    audioFile!!.asRequestBody(mediaType)
                val body: MultipartBody.Part =
                    MultipartBody.Part.createFormData(
                        "midi",
                        audioFile!!.name,
                        requestMidi
                    )
                return withContext(Dispatchers.IO) {
                    val responseTab = tabApi.midi2tabs(
                        body,
                        "Bearer_${accessToken}"
                    )
                    if (responseTab.code() == 200) {
                        activity?.runOnUiThread {
                            val tabs = responseTab.body()?.content
                            Log.e("URL", tabs.toString())
                            activity?.runOnUiThread {
                                binding.tabsView.text = tabs
                                binding.saveHolder.visibility =
                                    View.VISIBLE
                                binding.sendHolder.visibility = View.GONE
                            }
                        }
                    } else if (responseTab.code() == 500) {
                        val authApi: AuthApi =
                            retrofit.create(AuthApi::class.java)
                        if (refreshToken != null) {
                            val responseToken: Response<TokenResponse> =
                                authApi.refreshAccessToken(
                                    RefreshTokenRequest(
                                        refreshToken!!
                                    )
                                )
                            if (responseToken.code() == 200) {
                                accessToken =
                                    responseToken.body()?.accessToken.toString()
                                TokenManager.saveTokens(
                                    requireContext(), accessToken!!,
                                    refreshToken!!
                                )
                                activity?.runOnUiThread {
                                    tokensViewModel.accessToken.value =
                                        accessToken
                                    tokensViewModel.refreshToken.value =
                                        refreshToken
                                }
                                uploadMidi(retrofit)
                            } else {
                                TokenManager.clearTokens(requireContext())
                                activity?.runOnUiThread {
                                    tokensViewModel.accessToken.value =
                                        null
                                    tokensViewModel.refreshToken.value =
                                        null
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        } else {
            activity?.runOnUiThread {
                Toast.makeText(
                    requireContext(),
                    "Не удалось получить табы",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }
    }
}

} catch (e: Exception) {
    activity?.runOnUiThread {
        Toast.makeText(
            requireContext(),
            "Не удалось установить соединение с сервером",
            Toast.LENGTH_SHORT
        ).show()
    }
}
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = MidiFragment()
}

```

MyPostFragment.kt

```
package com.example.guitarlessons.fragments
```

```

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import android.widget.Toast
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.adapter.ArticleAdapter
import com.example.guitarlessons.databinding.FragmentMyPostBinding
import com.example.guitarlessons.retrofit.article.Article
import com.example.guitarlessons.retrofit.article.ArticleApi
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager

```

```

import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.viewmodel.ArticleViewModel
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class MyPostFragment : Fragment(), ArticleAdapter.ClickListener {
    private lateinit var binding: FragmentMyPostBinding
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private val articleViewModel: ArticleViewModel by activityViewModels()
    private var accessToken: String? = null
    private var refreshToken: String? = null
    private lateinit var articleAdapter: ArticleAdapter
    private var _retrofit: Retrofit? = null
    private var me: User? = null
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentMyPostBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        articleAdapter = ArticleAdapter(this)
        binding.postrcView.layoutManager = LinearLayoutManager(context)
        binding.postrcView.adapter = articleAdapter
        articleAdapter.changeVisibility(true)
        if ((activity as MainActivity).isConnected()) {
            val interceptor = HttpLoggingInterceptor()
            interceptor.level = HttpLoggingInterceptor.Level.BODY

            val client = OkHttpClient.Builder()
                .addInterceptor(interceptor)
                .connectTimeout(50, TimeUnit.MILLISECONDS)
                .build()

            val retrofit = Retrofit
                .Builder()
                .baseUrl(BuildConfig.API_URL)
                .client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build()
            _retrofit = retrofit
            CoroutineScope(Dispatchers.IO).launch {
                val user = getMe(retrofit)
                if (user != null) {
                    me = user
                }
            }
        }
    }
}

```

```

        }
    }
    getMyPosts()
}
binding.backSettingsBtn.setOnClickListener {
    replaceFragmentOnView(SettingsFragment.newInstance())
}

binding.messageEmptyBtn.setOnClickListener {
    replaceFragmentOnView(EditorFragment.newInstance())
}

}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

private fun getMyPosts() {
    if (_retrofit == null)
        return
    val articleApi = _retrofit!!.create(ArticleApi::class.java)
    CoroutineScope(Dispatchers.IO).launch {
        val responseArticles =
articleApi.getAllMyPosts("Bearer_${accessToken}")
        if (responseArticles.code() == 200) {
            activity?.runOnUiThread {
                if (responseArticles.body()?.articles?.isEmpty() == true)
                    binding.MessageEmptyHolder.visibility = View.VISIBLE
                else
                    binding.postrcView.visibility = View.VISIBLE
            }
            articleAdapter.submitList(responseArticles.body()?.articles)
        }
    }
}

}

}

companion object {
    @JvmStatic
    fun newInstance() = MyPostFragment()
}

override fun onClick(article: Article) {
    articleViewModel.id.value = article.id
    replaceFragmentOnView(ArticleFragment.newInstance())
}

override fun onLikeClick(article: Article, likeCount: TextView) {
    val articleApi = _retrofit!!.create(ArticleApi::class.java)
    CoroutineScope(Dispatchers.IO).launch {

```

```

        if (me != null) {
            val response = articleApi.like(me!!.id!!, article.id!!,
"Bearer_${accessToken}")
            if (response.code() == 200) {
                activity?.runOnUiThread {
                    likeCount.text = response.body()!!.count.toString()
                }
            }
        }
    }
    else{
        activity?.runOnUiThread {
            Toast.makeText(
                context,
                "Для этого необходимо авторизоваться",
                Toast.LENGTH_SHORT
            ).show()
        }
    }
}

}

}

override fun onAdminDelClick(article: Article) {
    val builder = AlertDialog.Builder(context)
    builder.setMessage("Вы уверены что хотите удалить урок?")
        .setCancelable(false)
        .setPositiveButton("Удалить") { dialog, id ->

            CoroutineScope(Dispatchers.IO).launch {
                delPost(article)
            }
        }
        .setNegativeButton("Отмена") { dialog, id ->
            dialog.dismiss()
        }
    val alert = builder.create()
    alert.show()
}

private suspend fun getMe(retrofit: Retrofit): User? {
    try {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        return withContext(Dispatchers.IO) {
            val responseUser: Response<User> =
authApi.getMe("Bearer_${accessToken}")
            if (responseUser.isSuccessful) {
                responseUser.body()
            } else if (responseUser.code() == 500) {
                if (refreshToken != null) {
                    val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                        RefreshTokenRequest(
                            refreshToken!!
                        )
                    )
                    if (responseToken.code() == 200) {
                        accessToken =
responseToken.body()?.accessToken.toString()
                        TokenManager.saveTokens(
                            requireContext(), accessToken!!,
                            refreshToken!!
                        )
                    }
                }
            }
        }
    }
}

```

```

        activity?.runOnUiThread {
            tokensViewModel.accessToken.value =
accessToken
            tokensViewModel.refreshToken.value =
refreshToken
        }
        getMe(retrofit)
    } else {
        TokenManager.clearTokens(requireContext())
        activity?.runOnUiThread {
            tokensViewModel.accessToken.value = null
            tokensViewModel.refreshToken.value = null
        }
        null
    }
} else
    null
} else
    null

    }
} catch (e: Exception) {
    return null
}
}

private suspend fun delPost(article: Article) {
    if (_retrofit == null)
        return
    val authApi: AuthApi = _retrofit!!.create(AuthApi::class.java)
    val articleApi: ArticleApi =
_retrofit!!.create(ArticleApi::class.java)
    val responseCode: Int =
        articleApi.deleteArticleById(article.id!!,
"Bearer_${accessToken}").code()
    if (responseCode == 200) {
        getMyPosts()
    } else if (responseCode == 500) {
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
            RefreshTokenRequest(
                refreshToken!!
            )
        )
        if (responseToken.code() == 200) {
            accessToken =
responseToken.body()?.accessToken.toString()
            TokenManager.saveTokens(
                requireContext(), accessToken!!,
                refreshToken!!
            )
            activity?.runOnUiThread {
                tokensViewModel.accessToken.value = accessToken
                tokensViewModel.refreshToken.value = refreshToken
            }
            delPost(article)
        } else {
            TokenManager.clearTokens(requireContext())
            activity?.runOnUiThread {
                tokensViewModel.accessToken.value = null
                tokensViewModel.refreshToken.value = null
            }
        }
    }
}

```

```

    }
}

}

}

```

MyTabsFragment.kt

```

package com.example.guitarlessons.fragments

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.core.view.isVisible
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.adapter.TabAdapter
import com.example.guitarlessons.databinding.FragmentMyTabsBinding
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.tab.RequestTabDto
import com.example.guitarlessons.retrofit.tab.TabApi
import com.example.guitarlessons.retrofit.tab.TabDto
import com.example.guitarlessons.retrofit.tab.TabsDto
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.utils.PdfCreator
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class MyTabsFragment : Fragment(), TabAdapter.ClickListener {
    private lateinit var binding: FragmentMyTabsBinding
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private var accessToken: String? = null
    private var refreshToken: String? = null
    private lateinit var tabAdapter: TabAdapter
    private var _retrofit: Retrofit? = null
    private var me: User? = null
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentMyTabsBinding.inflate(inflater, container, false)
    }
}

```

```

        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        tabAdapter = TabAdapter(this)
        binding.tabrcView.layoutManager = LinearLayoutManager(context)
        binding.tabrcView.adapter = tabAdapter
        tabAdapter.changeVisibility(true)
        tabAdapter.changeForAllVisibility(false)
        if ((activity as MainActivity).isConnected()) {
            val interceptor = HttpLoggingInterceptor()
            interceptor.level = HttpLoggingInterceptor.Level.BODY

            val client = OkHttpClient.Builder()
                .addInterceptor(interceptor)
                .connectTimeout(50, TimeUnit.MILLISECONDS)
                .build()

            val retrofit = Retrofit
                .Builder()
                .baseUrl(BuildConfig.API_URL).client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build()
            _retrofit = retrofit
            CoroutineScope(Dispatchers.IO).launch {
                val user = getMe(retrofit)
                if (user != null) {
                    me = user
                }
            }
            getMyTabs()
        }
        binding.backSettingsBtn.setOnClickListener {
            replaceFragmentOnView(SettingsFragment.newInstance())
        }

        binding.messageEmptyBtn.setOnClickListener {
            replaceFragmentOnView(MidiFragment.newInstance())
        }
    }
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

private fun getMyTabs() {
    if (_retrofit == null)
        return
    val tabApi = _retrofit!!.create(TabApi::class.java)
    CoroutineScope(Dispatchers.IO).launch {

```

```

        val response: Response<TabsDto> =
tabApi.getMy("Bearer_${accessToken}")
        if (response.code() == 200) {
            activity?.runOnUiThread {
                if (response.body()?.tabs?.isEmpty() == true)
                    binding.MessageEmptyHolder.visibility = View.VISIBLE
                else
                    binding.tabrcView.visibility = View.VISIBLE
                tabAdapter.submitList(response.body()?.tabs)
            }
        }
    }

}

companion object {
    @JvmStatic
    fun newInstance() = MyTabsFragment()
}

private suspend fun getMe(retrofit: Retrofit): User? {
    try {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        return withContext(Dispatchers.IO) {
            val responseUser: Response<User> =
authApi.getMe("Bearer_${accessToken}")
            if (responseUser.isSuccessful) {
                responseUser.body()
            } else if (responseUser.code() == 500) {
                if (refreshToken != null) {
                    val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                        RefreshTokenRequest(
                            refreshToken!!
                        )
                    )
                    if (responseToken.code() == 200) {
                        accessToken =
responseToken.body()?.accessToken.toString()
                        TokenManager.saveTokens(
                            requireContext(), accessToken!!,
                            refreshToken!!
                        )
                        activity?.runOnUiThread {
                            tokensViewModel.accessToken.value =
accessToken
                            tokensViewModel.refreshToken.value =
refreshToken
                        }
                    }
                    getMe(retrofit)
                } else {
                    TokenManager.clearTokens(requireContext())
                    activity?.runOnUiThread {
                        tokensViewModel.accessToken.value = null
                        tokensViewModel.refreshToken.value = null
                    }
                    null
                }
            } else
                null
        }
    }
}

```

```

        } else
            null

    }
} catch (e: Exception) {
    return null
}
}

private suspend fun delTab(tab: TabDto) {
    if (_retrofit == null)
        return
    val authApi: AuthApi = _retrofit!!.create(AuthApi::class.java)
    val tabApi: TabApi = _retrofit!!.create(TabApi::class.java)
    val responseCode: Int =
        tabApi.deleteTabById(tab.id!!, "Bearer_${accessToken}").code()
    if (responseCode == 200) {
        getMyTabs()
    } else if (responseCode == 500) {
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                RefreshTokenRequest(
                    refreshToken!!
                )
            )
            if (responseToken.code() == 200) {
                accessToken =
responseToken.body()?.accessToken.toString()
                TokenManager.saveTokens(
                    requireContext(), accessToken!!,
                    refreshToken!!
                )
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = accessToken
                    tokensViewModel.refreshToken.value = refreshToken
                }
                delTab(tab)
            } else {
                TokenManager.clearTokens(requireContext())
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = null
                    tokensViewModel.refreshToken.value = null
                }
            }
        }
    }
}

}

}

override fun onClick(view: TextView) {
    if (view.isVisible)
        view.visibility = View.GONE
    else
        view.visibility = View.VISIBLE
}

override fun onDeleteBtnClick(tab: TabDto) {
    val builder = AlertDialog.Builder(context)
    builder.setMessage("Вы уверены что хотите удалить урок?")
        .setCancelable(false)
        .setPositiveButton("Удалить") { dialog, id ->

```

```

        CoroutineScope(Dispatchers.IO).launch {
            delTab(tab)
        }
    }
    .setNegativeButton("Отмена") { dialog, id ->
        dialog.dismiss()
    }
    val alert = builder.create()
    alert.show()
}

override fun onPdfSaveBtnClick(tab: TabDto) {
    PdfCreator.createPdf(tab.title, tab.content, requireContext())
}

override fun onSharedBtnClick(tab: TabDto, view: TextView) {
    if (tab.isShared == true){

view.setTextColor(requireContext().resources.getColor(R.color.my_orange))
        view.text = "Опубликовать?"
        tab.isShared = false
        CoroutineScope(Dispatchers.IO).launch {

updateTab(RequestTabDto(tab.id!!, tab.title, tab.content, tab.created, false, tab.
author))
            }
        }
        else if (tab.isShared == false){

view.setTextColor(requireContext().resources.getColor(R.color.my_green))
        view.text = "Опубликовано"
        tab.isShared = true
        CoroutineScope(Dispatchers.IO).launch {

updateTab(RequestTabDto(tab.id!!, tab.title, tab.content, tab.created, true, tab.a
uthor))
            }
        }

    }

private suspend fun updateTab(tab: RequestTabDto){
    if (_retrofit == null)
        return
    val tabApi = _retrofit!!.create(TabApi::class.java)
    tabApi.update(tab, "Bearer_ ${accessToken}")
}
}

```

RegisterFragment.kt

```
package com.example.guitarlessons.fragments
```

```

import android.annotation.SuppressLint
import android.os.Bundle
import android.text.TextUtils
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Toast
import androidx.core.widget.doAfterTextChanged
import androidx.fragment.app.Fragment
import com.example.guitarlessons.BuildConfig

```

```

import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentRegisterBinding
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.auth.RegRequest
import com.example.guitarlessons.retrofit.auth.ResponseMessage
import com.google.gson.Gson
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

class RegisterFragment : Fragment() {

    private lateinit var binding: FragmentRegisterBinding

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentRegisterBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val interceptor = HttpLoggingInterceptor()
        interceptor.level = HttpLoggingInterceptor.Level.BODY

        val client = OkHttpClient.Builder()
            .addInterceptor(interceptor)
            .build()

        val retrofit = Retrofit
            .Builder()
            .baseUrl(BuildConfig.API_URL).client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        binding.apply {
            regButton.setOnClickListener {

                if ((activity as MainActivity).isConnected()) {
                    registration(retrofit)
                } else {
                    Toast.makeText(
                        activity, "Требуется интернет соединение",
                        Toast.LENGTH_LONG
                    ).show()
                }
            }

            usernameEditText.doAfterTextChanged {
                if (!isValidUsername())
                    usernameErrTextView.visibility = View.GONE
            }

            emailEditText.doAfterTextChanged {

```

```

        if (isEmailValid())
            emailErrTextView.visibility = View.GONE
    }

    passwordEditText.doAfterTextChanged {
        if (!isValidPassword())
            passwordErrTextView.visibility = View.GONE
        if (isMatchPasswords())
            repasswordErrTextView.visibility = View.GONE
    }

    repeatPasswordEditText.doAfterTextChanged {
        if (isMatchPasswords())
            repasswordErrTextView.visibility = View.GONE
    }

    backBtn.setOnClickListener {
        replaceFragmentOnView(LoginFragment.newInstance())
    }
}

private fun registration(retrofit: Retrofit) {
    if (isValid())
        return
    if ((activity as MainActivity).isConnected()) {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        CoroutineScope(Dispatchers.IO).launch {
            try {
                val response: Response<ResponseMessage> =
                    authApi.register(
                        RegRequest(
                            binding.usernameEditText.text.toString(),
                            binding.passwordEditText.text.toString(),
                            binding.emailEditText.text.toString(),
                        )
                    )

                if (response.code() == 201) {
                    activity?.runOnUiThread {
                        Toast.makeText(
                            context,
                            "Пользователь успешно зарегистрирован!",
                            Toast.LENGTH_LONG
                        ).show()

                        replaceFragmentOnView(LoginFragment.newInstance())
                    }
                } else {
                    val error: ResponseMessage = Gson().fromJson(
                        response.errorBody()?.string(),
                        ResponseMessage::class.java
                    )
                    Log.e("body", error.message)
                    if (response.code() == 409 && error.message == "user
exist") {

                        activity?.runOnUiThread {
                            binding.usernameErrTextView.visibility =
View.VISIBLE

```

```

        Toast.makeText(
            context,
            "Пользователь с таким именем уже
существует",
            Toast.LENGTH_LONG
        ).show()
    }
}
if (response.code() == 409 && error.message == "email
exist") {
    activity?.runOnUiThread {
        binding.emailErrTextView.visibility =
View.VISIBLE
        Toast.makeText(
            context,
            "Пользователь с такой почтой уже
существует",
            Toast.LENGTH_LONG
        ).show()
    }
}
}
} catch (e: Exception) {
    activity?.runOnUiThread {
        Toast.makeText(
            activity,
            "Ошибка подключения к серверу",
            Toast.LENGTH_LONG
        ).show()
    }
    e.printStackTrace()
}
}
} else
    Toast.makeText(
        activity, "Требуется интернет соединение",
        Toast.LENGTH_LONG
    ).show()
}

private fun isValid(): Boolean {
    var isValid = false
    binding.apply {
        if (isValidUsername()) {
            usernameErrTextView.visibility = View.VISIBLE
            isValid = true
        }
        if (isValidPassword()) {
            passwordErrTextView.visibility = View.VISIBLE
            isValid = true
        }
        if (!isValidEmail()) {
            emailErrTextView.visibility = View.VISIBLE
            isValid = true
        }
        if (!isValidMatchPasswords()) {
            repasswordErrTextView.visibility = View.VISIBLE
            isValid = true
        }
    }
}

```

```

        }

        }
        return isValid
    }

    private fun isValidUsername(): Boolean {
        return binding.usernameEditText.text.isEmpty() ||
binding.usernameEditText.text.length > 50 ||
binding.usernameEditText.text.length < 4
    }

    private fun isValidPassword(): Boolean {
        return binding.passwordEditText.text.isEmpty() ||
binding.passwordEditText.text.length > 255 ||
binding.passwordEditText.text.length < 8
    }

    private fun isValidEmail(): Boolean {
        return !TextUtils.isEmpty(binding.emailEditText.text) &&
android.util.Patterns.EMAIL_ADDRESS.matcher(
binding.emailEditText.text
).matches()
    }

    private fun isMatchPasswords(): Boolean {
        return binding.passwordEditText.text.toString() ==
binding.repeatPasswordEditText.text.toString()
    }

    @SuppressWarnings("CommitTransaction")
    fun replaceFragmentOnView(f: Fragment) {
        fragmentManager
            .beginTransaction()
            .replace(R.id.fragmentContainerView, f)
            .commit()
    }

    companion object {
        @JvmStatic
        fun newInstance() = RegisterFragment()
    }
}

```

SettingsFragment.kt

```
package com.example.guitarlessons.fragments
```

```

import android.annotation.SuppressLint
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.lifecycle.LifecycleScope
import com.bumptech.glide.Glide
import com.bumptech.glide.request.RequestOptions
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.databinding.FragmentSettingsBinding
import com.example.guitarlessons.retrofit.auth.AuthApi

```

```

import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.viewmodel.TokensViewModel
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class SettingsFragment : Fragment() {

    private val tokensViewModel: TokensViewModel by activityViewModels()

    private var accessToken: String? = null

    private var refreshToken: String? = null

    private lateinit var binding: FragmentSettingsBinding

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentSettingsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        binding.apply {
            logAndRegBtn.setOnClickListener {
                replaceFragmentOnView(LoginFragment.newInstance())
            }

            accountManagerBtn.setOnClickListener {
                replaceFragmentOnView(AccountManagerFragment.newInstance())
            }
            draftHolderBtn.setOnClickListener {
                replaceFragmentOnView(DraftsFragment.newInstance())
            }
            postHolderBtn.setOnClickListener {
                replaceFragmentOnView(MyPostFragment.newInstance())
            }
            myTabsHolderBtn.setOnClickListener {
                replaceFragmentOnView(MyTabsFragment.newInstance())
            }
        }
    }

    try {

```

```

        if ((activity as MainActivity).isConnected()) {

            val interceptor = HttpLoggingInterceptor()
            interceptor.level = HttpLoggingInterceptor.Level.BODY

            val client = OkHttpClient.Builder()
                .addInterceptor(interceptor)
                .connectTimeout(50, TimeUnit.MILLISECONDS)
                .build()

            val retrofit = Retrofit
                .Builder()
                .baseUrl(BuildConfig.API_URL).client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build()

            if (accessToken != null) {
                lifecycleScope.launch {
                    val user: User? = getMe(retrofit)
                    activity?.runOnUiThread {
                        if (user != null) {
                            authorized(user)
                        }
                        else {
                            unauthorized()
                        }
                    }
                }
            }
            else {
                unauthorized()
            }
        }
        else {
            unauthorized()
        }
    }
}

catch (e:Exception){
    unauthorized()
}

}

private fun authorized(user: User){
    binding.apply {
        avatarHolder.visibility = View.VISIBLE
        personalAccountTV.visibility = View.VISIBLE
        regDateView.text = "C ${user.registrationDate!!.substring(0,
user.registrationDate!!.indexOf(" "))}"
        usernameTV.text = user.username
        if (user.avatarPicture != null) {

            context?.let {
                Glide.with(it)
                    .load(user.avatarPicture)
                    .apply(
                        RequestOptions().override(
                            avatarHolder.width,
                            avatarHolder.height
                        )
                    )
                    .centerCrop()
                    .into(avatarIV)
            }
        }
        regDateView.visibility = View.VISIBLE
    }
}

```

```

        accountManagerBtn.visibility = View.VISIBLE
        draftHolderBtn.visibility = View.VISIBLE
        postHolderBtn.visibility = View.VISIBLE
        myTabsHolderBtn.visibility = View.VISIBLE

        logAndRegBtn.visibility = View.GONE
        settingsLayout.visibility = View.VISIBLE
    }
}

private fun unauthorized(){
    binding.apply {
        avatarHolder.visibility = View.GONE
        personalAccountTV.visibility = View.GONE
        regDateView.visibility = View.GONE
        draftHolderBtn.visibility = View.GONE
        usernameTV.text = "Личный кабинет"
        accountManagerBtn.visibility = View.GONE
        postHolderBtn.visibility = View.GONE
        myTabsHolderBtn.visibility = View.GONE

        logAndRegBtn.visibility = View.VISIBLE

        settingsLayout.visibility = View.VISIBLE
    }
}

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f: Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView, f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = SettingsFragment()
}

private suspend fun getMe(retrofit: Retrofit): User? {
    try {
        val authApi: AuthApi = retrofit.create(AuthApi::class.java)
        return withContext(Dispatchers.IO) {
            val responseUser: Response<User> =
authApi.getMe("Bearer_${accessToken}")
            if (responseUser.isSuccessful) { // Проверка статуса ответа
                responseUser.body()
            } else if (responseUser.code() == 500) {
                if (refreshToken != null) {
                    val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                        RefreshTokenRequest(
                            refreshToken!!
                        )
                    )
                    if (responseToken.code() == 200) {
                        accessToken =
responseToken.body()?.accessToken.toString()
                        TokenManager.saveTokens(
                            requireContext(), accessToken!!,
                            refreshToken!!
                        )
                    }
                }
            }
        }
    }
}

```

```

        )
        activity?.runOnUiThread {
            tokensViewModel.accessToken.value =
accessToken
            tokensViewModel.refreshToken.value =
refreshToken
        }
        getMe(retrofit)
    } else {
        TokenManager.clearTokens(requireContext())
        activity?.runOnUiThread {
            tokensViewModel.accessToken.value = null
            tokensViewModel.refreshToken.value = null
        }
        null
    }
} else
    null
} else
    null

}

}
catch (e:Exception){
    unauthorized()
    return null
}
}
}

```

TabsFragment.kt

```

package com.example.guitarlessons.fragments

```

```

import android.annotation.SuppressLint
import android.app.AlertDialog
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import android.widget.Toast
import androidx.core.view.isVisible
import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.LifecycleOwner
import androidx.lifecycle.lifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.guitarlessons.BuildConfig
import com.example.guitarlessons.MainActivity
import com.example.guitarlessons.R
import com.example.guitarlessons.adapter.TabAdapter
import com.example.guitarlessons.databinding.FragmentTabsBinding
import com.example.guitarlessons.retrofit.admin.AdminApi
import com.example.guitarlessons.retrofit.auth.AuthApi
import com.example.guitarlessons.retrofit.tab.TabApi
import com.example.guitarlessons.retrofit.tab.TabDto
import com.example.guitarlessons.retrofit.token.RefreshTokenRequest
import com.example.guitarlessons.retrofit.token.TokenManager
import com.example.guitarlessons.retrofit.token.TokenResponse
import com.example.guitarlessons.retrofit.user.User
import com.example.guitarlessons.utils.PdfCreator
import com.example.guitarlessons.utils.addOnScrollListenerToEnd
import com.example.guitarlessons.utils.removeOnScrollListenerToEnd
import com.example.guitarlessons.viewmodel.TokensViewModel

```

```

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Response
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

class TabsFragment : Fragment(), TabAdapter.ClickListener {
    private lateinit var tabAdapter: TabAdapter
    private lateinit var binding: FragmentTabsBinding
    private val tokensViewModel: TokensViewModel by activityViewModels()
    private var accessToken: String? = null
    private var refreshToken: String? = null
    private var me: User? = null
    private var isAuth: Boolean = false
    private var isAdmin: Boolean = false
    private var offset: Int = 0
    private val startLimit: Int = 10
    private var limit: Int = startLimit

    private var retrofit: Retrofit? = null
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        binding = FragmentTabsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        tokensViewModel.accessToken.observe(activity as LifecycleOwner, {
            accessToken = it
        })
        tokensViewModel.refreshToken.observe(activity as LifecycleOwner, {
            refreshToken = it
        })
        tabAdapter = TabAdapter(this)
        binding.rcView.layoutManager = LinearLayoutManager(context)
        binding.rcView.adapter = tabAdapter
        tabAdapter.changeForAllVisibility(true)
        binding.reloadBtn.setOnClickListener {
            replaceFragmentOnView(newInstance())
        }
        if((activity as MainActivity).isConnected()){
            try {
                //Если есть интернет соединение
                authorized()

                val interceptor = HttpLoggingInterceptor()
                interceptor.level = HttpLoggingInterceptor.Level.BODY

                val client = OkHttpClient.Builder()
                    .addInterceptor(interceptor)
                    .connectTimeout(50, TimeUnit.MILLISECONDS)
                    .build()

                val rf = Retrofit
                    .Builder()

```

```

        .baseUrl (BuildConfig.API_URL) .client (client)
        .addConverterFactory (GsonConverterFactory.create ())
        .build ()
    retrofit = rf
    lifecycleScope.launch {
        val user = getMe (retrofit!!)
        if (user != null) {

            me = user
            isAuth = true
            if (user.role!!.roleName == "ADMIN") {
                isAdmin = true
            }
        }
        tabAdapter.changeVisibility (isAdmin)
    }

    binding.rcView.addOnScrollListenerToEnd (binding.loadMoreBtn)

    binding.loadMoreBtn.setOnClickListener {
        lifecycleScope.launch {
            getAllSharedTabs (retrofit!!)
            binding.loadMoreBtn.visibility = View.GONE
            binding.rcView.smoothScrollBy (0, 100)
        }
    }
    lifecycleScope.launch {
        getAllSharedTabs (rf)
    }
} catch (e:Exception) {
    unauthorized ()
}
}
else {
    //Если нет интернет соединения
    unauthorized ()
}
}

private fun authorized () {
    binding.apply {
        rcView.visibility = View.VISIBLE
        ConnectionLostMessage.visibility = View.GONE
    }
}

private fun unauthorized () {
    binding.apply {
        rcView.visibility = View.GONE
        ConnectionLostMessage.visibility = View.VISIBLE
    }
}

private suspend fun getMe (retrofit: Retrofit): User? {
    try {
        val authApi: AuthApi = retrofit.create (AuthApi::class.java)
        return withContext (Dispatchers.IO) {
            val responseUser: Response<User> =
authApi.getMe ("Bearer_ ${accessToken}")
            if (responseUser.isSuccessful) {

```

```

        // Проверка статуса ответа
        responseUser.body()
    } else if (responseUser.code() == 500) {
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
                RefreshTokenRequest(
                    refreshToken!!
                )
            )
            if (responseToken.code() == 200) {
                accessToken =
responseToken.body()?.accessToken.toString()
                TokenManager.saveTokens(
                    requireContext(), accessToken!!,
                    refreshToken!!
                )
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value =
accessToken
                    tokensViewModel.refreshToken.value =
refreshToken
                }
                getMe(retrofit)
            } else {
                TokenManager.clearTokens(requireContext())
                activity?.runOnUiThread {
                    tokensViewModel.accessToken.value = null
                    tokensViewModel.refreshToken.value = null
                }
                null
            }
        } else {
            null
        }
    } else {
        null
    }
}

} catch (e: Exception) {
    return null
}
}

companion object {
    @JvmStatic
    fun newInstance() = TabsFragment()
}

override fun onClick(view: TextView) {
    if (view.isVisible)
        view.visibility = View.GONE
    else
        view.visibility = View.VISIBLE
}

override fun onDeleteBtnClick(tab: TabDto) {
    val builder = AlertDialog.Builder(context)
    builder.setMessage("Вы уверены что хотите удалить урок?")
        .setCancelable(false)
        .setPositiveButton("Удалить") { dialog, id ->

        CoroutineScope(Dispatchers.IO).launch {

```

```

        delTab(tab)
    }
}
.setNegativeButton("Отмена") { dialog, id ->
    dialog.dismiss()
}
val alert = builder.create()
alert.show()
}

private suspend fun getAllSharedTabs(retrofit: Retrofit){
    try {
        val tabApi = retrofit.create(TabApi::class.java)
        val response = tabApi.getAllShared(offset, limit)
        if (response.code() == 200){
            activity?.runOnUiThread {
                tabAdapter.submitList(response.body()!!.content)
                //articleAdapter.notifyItemRangeChanged(0,
articleAdapter.itemCount);
                limit+=startLimit
            }
        }
    } catch (e:Exception){
        activity?.runOnUiThread {
            Toast.makeText(context, "Ошибка подключения к серверу",
Toast.LENGTH_SHORT).show()
            unauthorized()
        }
    }
}

private suspend fun delTab(tab: TabDto) {
    if (retrofit == null)
        return
    val authApi: AuthApi = retrofit!!.create(AuthApi::class.java)
    val adminApi: AdminApi = retrofit!!.create(AdminApi::class.java)
    val responseCode: Int =
        adminApi.deleteTabById(tab.id!!, "Bearer_${accessToken}").code()
    if (responseCode == 200) {
        getAllSharedTabs(retrofit!!)
    } else if (responseCode == 500) {
        if (refreshToken != null) {
            val responseToken: Response<TokenResponse> =
authApi.refreshAccessToken(
            RefreshTokenRequest(
                refreshToken!!
            )
        )
        if (responseToken.code() == 200) {
            accessToken =
responseToken.body()?.accessToken.toString()
            TokenManager.saveTokens(
                requireContext(), accessToken!!,
                refreshToken!!
            )
            activity?.runOnUiThread {
                tokensViewModel.accessToken.value = accessToken
                tokensViewModel.refreshToken.value = refreshToken
            }
            delTab(tab)
        } else {
            TokenManager.clearTokens(requireContext())

```



```

@SuppressLint("CommitTransaction")
fun replaceFragmentOnView(f:Fragment) {
    parentFragmentManager
        .beginTransaction()
        .replace(R.id.fragmentContainerView,f)
        .commit()
}

companion object {
    @JvmStatic
    fun newInstance() = ToolsChoiceFragment()
}
}

```

Исходный код серверной части

SecurityConfig.kt

```
package com.diplom.guitartoolsserver.config

import com.diplom.guitartoolsserver.security.jwt.JwtConfigurer
import com.diplom.guitartoolsserver.security.jwt.JwtTokenProvider
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.context.annotation.Bean
import org.springframework.context.annotation.Configuration
import org.springframework.security.authentication.AuthenticationManager
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration
import org.springframework.security.config.annotation.web.builders.HttpSecurity
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity
import org.springframework.security.config.annotation.web.invoke
import org.springframework.security.config.http.SessionCreationPolicy
import org.springframework.security.web.SecurityFilterChain

@Configuration
@EnableWebSecurity
class SecurityConfig @Autowired constructor(val jwtTokenProvider:
JwtTokenProvider) {
    @Bean
    fun authenticationManager(config: AuthenticationConfiguration):
AuthenticationManager {
        return config.authenticationManager
    }

    @Bean
    fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
        http {
            csrf { disable() }
            sessionManagement { sessionCreationPolicy =
SessionCreationPolicy.STATELESS }
            authorizeHttpRequests {
                authorize("/api/v1/auth", permitAll)
                authorize("/api/v1/auth/refresh", permitAll)
                authorize("/api/v1/auth/login", permitAll)
                authorize("/api/v1/auth/register", permitAll)
                authorize("/api/v1/uploadImage/**", permitAll)
                authorize("/api/v1/uploadImage", permitAll)
                authorize("/api/v1/articles/id/**", permitAll)
                authorize("/api/v1/articles/main", permitAll)
                authorize("/api/v1/articles/community", permitAll)
                authorize("/api/v1/articles/search**", permitAll)
                authorize("/api/v1/tabs/**", permitAll)
                authorize("/api/v1/comments/**", permitAll)
                authorize("/swagger-ui/**", permitAll)
                authorize("/swagger-ui.html", permitAll)
                authorize("/v3/api-docs", permitAll)
                authorize("/v3/api-docs/**", permitAll)
            }
        }
    }
}
```

```

        //authorize( "/api/v1/user/**", hasRole("USER"))
        //authorize("/api/v1/admin/**", hasRole("ADMIN"))
        authorize(anyRequest, authenticated)

    }
    apply( JwtConfigurer(jwtTokenProvider))
}

return http.build()
}
}

```

ArticleRestController.kt

```

package com.diplom.guitartoolsserver.controller

import com.diplom.guitartoolsserver.dto.Article.ArticleDto
import com.diplom.guitartoolsserver.dto.Article.ArticlePartDto
import com.diplom.guitartoolsserver.dto.Article.ArticlesDto
import com.diplom.guitartoolsserver.dto.like.Likes
import com.diplom.guitartoolsserver.dto.mapper.ArticleMapper
import com.diplom.guitartoolsserver.model.Article
import com.diplom.guitartoolsserver.service.ArticleService
import jakarta.validation.constraints.Max
import jakarta.validation.constraints.Min
import mu.KotlinLogging
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.data.domain.Page
import org.springframework.data.domain.PageRequest
import org.springframework.data.domain.Sort
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.security.access.prepost.PreAuthorize
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity
import org.springframework.web.bind.annotation.*
import java.security.Principal

private val logger = KotlinLogging.logger {}

@RestController
@RequestMapping("/api/v1/articles")
@EnableMethodSecurity
class ArticleRestController @Autowired constructor(
    val articleService: ArticleService,
) {

    @PreAuthorize("hasAnyRole('ROLE_ADMIN')")
    @GetMapping
    fun getAll(): ResponseEntity<List<ArticleDto>> {
        val articles: List<Article> = articleService.getAll()
        val articlesDto = articles.map { ArticleMapper.toDto(it) }
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @GetMapping("/{id}/{id}")
    fun getById(@PathVariable id: Long): ResponseEntity<ArticleDto> {
        val article = articleService.findById(id) ?: return
        ResponseEntity(HttpStatus.NOT_FOUND)
        if (article.status.id == 1.toLong())
            return ResponseEntity(HttpStatus.FORBIDDEN)
    }
}

```

```

        val articleDto = ArticleMapper.toDto(article)
        return ResponseEntity(articleDto, HttpStatus.OK)
    }

    @PostMapping("/like/{userId}/{articleId}")
    fun likeArticle(@PathVariable userId: Long, @PathVariable articleId:
Long): ResponseEntity<Likes> {
        val likes: Likes = articleService.likeArticle(userId, articleId) ?:
return ResponseEntity(HttpStatus.NOT_FOUND)
        return ResponseEntity(likes, HttpStatus.OK)
    }

    @GetMapping("/draft/id/{id}")
    fun getDraftById(@PathVariable id: Long, principal: Principal):
ResponseEntity<ArticleDto> {
        val article = articleService.findById(id) ?: return
ResponseEntity(HttpStatus.NOT_FOUND)
        if (article.status.id != 1.toLong())
            return ResponseEntity(HttpStatus.NOT_FOUND)
        if (article.author.username != principal.name)
            return ResponseEntity(HttpStatus.FORBIDDEN)
        val articleDto = ArticleMapper.toDto(article)
        return ResponseEntity(articleDto, HttpStatus.OK)
    }

    @GetMapping("/status/published")
    fun getAllPublished(): ResponseEntity<ArticlesDto> {
        val articles: MutableList<Article> =
articleService.findByStatusName("published")
        ?: return ResponseEntity(HttpStatus.NOT_FOUND)
        val articlesDto = ArticlesDto(articles.map { ArticleMapper.toDto(it)
}).toMutableList()
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @PreAuthorize("hasAnyRole('ROLE_ADMIN')")
    @GetMapping("/status/{name}")
    fun getByStatusName(@PathVariable name: String):
ResponseEntity<ArticlesDto> {
        val articles: MutableList<Article> =
articleService.findByStatusName(name)
        ?: return ResponseEntity(HttpStatus.NOT_FOUND)
        val articlesDto = ArticlesDto(articles.map { ArticleMapper.toDto(it)
}).toMutableList()
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @GetMapping("/title/{title}")
    fun getTitle(@PathVariable title: String): ResponseEntity<ArticlesDto>
{
        val articles = articleService.findByTitle(title) ?: return
ResponseEntity(HttpStatus.NOT_FOUND)
        val articlesDto = ArticlesDto(articles.map { ArticleMapper.toDto(it)
}).toMutableList()
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @PostMapping("/addOrUpdate")
    fun addOrUpdate(@RequestBody article: ArticlePartDto, principal:
Principal): ResponseEntity<ArticleDto> {
        val existArticle: Article? = article.id?.let {
articleService.findById(it) }

```

```

        if (existArticle != null) {
            return if (principal.name == existArticle.author.username) {
                ResponseEntity(articleService.update(article)?.let {
                    ArticleMapper.toDto(it) }, HttpStatus.OK)
            } else
                ResponseEntity(HttpStatus.CONFLICT)
        }
        val responseArticle = articleService.create(article, principal.name)
        ?: return ResponseEntity(HttpStatus.NOT_FOUND)
        val articleDto = ArticleMapper.toDto(responseArticle)
        return ResponseEntity(articleDto, HttpStatus.OK)
    }

    @GetMapping("/drafts")
    fun getMyDraft(principal: Principal): ResponseEntity<ArticlesDto> {
        val articles: List<Article> =
            articleService.findAllByAuthorUsernameAndStatusName(principal.name, "draft")
        val articlesDto = ArticlesDto(articles.map { ArticleMapper.toDto(it) }
            ).toMutableList()
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @GetMapping("/me")
    fun getAllMyPosts(principal: Principal): ResponseEntity<ArticlesDto> {
        val articles: List<Article> =
            articleService.findArticleByAuthorUsername(principal.name)
        val articlesDto = ArticlesDto(articles.map { ArticleMapper.toDto(it) }
            ).toMutableList()
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @GetMapping("/community")
    fun getAllCommunityAndSort(
        @RequestParam(value = "offset", defaultValue = "0") @Min(0) offset:
        Int,
        @RequestParam(value = "limit", defaultValue = "20") @Min(1) @Max(100)
        limit: Int,
        @RequestParam("sort") sortField: String,
    ): ResponseEntity<Page<ArticleDto>> {
        val direction: Sort.Direction = when(sortField) {
            "publicationDate" -> Sort.Direction.DESC
            "rating" -> Sort.Direction.DESC
            else -> Sort.Direction.ASC
        }
        val articles =
            articleService.findAllByStatusIdAndAuthorRoleIdNot(2, 2, PageRequest.of(offset,
            limit, Sort.by(direction, sortField)))
        val articlesDto = articles.map(ArticleMapper::toDto)
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @GetMapping("/main")
    fun getAllMainAndSort(
        @RequestParam(value = "offset", defaultValue = "0") @Min(0) offset:
        Int,
        @RequestParam(value = "limit", defaultValue = "20") @Min(1) @Max(100)
        limit: Int,
        @RequestParam("sort") sortField: String,
    ): ResponseEntity<Page<ArticleDto>> {
        val direction: Sort.Direction = when(sortField) {
            "publicationDate" -> Sort.Direction.DESC
            "rating" -> Sort.Direction.DESC
            else -> Sort.Direction.ASC
        }
    }

```

```

        val articles =
        articleService.findAllByStatusIdAndAuthorRoleId(2,2,PageRequest.of(offset,limit, Sort.by(direction,sortField)))
        val articlesDto = articles.map(ArticleMapper::toDto)
        return ResponseEntity(articlesDto, HttpStatus.OK)
    }

    @DeleteMapping("/delete/id/{id}")
    fun deleteArticleById(@PathVariable id: Long,principal: Principal):
    ResponseEntity<Any> {
        val article = articleService.findById(id) ?: return
        ResponseEntity(HttpStatus.NOT_FOUND)
        if (principal.name != article.author.username) {
            return ResponseEntity(HttpStatus.CONFLICT)
        }
        articleService.deleteById(id)
        return ResponseEntity.ok(HttpStatus.OK)
    }

    @GetMapping("/search")
    fun searchPublishedByTitle(@RequestParam(value = "title")title:
    String,@RequestParam(value = "roleId") roleId:Long):
    ResponseEntity<List<ArticleDto>> {
        val article =
        articleService.findAllByTitleContainingIgnoreCaseAndAuthorRoleIdAndStatusId(title,roleId,2)
        val articlesDto = article.map { ArticleMapper.toDto(it) }
        return ResponseEntity(articlesDto,HttpStatus.OK)
    }
}

```

AuthenticationRestController.kt

```
package com.diplom.guitartoolsserver.controller
```

```

import com.diplom.guitartoolsserver.dto.*
import com.diplom.guitartoolsserver.dto.auth.RefreshTokenRequest
import com.diplom.guitartoolsserver.dto.auth.RegisterRequestDto
import com.diplom.guitartoolsserver.dto.auth.TokenResponseDto
import com.diplom.guitartoolsserver.dto.auth.UserDto
import com.diplom.guitartoolsserver.dto.mapper.UserMapper
import com.diplom.guitartoolsserver.security.AuthenticationService
import com.diplom.guitartoolsserver.security.jwt.AuthenticationRequest
import com.diplom.guitartoolsserver.security.jwt.AuthenticationResponse
import com.diplom.guitartoolsserver.security.jwt.JwtTokenProvider
import com.diplom.guitartoolsserver.service.UserService
import jakarta.validation.Valid
import mu.KotlinLogging
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.security.authentication.AuthenticationManager
import org.springframework.web.bind.annotation.*
import java.security.Principal

```

```
private val logger = KotlinLogging.logger {}
```

```

@RestController
@RequestMapping("/api/v1/auth")
class AuthenticationRestController @Autowired constructor(
    val authenticationManager: AuthenticationManager,
    val jwtTokenProvider: JwtTokenProvider,
    val userService: UserService,

```

```

        val authenticationService: AuthenticationService
    ) {

        @PostMapping
        fun authenticate(
            @RequestBody authRequest: AuthenticationRequest
        ): AuthenticationResponse =
            authenticationService.authentication(authRequest)

        @PostMapping("/refresh")
        fun refreshAccessToken(@RequestBody request: RefreshTokenRequest):
        ResponseEntity<TokenResponseDto> {
            return ResponseEntity( TokenResponseDto(
                authenticationService.refreshAccessToken(request.refreshToken)
            ), HttpStatus.OK)
        }

        @PostMapping("/register")
        fun register(@Valid @RequestBody registerRequest:
        RegisterRequestDto): ResponseEntity<ResponseMessage>{
            return try {
                if(userService.existsByUsername(registerRequest.username)) {
                    return
                    ResponseEntity.status(HttpStatus.CONFLICT).body(ResponseMessage("user
                    exist"))
                }
                if(userService.existsByEmail(registerRequest.email)) {
                    return
                    ResponseEntity.status(HttpStatus.CONFLICT).body(ResponseMessage("email
                    exist"))
                }
                userService.register(registerRequest)
                ResponseEntity(ResponseMessage("User
                created"),HttpStatus.CREATED)
            } catch (e:IllegalArgumentException) {
                ResponseEntity(HttpStatus.BAD_REQUEST)
            }
        }

        @GetMapping("/me")
        fun getCurrentUser(principal: Principal): ResponseEntity<UserDto> {
            val user = userService.findByUsername(principal.name) ?: return
            ResponseEntity.status(HttpStatus.NO_CONTENT).build()
            val userDto = UserMapper.toDto(user)
            return ResponseEntity.ok(userDto)
        }
    }
}

```

CommentRestController.kt

```
package com.diplom.guitartoolsserver.controller
```

```

import com.diplom.guitartoolsserver.dto.Article.ArticlesDto
import com.diplom.guitartoolsserver.dto.comment.CommentDto
import com.diplom.guitartoolsserver.dto.comment.CommentRequestDto
import com.diplom.guitartoolsserver.dto.mapper.ArticleMapper
import com.diplom.guitartoolsserver.dto.mapper.CommentMapper
import com.diplom.guitartoolsserver.model.Comments
import com.diplom.guitartoolsserver.service.ArticleService
import com.diplom.guitartoolsserver.service.CommentService
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity

```

```

import org.springframework.web.bind.annotation.*
import java.security.Principal

@RestController
@RequestMapping( "/api/v1/comments")
class CommentRestController @Autowired constructor(
    val commentService: CommentService
){
    @GetMapping("/article/{id}")
    fun getCommentsByArticleId(@PathVariable id: Long) :
    ResponseEntity<List<CommentDto>> {
        val comments = commentService.getCommentsByArticleId(id)
        val commentsDto = comments.map(CommentMapper::toDto)
        return ResponseEntity(commentsDto, HttpStatus.OK)
    }

    @PostMapping("/add")
    fun addComment(@RequestBody comment: CommentRequestDto,principal:
    Principal) : ResponseEntity<CommentDto> {
        val newComment = commentService.create(comment,principal.name)
        val newCommentDto = CommentMapper.toDto(newComment)
        return ResponseEntity(newCommentDto,HttpStatus.OK)
    }

    @DeleteMapping("/delete/{id}")
    fun deleteComment(@PathVariable id: Long,principal: Principal) :
    ResponseEntity<CommentDto> {
        val comment = commentService.findById(id)
        if (principal.name != (comment?.author?.username ?: ""))
            return ResponseEntity(HttpStatus.UNAUTHORIZED)
        commentService.deleteById(id)
        return ResponseEntity(HttpStatus.OK)
    }
}

```

TabRestController.kt

```

package com.diplom.guitartoolsserver.controller

import com.diplom.guitartoolsserver.dto.Article.ArticleDto
import com.diplom.guitartoolsserver.dto.Article.ArticlePartDto
import com.diplom.guitartoolsserver.dto.Article.ArticlesDto
import com.diplom.guitartoolsserver.dto.mapper.ArticleMapper
import com.diplom.guitartoolsserver.dto.note.NoteRequest
import com.diplom.guitartoolsserver.dto.tabs.TabDto
import com.diplom.guitartoolsserver.dto.tabs.TabRequest
import com.diplom.guitartoolsserver.dto.mapper.TabMapper
import com.diplom.guitartoolsserver.dto.tabs.RequestTabDto
import com.diplom.guitartoolsserver.dto.tabs.TabsDto
import com.diplom.guitartoolsserver.model.Tabs
import com.diplom.guitartoolsserver.security.AuthenticationService
import com.diplom.guitartoolsserver.service.TabService
import com.diplom.guitartoolsserver.service.UserService
import com.diplom.guitartoolsserver.utils.MidiGenerator
import com.diplom.guitartoolsserver.utils.PythonUtils
import com.example.guitarlessons.utils.NoteUtils
import jakarta.validation.constraints.Max
import jakarta.validation.constraints.Min
import mu.KotlinLogging
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.beans.factory.annotation.Value
import org.springframework.data.domain.Page
import org.springframework.data.domain.PageRequest
import org.springframework.data.domain.Sort

```

```

import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity
import org.springframework.web.bind.annotation.*
import org.springframework.web.multipart.MultipartFile
import java.nio.file.Files
import java.nio.file.Paths
import java.security.Principal
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter

private val logger = KotlinLogging.logger {}
@RestController
@RequestMapping( "/api/v1/tabs/" )
@EnableMethodSecurity
class TabsRestController@Autowired constructor(
    val tabService: TabService,
    val authenticationService: AuthenticationService,
    private val userService: UserService
) {
    @Value( "\\${upload.temp.dir}" )
    private val uploadDir: String? = null // Директория для сохранения файлов

    @PostMapping( "/midi2tabs" )
    fun midiToTabs( @RequestParam( "midi" ) midi: MultipartFile, principal:
Principal ): ResponseEntity<TabDto> {
        if ( uploadDir == null ) {
            return ResponseEntity.status( HttpStatus.NO_CONTENT ).build()
        }
        val uploadPath = Paths.get( uploadDir )

        if ( !Files.exists( uploadPath ) ) {
            Files.createDirectories( uploadPath )
        }

        val filePath = uploadPath.resolve( midi.originalFilename.toString() )
        midi.transferTo( filePath )
        val tabs =
PythonUtils.runPythonScript( "src/main/resources/script/MidiToTabs.exe", filePath.toString() )
        filePath.toFile().delete()
        val tabsDto =
TabDto( null, "", tabs, LocalDateTime.now().format( DateTimeFormatter.ofPattern( "d.MM.yyyy HH:mm" ) ), null, null )
        return ResponseEntity.ok( tabsDto )
    }

    @PostMapping( "/audio2midi" )
    fun audioToMidi( @RequestBody noteRequest: NoteRequest, principal:
Principal ): ResponseEntity<TabDto> {
        val midiGenerator = MidiGenerator( 24, 0, 26 )
        noteRequest.notes.forEach { midiGenerator.addNote(
            NoteUtils.noteToMidi( it.strNote ) ?: 0,
            127,
            NoteUtils.secondsToMidiTicks( it.startTime, noteRequest.bpm ),
            NoteUtils.secondsToMidiTicks( it.duration, noteRequest.bpm ) ) }
        val filePath = Paths.get( "temp/${noteRequest.title}.mid" )
        midiGenerator.writeToFile( filePath.toString() )
        val tabs =
PythonUtils.runPythonScript( "src/main/resources/script/MidiToTabs.exe", filePath.toString() )
    }
}

```

```

        filePathToFile().delete()
        val tabsDto =
TabDto(null,noteRequest.title,tabs,LocalDateTime.now().format(DateTimeFormatt
er.ofPattern("dd.MM.yyyy HH:mm")),null,null)
        return ResponseEntity.ok(tabsDto)
    }

    @PostMapping("/save")
    fun save(@RequestBody tabRequest: TabRequest, principal:
Principal):ResponseEntity<TabDto> {
        val author = userService.findByUsername(principal.name) ?: return
ResponseEntity.status(HttpStatus.FORBIDDEN).build()
        val tab =
tabService.create(Tabs(null,tabRequest.title,tabRequest.content,null,author,t
abRequest.isShared))
        ?: return ResponseEntity.status(HttpStatus.FORBIDDEN).build()
        return ResponseEntity.ok(TabMapper.toDto(tab))
    }

    @GetMapping("/my")
    fun getMy(principal: Principal): ResponseEntity<TabsDto> {
        val myTabs = tabService.findByAuthorUsername(principal.name) ?: return
ResponseEntity.status(HttpStatus.FORBIDDEN).build()
        val myTabsDto = TabsDto(myTabs.map { TabMapper.toDto(it)
}.toMutableList())
        return ResponseEntity.ok(myTabsDto)
    }

    @GetMapping("/shared")
    fun getAllShared(
        @RequestParam(value = "offset", defaultValue = "0")@Min(0) offset:
Int,
        @RequestParam(value = "limit", defaultValue = "20")@Min(1) @Max(100)
limit: Int,
    ): ResponseEntity<Page<TabDto>> {

        val tabs = tabService.findAllByisSharedIsTrue(
            PageRequest.of(offset,limit))
        val tabsDto = tabs.map(TabMapper::toDto)

        return ResponseEntity(tabsDto, HttpStatus.OK)
    }

    @DeleteMapping("/delete/id/{id}")
    fun deleteTabById(@PathVariable id: Long,principal: Principal):
ResponseEntity<Any> {
        val tab = tabService.findById(id) ?: return
ResponseEntity(HttpStatus.NOT_FOUND)
        if (principal.name != tab.author.username) {
            return ResponseEntity(HttpStatus.CONFLICT)
        }
        tabService.deleteById(id)
        return ResponseEntity.ok(HttpStatus.OK)
    }

    @PostMapping("/update")
    fun update(@RequestBody tab: RequestTabDto, principal: Principal):
ResponseEntity<TabDto>{
        val existTab = tabService.findById(tab.id) ?: return
ResponseEntity(HttpStatus.NOT_FOUND)
        if (principal.name != existTab.author.username) {
            return ResponseEntity(HttpStatus.CONFLICT)
        }
        val updateTab = tabService.update(tab,existTab) ?: return

```

```

        ResponseEntity(HttpStatus.NOT_FOUND)
        val updateTabDto = TabMapper.toDto(updateTab)
        return ResponseEntity(updateTabDto, HttpStatus.OK)
    }
}

```

UploadRestController.kt

```
package com.diplom.guitartoolsserver.controller
```

```

import org.springframework.beans.factory.annotation.Value
import org.springframework.core.env.Environment
import org.springframework.core.io.FileSystemResource
import org.springframework.http.HttpStatus
import org.springframework.http.MediaType
import org.springframework.http.ResponseEntity
import org.springframework.web.bind.annotation.*
import org.springframework.web.multipart.MultipartFile
import java.io.IOException
import java.nio.file.Files
import java.nio.file.Paths
import java.util.*

@RestController
@RequestMapping( "/api/v1/uploadImage")
class UploadImageRestController {
    @Value("${upload.avatar.dir}")
    private val uploadDir: String? = null // Директория для сохранения файлов

    @Value("${server.address}")
    private val serverAddress: String? = null

    @Value("${server.port}")
    private val serverPort: String? = null

    @PostMapping
    fun uploadImage(@RequestParam("image") image: MultipartFile):
    ResponseEntity<String> {
        try {
            if(uploadDir == null) {
                return
                ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build()
            }
            // 1. Проверка, что файл не пустой
            if (image.isEmpty) {
                return ResponseEntity.badRequest().body("Please select a file
to upload.")
            }

            // 2. Получение имени файла и расширения
            val originalFileName = image.originalFilename
            val fileExtension = originalFileName?.substringAfterLast(".") ?:
            "jpg" // Default to jpg if no extension

            // 3. Генерация уникального имени файла, чтобы избежать
            конфликтов
            val uniqueFileName = UUID.randomUUID().toString() + "." +
            fileExtension

            // 4. Создание пути к файлу
            val uploadPath = Paths.get(uploadDir)
            if (!Files.exists(uploadPath)) {
                Files.createDirectories(uploadPath)
            }
            val filePath = uploadPath.resolve(uniqueFileName)

```

```

        // 5. Сохранение файла на сервере
        image.transferTo(filePath)

        // 6. Возвращение успешного ответа с путем к файлу
        return
    } catch (e: IOException) {
        e.printStackTrace()
        return
    }
    ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Failed to
    upload image: ${e.message}")
    } catch (e: Exception) {
        e.printStackTrace()
        return
    }
    ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("An error
    occurred: ${e.message}")
    }
}

@PostMapping("/many")
fun uploadManyImage(@RequestParam("images") images: List<MultipartFile>):
ResponseEntity<List<String>> {
    try{

        val endPath:MutableList<String> = mutableListOf()
        if(uploadDir == null) {
            return
        }
        ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build()
        if (images.isEmpty()) {
            return ResponseEntity(mutableListOf("Please select a file to
            upload."), HttpStatus.BAD_REQUEST)
        }
        images.forEach { image ->

            // 2. Получение имени файла и расширения
            val originalFileName = image.originalFilename
            val fileExtension = originalFileName?.substringAfterLast(".")
            ?: "jpg" // Default to jpg if no extension

            // 3. Генерация уникального имени файла, чтобы избежать
            конфликтов
            val uniqueFileName = UUID.randomUUID().toString() + "." +
            fileExtension

            // 4. Создание пути к файлу
            val uploadPath = Paths.get(uploadDir)
            if (!Files.exists(uploadPath)) {
                Files.createDirectories(uploadPath)
            }
            val filePath = uploadPath.resolve(uniqueFileName)

            // 5. Сохранение файла на сервере
            image.transferTo(filePath)

            endPath.add("http://$serverAddress:$serverPort/api/v1/uploadImage/images/avat
            ars/$uniqueFileName")
        }
        // 1. Проверка, что файл не пустой

```

```

        // 6. Возвращение успешного ответа с путем к файлу
        return ResponseEntity.ok(endPath)

    } catch (e: IOException) {
        e.printStackTrace()
        return ResponseEntity.mutableListOf("Failed upload
image"), HttpStatus.INTERNAL_SERVER_ERROR)
    } catch (e: Exception) {
        e.printStackTrace()
        return ResponseEntity.mutableListOf("An error occurred:
${e.message}"), HttpStatus.INTERNAL_SERVER_ERROR)
    }
}

@GetMapping("/images/avatars/{filename}")
fun getImage(@PathVariable filename: String):
ResponseEntity<org.springframework.core.io.Resource> {
    try {
        if(uploadDir == null) {
            return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build()
        }
        val filePath = Paths.get(uploadDir).resolve(filename).normalize()
        val resource = FileSystemResource(filePath)

        return if (resource.exists()) {
            ResponseEntity.ok()
                .contentType(MediaType.IMAGE_JPEG) // Важно: Установите
правильный MediaType
                .body(resource)
        } else {
            ResponseEntity.notFound().build()
        }
    } catch (e: Exception) {
        e.printStackTrace()
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build()
    }
}
}

```

UserRestController.kt

```

package com.diplom.guitartoolsserver.controller

import com.diplom.guitartoolsserver.model.Users
import com.diplom.guitartoolsserver.security.AuthenticationService
import com.diplom.guitartoolsserver.service.UserService
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.security.config.annotation.method.configuration.EnableMet
hodSecurity
import org.springframework.web.bind.annotation.*
import java.security.Principal

@RestController
@RequestMapping( "/api/v1/users/")
@EnableMethodSecurity
class UserRestController@Autowired constructor(

```

```

        val userService: UserService,
        val authenticationService: AuthenticationService
    ) {

        @PatchMapping("/{username}/update")
        fun updateUser(@PathVariable username: String, @RequestBody updatedUser:
Users, principal: Principal): ResponseEntity<Users> {
            val userId = userService.findByUsername(username)?.id ?: return
ResponseEntity.status(HttpStatus.NOT_FOUND).build()

            if (principal.name != username) {
                return ResponseEntity.status(HttpStatus.FORBIDDEN).build()
            }

            if (userService.existsByUsernameAndIdNot(updatedUser.username, userId)) {
                return ResponseEntity.status(HttpStatus.CONFLICT).build()
            }

            if (userService.existsByEmailAndIdNot(updatedUser.email.toString(), userId)) {
                return ResponseEntity.status(HttpStatus.CONFLICT).build()
            }
            val resultUser: Users = userService.updateUser(userId, updatedUser)
            return ResponseEntity.ok(resultUser)
        }

        @DeleteMapping("/delete/me")
        fun deleteUser(principal: Principal): ResponseEntity<Any> {
            val userid: Long = userService.findByUsername(principal.name)?.id
                ?: return ResponseEntity.status(HttpStatus.NO_CONTENT).build()
            userService.deleteById(userid)
            return ResponseEntity.ok(HttpStatus.OK)
        }
    }
}

```

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский государственный университет
аэрокосмического приборостроения»

ОТЗЫВ РУКОВОДИТЕЛЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

на тему Разработка приложения для обучения игре на гитаре
с функцией генерации табулатуры

выполненную студентом группы № Z0431

Орловым Владимиром Ивановичем

фамилия, имя, отчество студента

по направлению подготовки / специальности	09.03.04 (код)	Программная инженерия (наименование направления подготовки/специальности)
--	-------------------	--

Актуальность темы работы:

Эффективность и скорость обучения игре на гитаре можно значительно увеличить за счет современных технологий. Орлов В.И. разработал приложение распознавания частоты звука и генерации табулатуры, что упрощает начинающим музыкантам осваивать инструмент.

Цель и задачи работы:

Разработка приложения для обучения игре на гитаре сочетающего структурированные уроки, справочник популярных аккордов и функцию генерации табулатуры.

Общая оценка выполнения поставленной перед студентом задачи, основные достоинства и недостатки работы:

В ходе выполнения квалификационной работы Орлов В.И. провел исследование предметной области, рассмотрел аналоги приложений обучения игры на гитаре, описал методы определения частот основного тона, спроектировал и разработал клиент-серверное приложение обучения игре на гитаре с функцией генерации табулатуры. Клиентская часть реализована как мобильное приложение для устройств под управлением операционной системы Android. Для реализации приложения Орлов В.И. использовал язык программирования Kotlin, фреймворк Spring, систему управления базами данных PostgreSQL, библиотеку цифровой обработки TarsosDSP, библиотеку загрузки и отображения изображений Glide, приложение для отображения аккордов ChordDroid.

Степень самостоятельности и способности к исследовательской работе студента (умение и навыки поиска, обобщения, анализа материала и формулирования выводов):

В ходе работы Орлов В.И. проявил способности к исследовательской работе, умение работы с различными источниками информации, навыки поиска и обобщения материала, необходимого для реализации системы, формулирования выводов.

Проверка текста выпускной квалификационной работы с использованием системы antiplagiat, проводившаяся 08.06.2025, показывает оригинальность содержания на уровне 77.55%.

Текст пояснительной записки оформлен грамотно, изложение последовательное.

Оценка деятельности студента в период подготовки выпускной квалификационной работы (добросовестность, работоспособность, ответственность, аккуратность и т.п.):

Орлов В.И. проявил добросовестность, ответственность и аккуратность при работе над квалификационной работой.

В работе не содержится информация с ограниченным доступом и отсутствуют сведения, представляющие коммерческую ценность.

Общий вывод:

В ходе квалификационной работы Орлов В.И. разработал мобильное приложение для обучения игры на гитаре с функцией генерации табулатуры из аудиофайла, микрофонного входа или файла формата MIDI. Орлов В.И. решил сложную задачу, заслуживает присвоения квалификации бакалавра по направлению 09.03.04.

Руководитель

доц., канд. тех. наук.

должность, уч. степень, звание

 06.06.25

подпись, дата

А.В. Фомин

инициалы, фамилия



Разработка приложения для обучения игре на гитаре с функцией генерации табулатуры

Орлов Владимир Иванович, Z0431

Фомин Александр Владимирович, доцент, канд. техн. наук.

2025



	Yousician	Fender play	Ultimate Guitar	Chord ai	GuitarTuna
Простой интерфейс	✗	✓	✗	✓	✓
Метроном	✓	✓	✗	✗	✓
Обучающие уроки	✓	✓	✗	✗	✓
Социальное взаимодействие	✓✗	✓✗	✓✗	✗	✗
Справочные материалы	✓	✓	✓	✓	✓
Генерация табулатуры	✗	✗	✗	✗	✗

Цель работы - разработка приложения для обучения игре на гитаре, которое должно помочь пользователям систематизировать и улучшить процесс обучения, а также предоставить инструментарий для комфортного освоения музыкального инструмента.

Задачи которые необходимо реализовать:

- Система авторизации и регистрации пользователей
- Личный кабинет
- Редактор уроков
- Генерация табулатуры из аудиофайла и файла формата MIDI
- Сохранение табулатуры в личный кабинет и в PDF файл
- Просмотр и поиск аккордов по названию
- Метроном



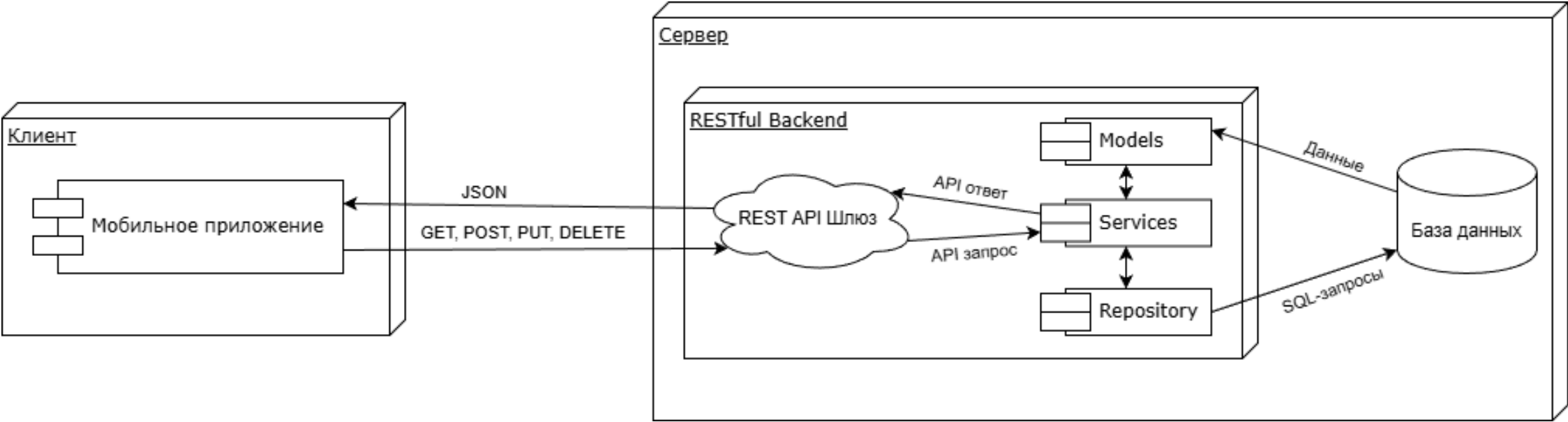
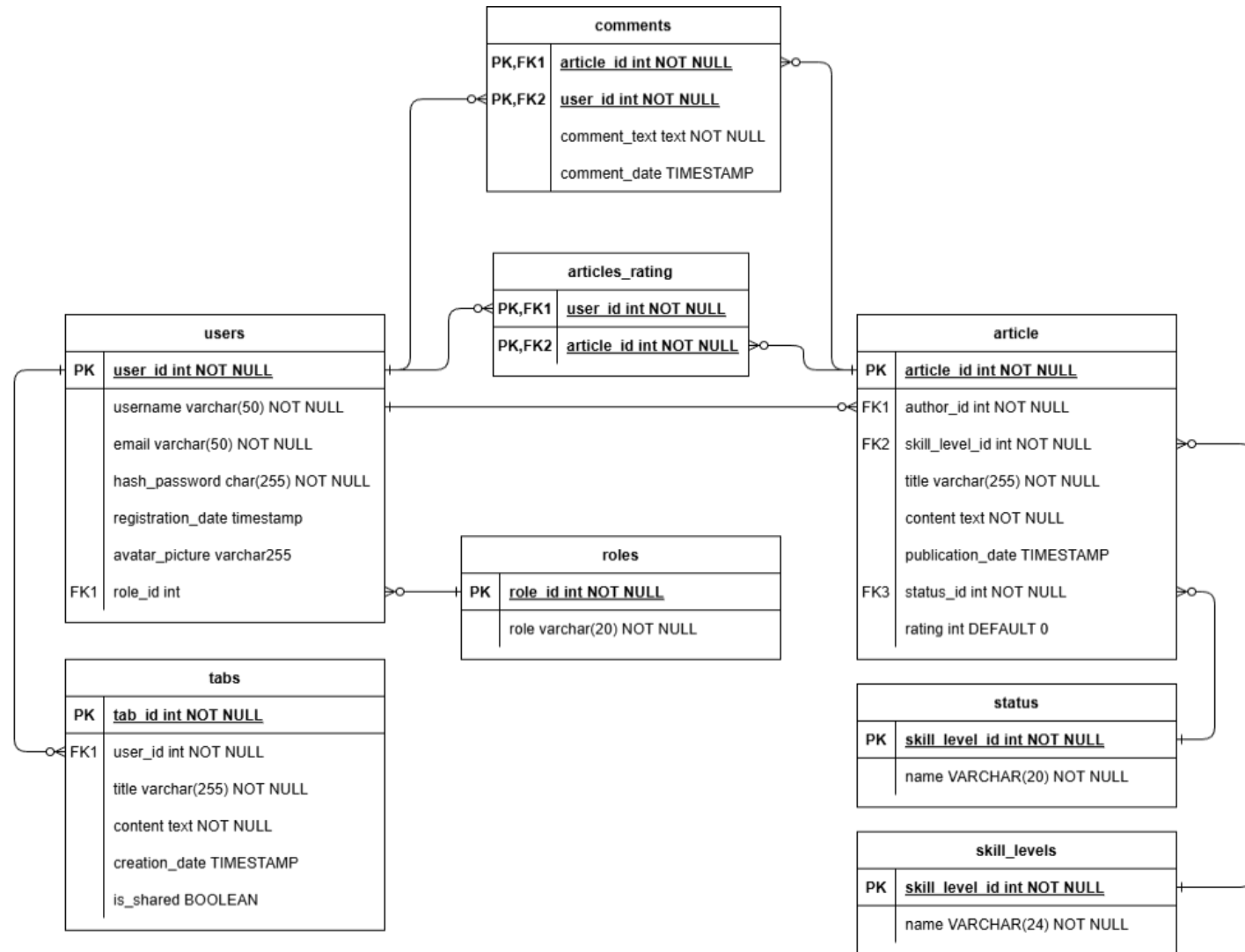
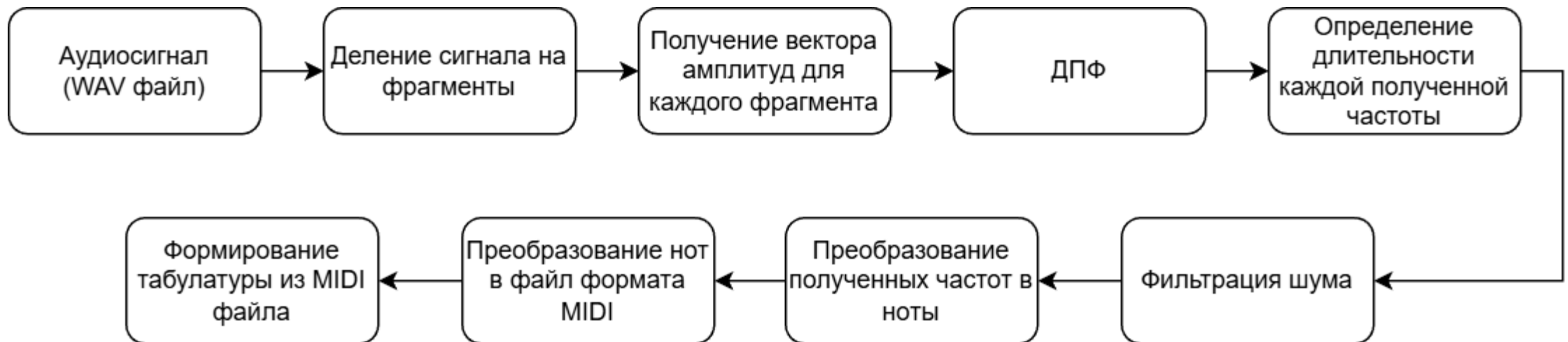
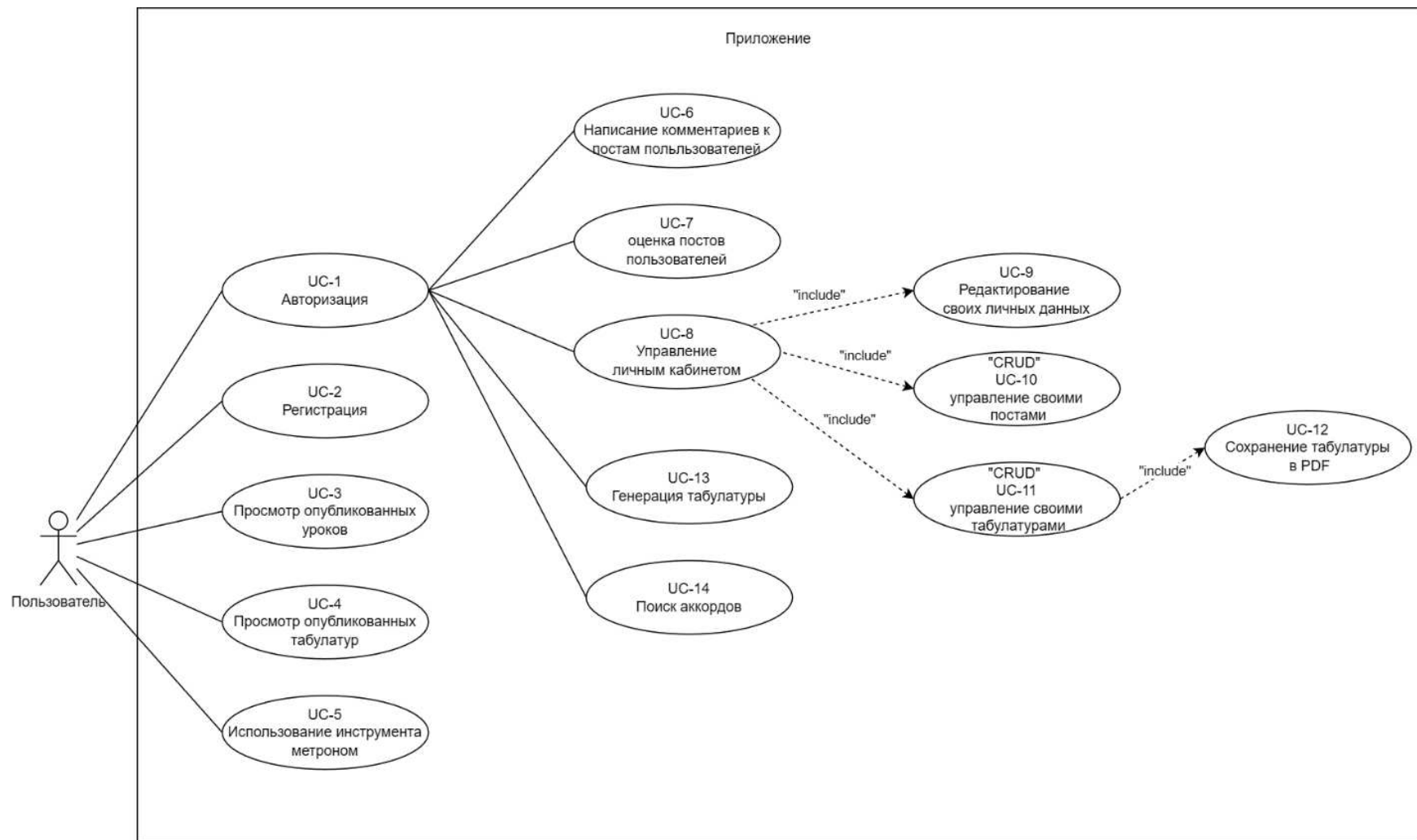
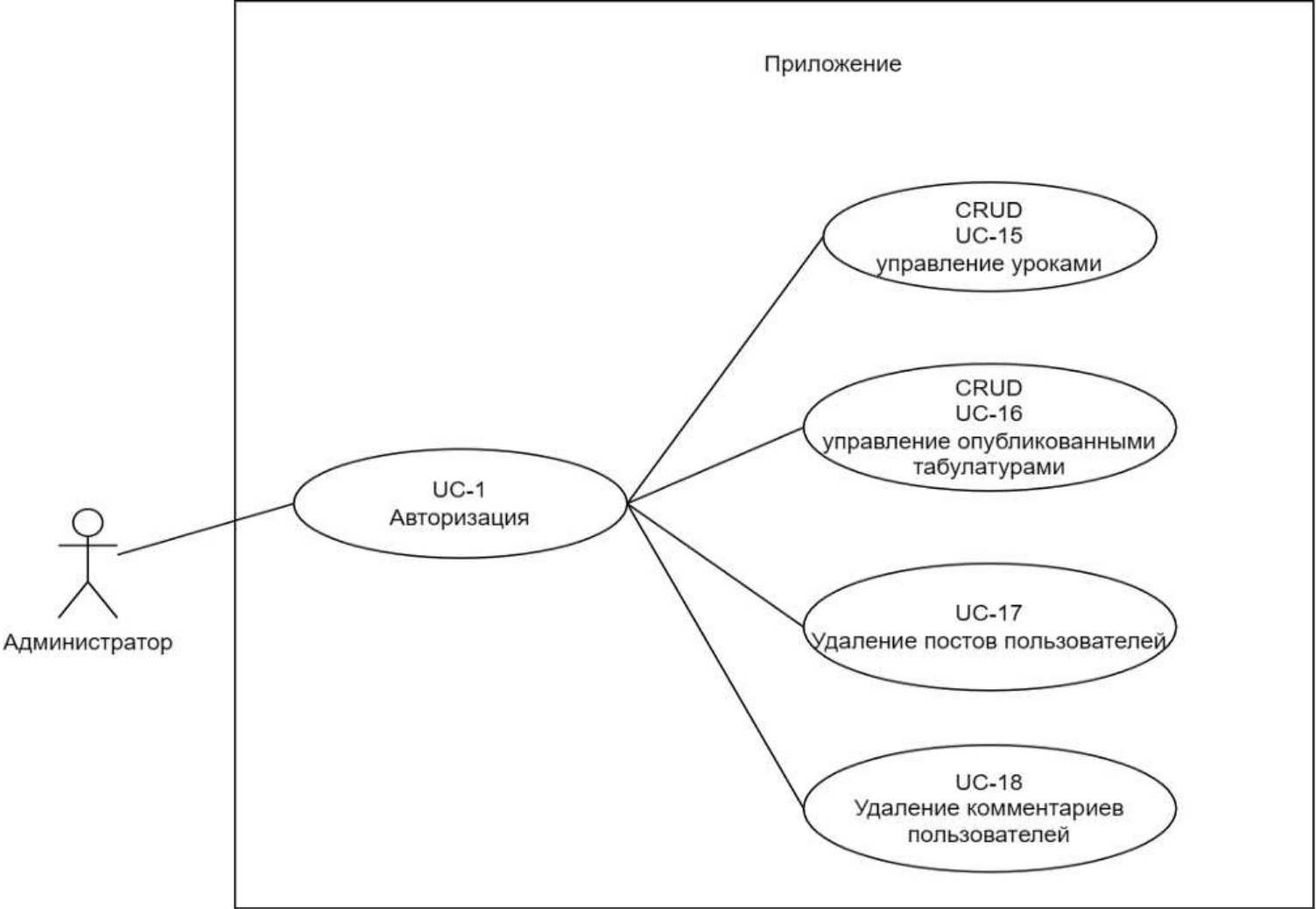


Диаграмма «Сущность-связь» базы данных

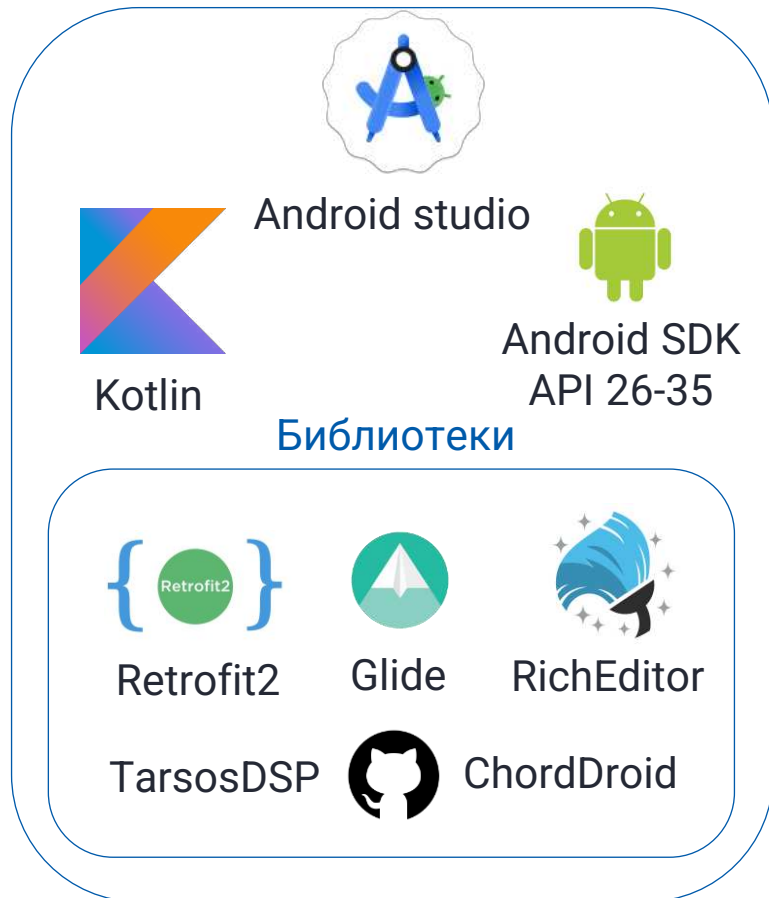




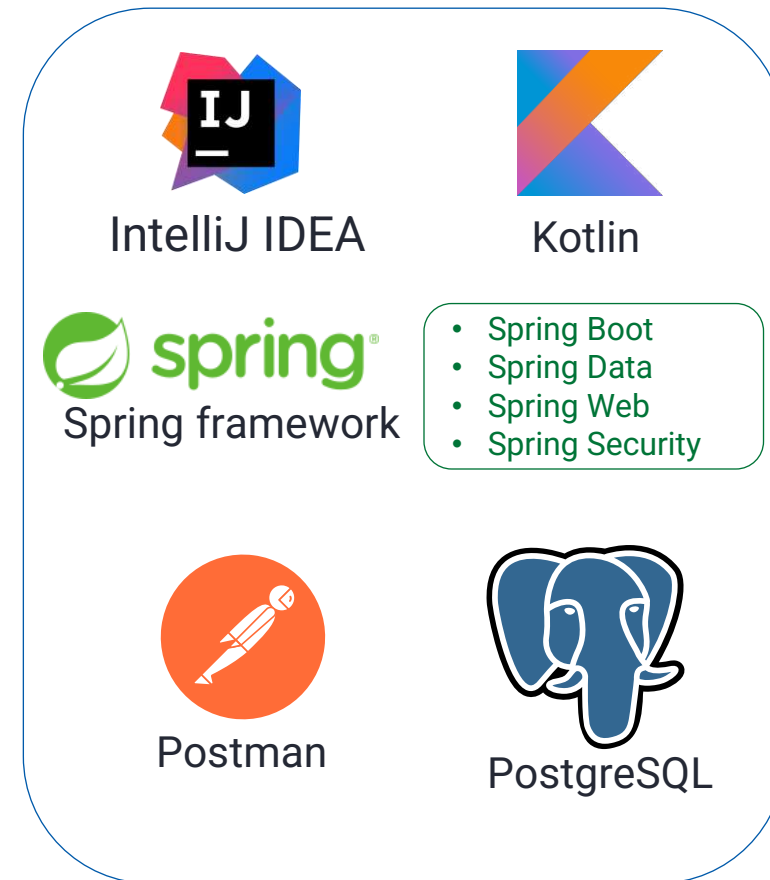




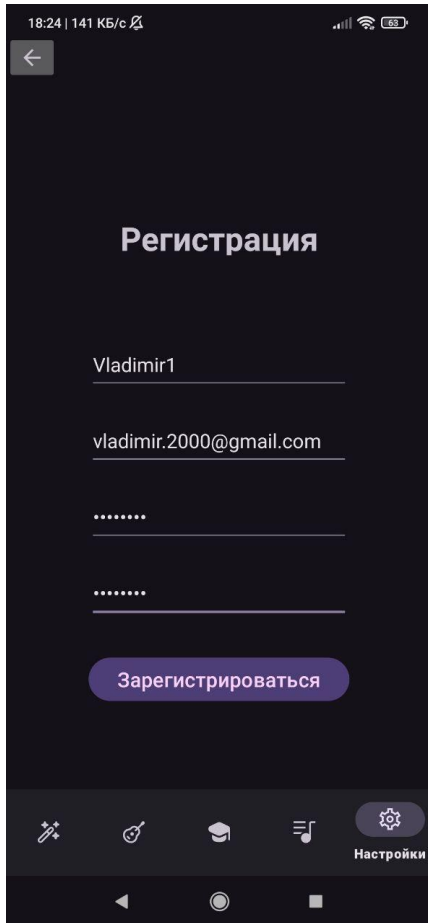
Технологии и модули используемые для клиента



Технологии и модули используемые для сервера



Результаты. Регистрация и авторизация



18:24 | 141 КБ/с

←

Регистрация

Vladimir1

vladimir.2000@gmail.com

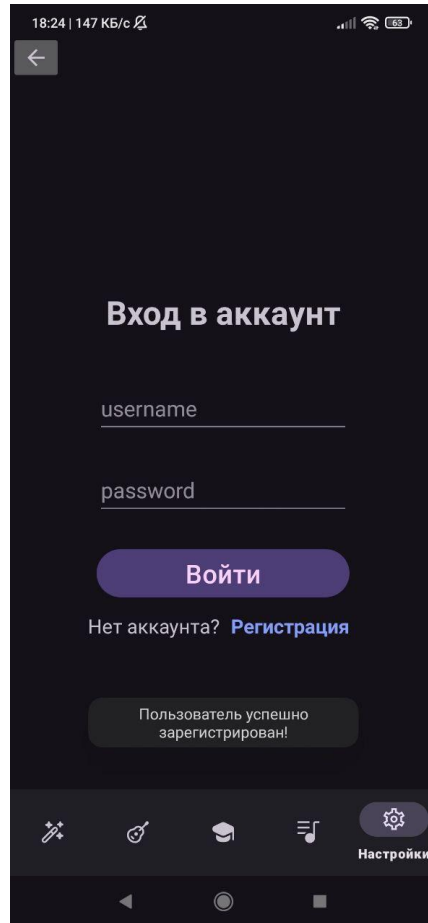
.....

.....

Зарегистрироваться

Настройки

Ввод данных
для регистрации



18:24 | 147 КБ/с

←

Вход в аккаунт

username

password

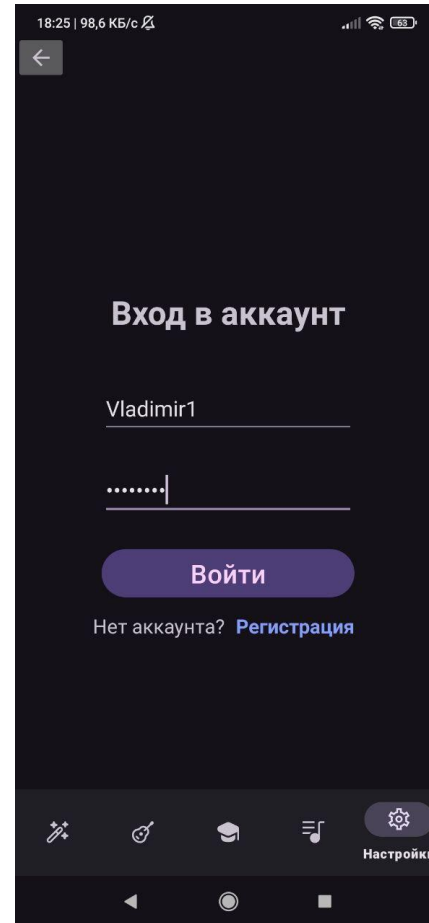
Войти

Нет аккаунта? [Регистрация](#)

Пользователь успешно
зарегистрирован!

Настройки

Подтверждение
регистрации



18:25 | 98,6 КБ/с

←

Вход в аккаунт

Vladimir1

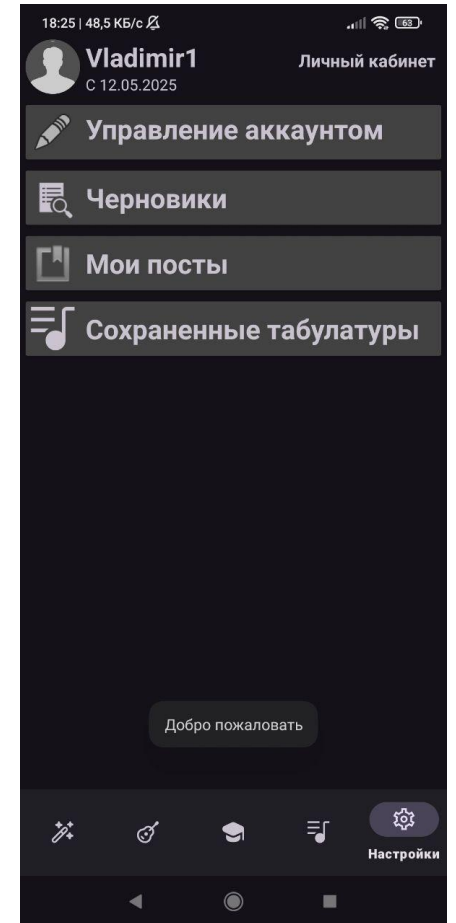
.....

Войти

Нет аккаунта? [Регистрация](#)

Настройки

Авторизация
пользователя



18:25 | 48,5 КБ/с

←

 **Vladimir1** Личный кабинет
С 12.05.2025

 Управление аккаунтом

 Черновики

 Мои посты

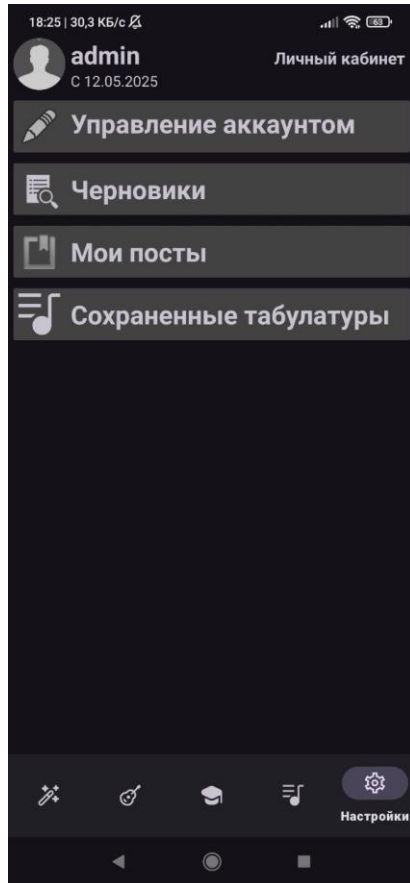
 Сохраненные табулатуры

Добро пожаловать

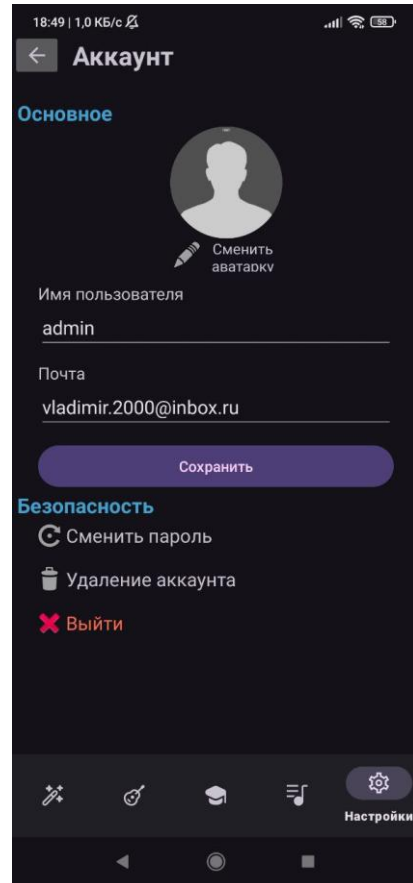
Настройки

Доступ к
личному кабинету

Результаты. Личный кабинет



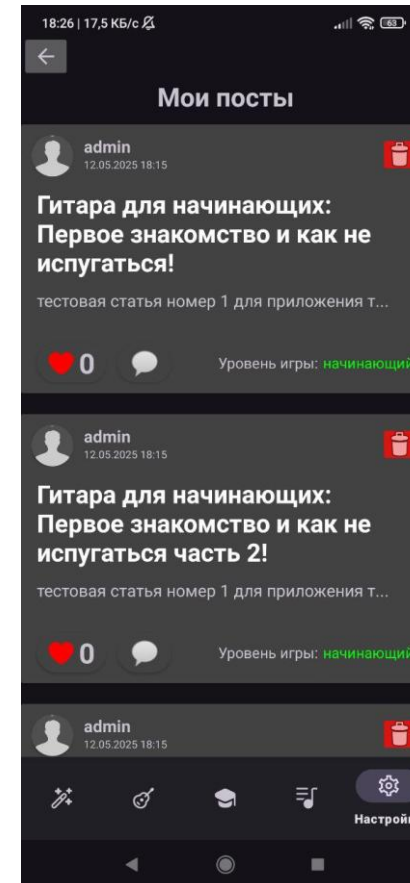
Личный кабинет
пользователя



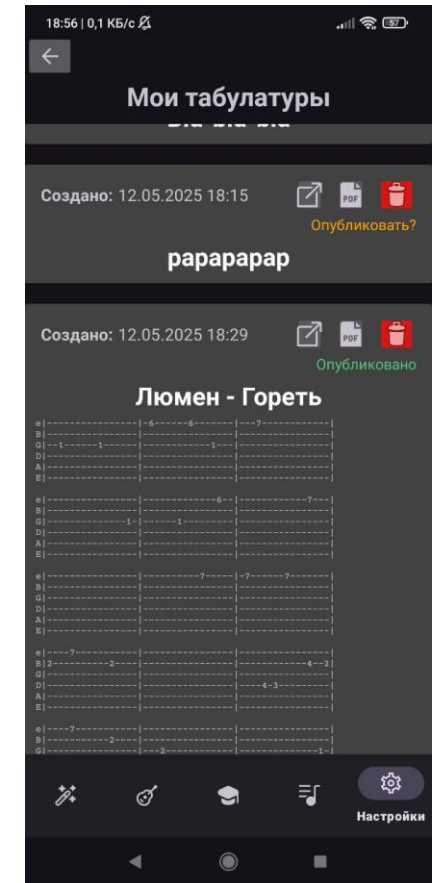
Меню
управление аккаунтом



Меню
сохраненных
черновиков

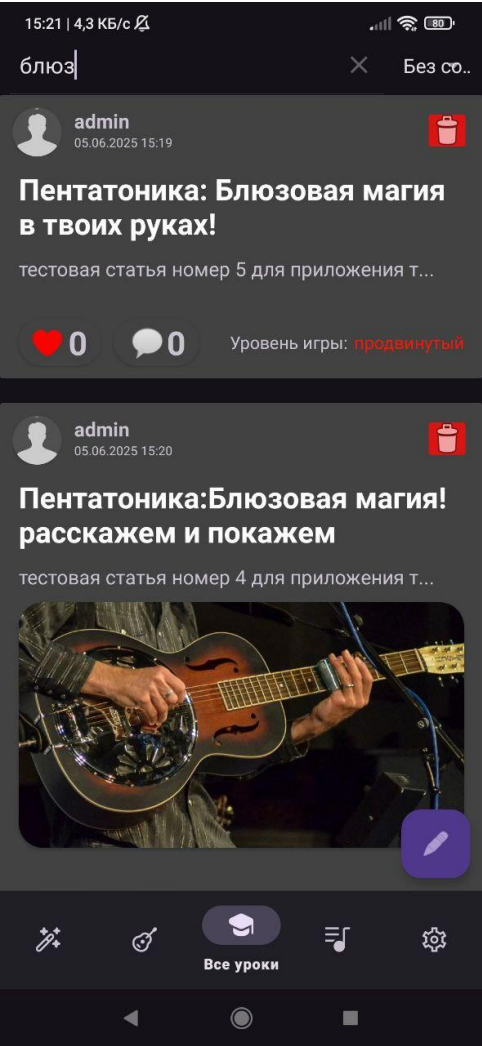


Меню
Мои посты

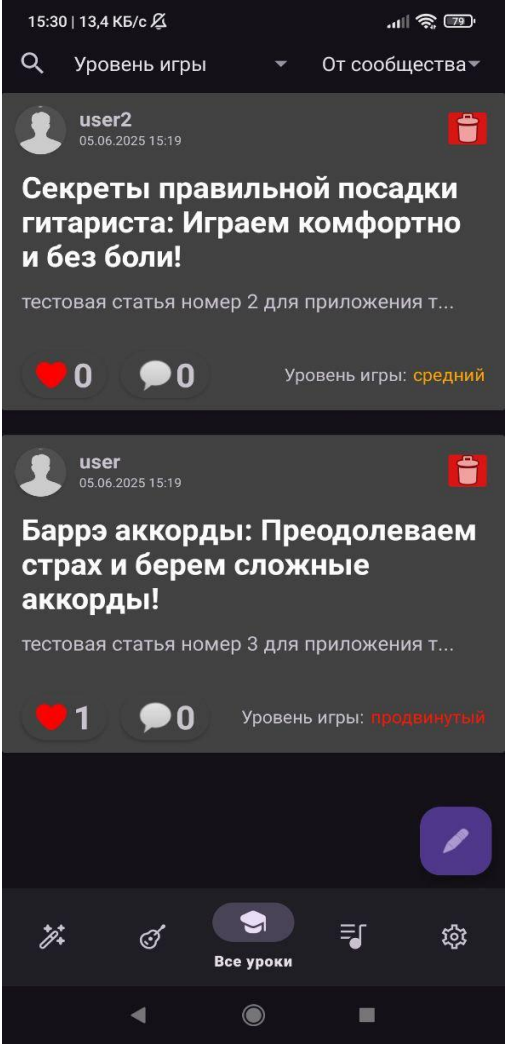


Меню
Мои табулатуры

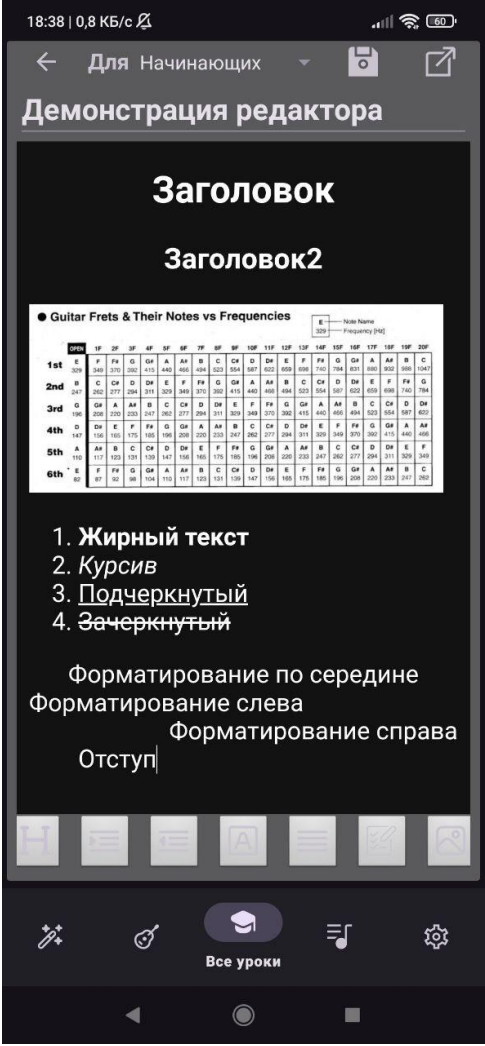
Результаты. Лента уроков и редактор



Поиск по заголовку

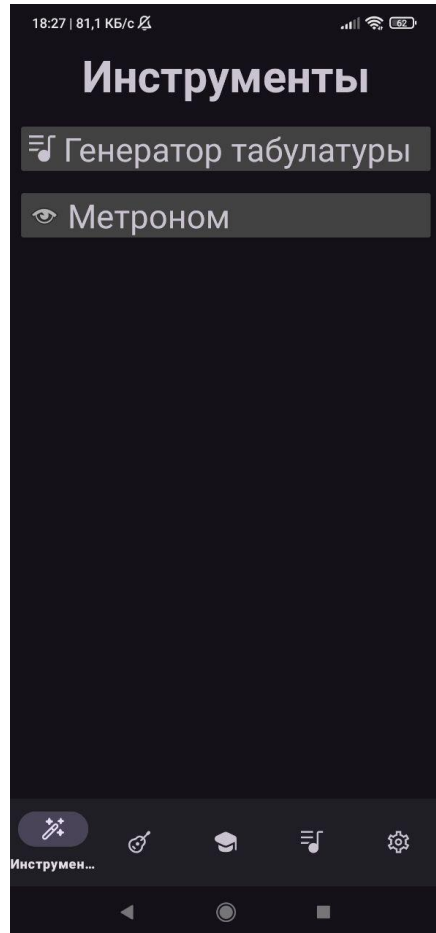


Раздел постов
от сообщества

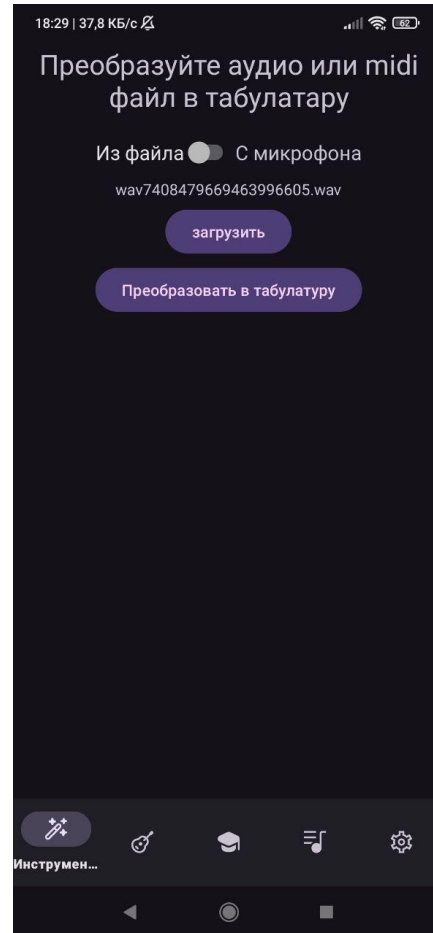


Встроенный редактор

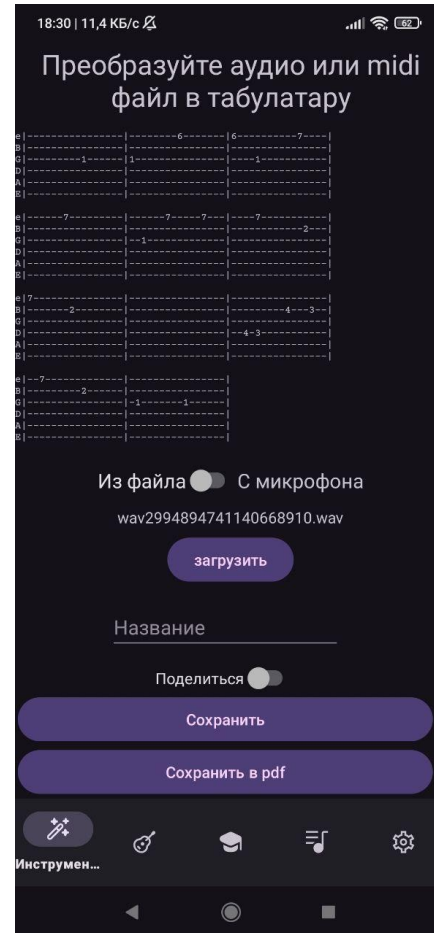
Результаты. Генерация табулатуры



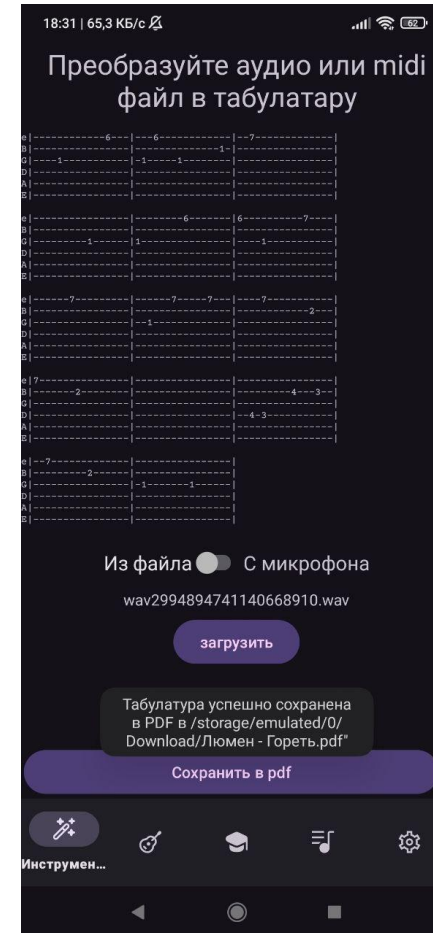
Меню выбора инструментов



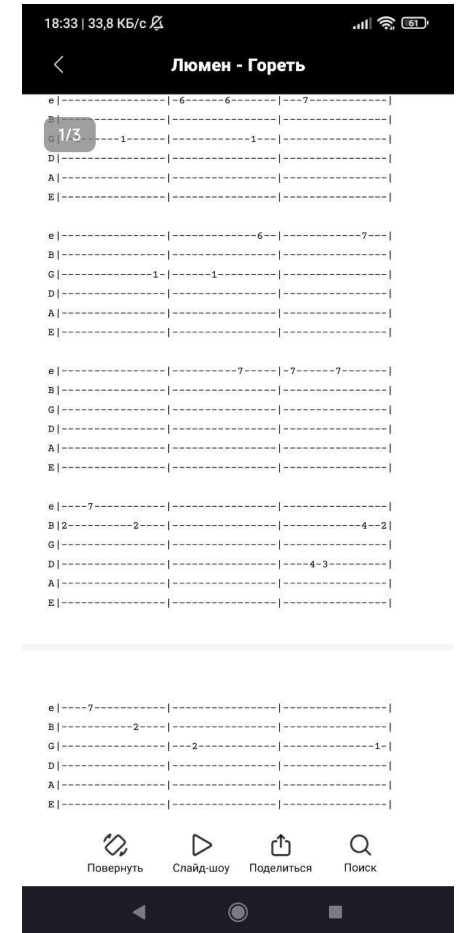
Выбор файла



Меню выбора после генерации



Сохранение табулатуры



Открытие сохраненного pdf файла



Демонстрация поиска
аккорда Am



Аккорд в другой позиции

В рамках выпускной квалификационной работы создано приложение для обучения игре на гитаре. Оно позволяет создавать и просматривать уроки, предоставляет справочник аккордов, а также включает систему авторизации, личный кабинет, редактор уроков, сортировку и поиск, генерацию табулатуры из аудио-, MIDI-файлов и микрофона с сохранением в PDF и возможностью делиться ими с пользователями приложения. Также реализован метроном.

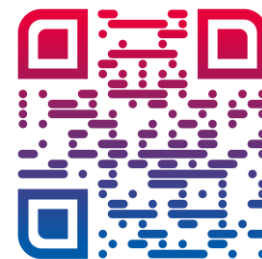
Дальнейшее расширение функционала приложения:

- Улучшение алгоритма выделения нот с дополнительным использованием нейронных сетей для повышения качества распознавания.
- Создание удобного редактора табулатуры с прослушиванием результата.
- Расширение инструментов с добавлением распознавания аккордов и тюнера.



ГУАП

<https://guap.ru>



Спасибо за внимание!